

Kit de Componentes Web Reutilizables

57 Funcionalidades Prácticas

Guía completa de implementación, código fuente real, explicación técnica, tablas de elementos y pasos de reciclaje. Esta versión fue regenerada para evitar código incompleto: cada bloque CSS y JavaScript fue validado y se incluye el código fuente completo del proyecto como anexo.

Campo	Detalle
Tecnologías	HTML, CSS, JavaScript modular, JavaScript normal y Bootstrap cuando aplica
Fuente base	Repositorio material-reutilizable, archivos index.html, styles.css, funciones.js, script.js y script-normal.js
Validaciones	57 funcionalidades, bloques CSS balanceados, snippets JS balanceados, anexos completos y sin placeholders genéricos como código principal

Tabla de contenidos

#	Funcionalidad	Categoría
1	Navbar fija glass	Navegación e interfaz base
2	Sidebar ocultable	Navegación e interfaz base
3	Overlay para cerrar menú lateral	Navegación e interfaz base
4	Submenús desplegables dentro del sidebar	Navegación e interfaz base
5	Cambio de secciones del body con data-section	Navegación e interfaz base
6	Link activo en navbar y sidebar	Navegación e interfaz base
7	Dropdown superior	Navegación e interfaz base
8	Tema claro/oscuro	Estado visual y experiencia de usuario
9	Guardar tema en localStorage	Estado visual y experiencia de usuario
10	Barra de progreso de scroll	Estado visual y experiencia de usuario
11	Botón volver arriba	Estado visual y experiencia de usuario
12	Animación reveal on scroll	Estado visual y experiencia de usuario
13	Contador animado	Estado visual y experiencia de usuario
14	Login demo	Formularios, entradas y almacenamiento
15	Logout demo	Formularios, entradas y almacenamiento
16	Recordar usuario	Formularios, entradas y almacenamiento
17	Mostrar/ocultar contraseña	Formularios, entradas y almacenamiento
18	Validación de formulario	Formularios, entradas y almacenamiento
19	Mensajes de error por campo	Formularios, entradas y almacenamiento
20	Contador de caracteres	Formularios, entradas y almacenamiento
21	Range slider con valor visible	Formularios, entradas y almacenamiento
22	Información de archivo seleccionado	Formularios, entradas y almacenamiento
23	Autoguardado de borrador	Formularios, entradas y almacenamiento
24	Stepper de pasos	Formularios, entradas y almacenamiento
25	Skeleton loader	Formularios, entradas y almacenamiento
26	Select personalizado editable	Componentes interactivos reutilizables
27	Tabs	Componentes interactivos reutilizables
28	Acordeón	Componentes interactivos reutilizables
29	Modal	Componentes interactivos reutilizables
30	Toast notifications	Componentes interactivos reutilizables
31	Carrusel	Componentes interactivos reutilizables
32	Tooltip	Componentes interactivos reutilizables
33	Badge de notificación	Componentes interactivos reutilizables

34	Copiar al portapapeles	Componentes interactivos reutilizables
35	Cards dinámicas desde array	Datos, búsqueda, tablas y almacenamiento
36	Búsqueda con debounce	Datos, búsqueda, tablas y almacenamiento
37	Filtros con chips	Datos, búsqueda, tablas y almacenamiento
38	Ordenamiento de cards	Datos, búsqueda, tablas y almacenamiento
39	Paginación de cards	Datos, búsqueda, tablas y almacenamiento
40	Tabla ordenable	Datos, búsqueda, tablas y almacenamiento
41	Filtro de tabla	Datos, búsqueda, tablas y almacenamiento
42	Notas con localStorage	Datos, búsqueda, tablas y almacenamiento
43	Exportar notas a JSON	Datos, búsqueda, tablas y almacenamiento
44	Lista de tareas	Utilidades avanzadas y herramientas DOM
45	Drag and drop en tareas	Utilidades avanzadas y herramientas DOM
46	Borrar tareas	Utilidades avanzadas y herramientas DOM
47	Conversor con botones C/F/K	Utilidades avanzadas y herramientas DOM
48	Cambio de fondo según unidad	Utilidades avanzadas y herramientas DOM
49	Cálculo dinámico de temperatura	Utilidades avanzadas y herramientas DOM
50	Imprimir página	Utilidades avanzadas y herramientas DOM
51	Reset visual de demo	Utilidades avanzadas y herramientas DOM
52	Helpers reutilizables para DOM, storage y debounce	Utilidades avanzadas y herramientas DOM
53	Tarjeta de presentación Bootstrap como actividad guiada	Actividades guiadas y componentes finales
54	Carrito de compras DOM como actividad guiada	Actividades guiadas y componentes finales
55	Footer reutilizable como pie de página profesional	Actividades guiadas y componentes finales
56	Tarjetas de precios Bootstrap como actividad guiada completa	Actividades guiadas y componentes finales
57	Barra de desplazamiento personalizada	Actividades guiadas y componentes finales

Introducción y regla de uso

Este kit es un banco de componentes. Cada funcionalidad puede reciclarse en otro proyecto copiando su HTML, su CSS asociado y la clase o función JavaScript indicada.

Regla principal: usa una sola versión de JavaScript por página. Si usas módulos, carga `script.js` con `type="module"`. Si no usas módulos, carga `script-normal.js`. No cargues ambas versiones al tiempo.

Archivo	Uso	Precaución
index.html	Estructura y elementos visibles.	No romper ids ni data-* usados por JS.
styles.css	Diseño completo, variables, responsive y estados.	Si renombras clases, actualiza HTML y JS.
funciones.js	Versión modular con export.	Debe importarse desde script.js.
script.js	Arranque modular.	Requiere <code>type="module"</code> .
script-normal.js	Versión tradicional sin módulos.	No se mezcla con la modular.

Nota de integridad: además del CSS por funcionalidad, el anexo final incluye el archivo `styles.css` completo para que ningún estudiante quede con código cortado.

Base obligatoria del proyecto

Estos archivos son la base real del repo. Si una funcionalidad usa clases globales como `glass-panel`, `primary-button`, `muted` o variables CSS, esas reglas están completas aquí y en el anexo.

script.js real del proyecto

```
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

Helpers base de funciones.js

```
export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}
```

1. Navbar fija glass

Navegación e interfaz base

ID 1

Descripción

Este bloque organiza la navegación principal, el cambio de secciones y los estados visuales de la interfaz. Es fundamental para páginas de una sola vista, dashboards o portafolios.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	glass-navbar, brand, navbar-links, nav-link, navbar-actions, icon-button	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<header class="glass-navbar" id="topbar">
  <a href="#inicio" class="brand" data-section="inicio" aria-label="Ir al inicio">
    <span class="brand__mark">CW</span>
    <span class="brand__text">Componentes Web</span>
```

```

</a>

<nav class="navbar-links" aria-label="Navegacion principal">
  <button class="nav-link is-active" data-section="inicio">Inicio</button>
  <button class="nav-link" data-section="componentes">Componentes</button>
  <button class="nav-link" data-section="formularios">Formularios</button>
  <button class="nav-link" data-section="datos">Datos</button>
  <button class="nav-link" data-section="utilidades">Utilidades</button>

  <div class="dropdown" data-dropdown>
    <button class="nav-link dropdown__button" data-dropdown-button type="button">Mas ▼</button>
    <div class="dropdown__menu" data-dropdown-menu>
      <button data-section="login">Login demo</button>
      <button data-section="storage">Storage</button>
      <button data-section="guia">Guia</button>
    </div>
  </div>
</nav>

<div class="navbar-actions">
  <button class="icon-button" id="themeToggle" type="button" aria-label="Cambiar tema"><</button>
  <button class="icon-button" id="openSidebar" type="button" aria-label="Abrir menu lateral">≡</
button>
</div>
</header>

```

styles.css - CSS asociado validado

```

.glass-navbar {
  position: fixed;
  top: 16px;
  left: 50%;
  transform: translateX(-50%);
  z-index: 1000;
  width: min(1120px, calc(100% - 32px));
  min-height: var(--navbar-height);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 12px 16px;
  background: var(--glass);
  border: 1px solid var(--border);
  border-radius: 999px;
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  box-shadow: var(--shadow);
}

.brand {
  display: inline-flex;
  align-items: center;
  gap: 10px;
  color: var(--text);
  text-decoration: none;
  font-weight: 800;
}

.brand__mark {
  width: 42px;
  height: 42px;
  display: grid;
  place-items: center;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--primary), var(--secondary));
}

```

```

    color: #fff;
}

.navbar-links,
.navbar-actions,
.hero-actions,
.badge-row,
.carousel__actions,
.utilities-row,
.unit-buttons,
.chips,
.pagination {
    display: flex;
    align-items: center;
    gap: 10px;
    flex-wrap: wrap;
}

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
    border: 1px solid var(--border);
    color: var(--text);
    background: rgba(255, 255, 255, 0.1);
    border-radius: 999px;
    padding: 10px 14px;
    transition: transform var(--transition), background var(--transition), border-color var(--transition), box-
shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
    background: var(--glass-strong);
    transform: translateY(-1px);
}

.nav-link.is-active,
.side-link.is-active,
.chip.is-active,
.pagination button.is-active {
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    color: #ffffff;
    border-color: transparent;
}

.icon-button {
    width: 44px;
    height: 44px;
    display: inline-grid;
    place-items: center;

```

```

padding: 0;
font-size: 20px;
}

@media (max-width: 900px) {
  .navbar-links {
    display: none;
  }

  .stats-grid,
  .component-grid,
  .two-column,
  .card-grid,
  .feature-grid,
  .data-toolbar {
    grid-template-columns: 1fr;
  }

  h1 {
    font-size: 36px;
  }
}

@media (max-width: 520px) {
  .page-shell {
    width: min(100% - 24px, 1120px);
    padding-top: 104px;
  }

  .glass-navbar {
    width: calc(100% - 20px);
    top: 10px;
    border-radius: 22px;
  }

  .brand__text {
    display: none;
  }

  .glass-panel {
    padding: 18px;
  }
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class RouterSections {
  constructor(toast) {
    this.sections = $$('[data-view-section]');
    this.buttons = $$('[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.showSection(button.dataset.section));
    });
  }

  showSection(sectionId) {

```

```

const target = document.getElementById(sectionId);
if (!target) return;

this.sections.forEach((section) => section.classList.remove("is-active"));
target.classList.add("is-active");

this.buttons.forEach((button) => {
    button.classList.toggle("is-active", button.dataset.section === sectionId);
});

window.scrollTo({ top: 0, behavior: "smooth" });
this.toast?.show(`Seccion activa: ${sectionId}`);
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class RouterSections {
    constructor(toast) {
        this.sections = $$('.[data-view-section]');
        this.buttons = $$('.[data-section]');
        this.toast = toast;
        this.init();
    }

    init() {
        this.buttons.forEach((button) => {
            button.addEventListener("click", () => this.showSection(button.dataset.section));
        });
    }

    showSection(sectionId) {
        const target = document.getElementById(sectionId);
        if (!target) return;

        this.sections.forEach((section) => section.classList.remove("is-active"));
        target.classList.add("is-active");

        this.buttons.forEach((button) => {
            button.classList.toggle("is-active", button.dataset.section === sectionId);
        });

        window.scrollTo({ top: 0, behavior: "smooth" });
        this.toast?.show(`Seccion activa: ${sectionId}`);
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.

2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

2. Sidebar ocultable

Navegación e interfaz base

ID 2

Descripción

Este bloque organiza la navegación principal, el cambio de secciones y los estados visuales de la interfaz. Es fundamental para páginas de una sola vista, dashboards o portafolios.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	sidebar, overlay, openSidebar, closeSidebar	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<aside class="sidebar" id="sidebar" aria-label="Menu lateral">
  <div class="sidebar_header">
    <strong>Menu lateral</strong>
    <button class="icon-button" id="closeSidebar" type="button" aria-label="Cerrar menu lateral"></button>
  </div>
```

```

<div class="sidebar__content">
  <button class="side-link is-active" data-section="inicio">Panel principal</button>

  <div class="side-group">
    <button class="side-group__button" type="button">Interfaz ▼</button>
    <div class="side-group__content">
      <button data-section="componentes">Componentes UI</button>
      <button data-section="formularios">Formularios</button>
      <button data-section="utilidades">Conversor dinamico</button>
    </div>
  </div>

  <div class="side-group">
    <button class="side-group__button" type="button">Datos y estado ▼</button>
    <div class="side-group__content">
      <button data-section="datos">Catalogos y tablas</button>
      <button data-section="storage">LocalStorage</button>
      <button data-section="login">Login demo</button>
    </div>
  </div>

  <button class="side-link" data-section="guia">Guia de implementacion</button>
</div>
</aside>

<div class="overlay" id="overlay"></div>

```

styles.css - CSS asociado validado

```

.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1500;
  width: min(340px, 86vw);
  height: 100vh;
  padding: 20px;
  background: rgba(15, 23, 42, 0.9);
  color: #ffffff;
  border-right: 1px solid rgba(255, 255, 255, 0.18);
  backdrop-filter: blur(20px);
  transform: translateX(-105%);
  transition: transform var(--transition);
}

.sidebar.is-open {
  transform: translateX(0);
}

.sidebar__header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 20px;
}

.sidebar__content,
.side-group__content {
  display: grid;
  gap: 10px;
}

.overlay {
  position: fixed;
  inset: 0;
}

```

```

    z-index: 1400;
    background: rgba(0, 0, 0, 0.45);
    opacity: 0;
    pointer-events: none;
    transition: opacity var(--transition);
  }

  .overlay.is-active {
    opacity: 1;
    pointer-events: auto;
  }

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class SidebarMenu {
  constructor() {
    this.sidebar = $("#sidebar");
    this.overlay = $("#overlay");
    this.openButton = $("#openSidebar");
    this.closeButton = $("#closeSidebar");
    this.groupButtons = $(".side-group__button");
    this.sectionButtons = $("#sidebar [data-section]");
    this.init();
  }

  init() {
    this.openButton.addEventListener("click", () => this.open());
    this.closeButton.addEventListener("click", () => this.close());
    this.overlay.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
    this.groupButtons.forEach((button) => {
      button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
    });
    this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
  }

  open() {
    this.sidebar.classList.add("is-open");
    this.overlay.classList.add("is-active");
  }

  close() {
    this.sidebar.classList.remove("is-open");
    this.overlay.classList.remove("is-active");
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class SidebarMenu {
  constructor() {
    this.sidebar = $("#sidebar");
    this.overlay = $("#overlay");
    this.openButton = $("#openSidebar");
    this.closeButton = $("#closeSidebar");
    this.groupButtons = $(".side-group__button");
    this.sectionButtons = $$("#sidebar [data-section]");
    this.init();
  }

  init() {
    this.openButton.addEventListener("click", () => this.open());
    this.closeButton.addEventListener("click", () => this.close());
    this.overlay.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
    this.groupButtons.forEach((button) => {
      button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
    });
    this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
  }

  open() {
    this.sidebar.classList.add("is-open");
    this.overlay.classList.add("is-active");
  }

  close() {
    this.sidebar.classList.remove("is-open");
    this.overlay.classList.remove("is-active");
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

3. Overlay para cerrar menú lateral

Navegación e interfaz base

ID 3

Descripción

Este bloque organiza la navegación principal, el cambio de secciones y los estados visuales de la interfaz. Es fundamental para páginas de una sola vista, dashboards o portafolios.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	overlay, sidebar	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="overlay" id="overlay"></div>
<button class="icon-button" id="openSidebar" type="button" aria-label="Abrir menú lateral">☰</button>
```

styles.css - CSS asociado validado

```
.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1500;
  width: min(340px, 86vw);
  height: 100vh;
  padding: 20px;
  background: rgba(15, 23, 42, 0.9);
  color: #ffffff;
  border-right: 1px solid rgba(255, 255, 255, 0.18);
  backdrop-filter: blur(20px);
  transform: translateX(-105%);
  transition: transform var(--transition);
}

.sidebar.is-open {
  transform: translateX(0);
}

.sidebar__header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 20px;
}

.sidebar__content,
.side-group__content {
  display: grid;
  gap: 10px;
}

.overlay {
  position: fixed;
  inset: 0;
  z-index: 1400;
  background: rgba(0, 0, 0, 0.45);
  opacity: 0;
  pointer-events: none;
  transition: opacity var(--transition);
}

.overlay.is-active {
  opacity: 1;
  pointer-events: auto;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class SidebarMenu {
  constructor() {
    this.sidebar = $("#sidebar");
    this.overlay = $("#overlay");
    this.openButton = $("#openSidebar");
    this.closeButton = $("#closeSidebar");
    this.groupButtons = $(".side-group__button");
    this.sectionButtons = $("#sidebar [data-section]");
    this.init();
  }
}
```

```

}

init() {
  this.openButton.addEventListener("click", () => this.open());
  this.closeButton.addEventListener("click", () => this.close());
  this.overlay.addEventListener("click", () => this.close());
  document.addEventListener("keydown", (event) => {
    if (event.key === "Escape") this.close();
  });
  this.groupButtons.forEach((button) => {
    button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
  });
  this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
}

open() {
  this.sidebar.classList.add("is-open");
  this.overlay.classList.add("is-active");
}

close() {
  this.sidebar.classList.remove("is-open");
  this.overlay.classList.remove("is-active");
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class SidebarMenu {
  constructor() {
    this.sidebar = $("#sidebar");
    this.overlay = $("#overlay");
    this.openButton = $("#openSidebar");
    this.closeButton = $("#closeSidebar");
    this.groupButtons = $(".side-group_button");
    this.sectionButtons = $(".#sidebar [data-section]");
    this.init();
  }

  init() {
    this.openButton.addEventListener("click", () => this.open());
    this.closeButton.addEventListener("click", () => this.close());
    this.overlay.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
    this.groupButtons.forEach((button) => {
      button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
    });
    this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
  }
}

```

```
open() {
  this.sidebar.classList.add("is-open");
  this.overlay.classList.add("is-active");
}

close() {
  this.sidebar.classList.remove("is-open");
  this.overlay.classList.remove("is-active");
}
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

4. Submenús despleguables dentro del sidebar

Navegación e interfaz base

ID 4

Descripción

Este bloque organiza la navegación principal, el cambio de secciones y los estados visuales de la interfaz. Es fundamental para páginas de una sola vista, dashboards o portafolios.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	side-group, side-group__button, side-group__content	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="side-group">
  <button class="side-group__button" type="button">Interfaz ▼</button>
  <div class="side-group__content">
    <button data-section="componentes">Componentes UI</button>
    <button data-section="formularios">Formularios</button>
```

```

        <button data-section="utilidades">Convertor dinamico</button>
      </div>
    </div>

    <div class="side-group">
      <button class="side-group__button" type="button">Datos y estado ▼</button>
      <div class="side-group__content">
        <button data-section="datos">Catalogos y tablas</button>
        <button data-section="storage">LocalStorage</button>
        <button data-section="login">Login demo</button>
      </div>
    </div>

```

styles.css - CSS asociado validado

```

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-
shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
  background: var(--glass-strong);
  transform: translateY(-1px);
}

.sidebar__content,
.side-group__content {
  display: grid;
  gap: 10px;
}

.side-link,
.side-group__button,
.side-group__content button {
  width: 100%;
  text-align: left;
}

.side-group__content {
  max-height: 0;
  overflow: hidden;
}

```

```

    transition: max-height var(--transition);
    padding-left: 12px;
  }

  .side-group.is-open .side-group__content {
    max-height: 220px;
  }

  .side-group__content button {
    border: 0;
    border-radius: var(--radius-sm);
    padding: 10px 12px;
    color: #ffffff;
    background: rgba(255, 255, 255, 0.08);
  }

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class SidebarMenu {
  constructor() {
    this.sidebar = $("#sidebar");
    this.overlay = $("#overlay");
    this.openButton = $("#openSidebar");
    this.closeButton = $("#closeSidebar");
    this.groupButtons = $(".side-group__button");
    this.sectionButtons = $("#sidebar [data-section]");
    this.init();
  }

  init() {
    this.openButton.addEventListener("click", () => this.open());
    this.closeButton.addEventListener("click", () => this.close());
    this.overlay.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
    this.groupButtons.forEach((button) => {
      button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
    });
    this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
  }

  open() {
    this.sidebar.classList.add("is-open");
    this.overlay.classList.add("is-active");
  }

  close() {
    this.sidebar.classList.remove("is-open");
    this.overlay.classList.remove("is-active");
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {

```

```
initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class SidebarMenu {
  constructor() {
    this.sidebar = $("#sidebar");
    this.overlay = $("#overlay");
    this.openButton = $("#openSidebar");
    this.closeButton = $("#closeSidebar");
    this.groupButtons = $(".side-group__button");
    this.sectionButtons = $("#sidebar [data-section]");
    this.init();
  }

  init() {
    this.openButton.addEventListener("click", () => this.open());
    this.closeButton.addEventListener("click", () => this.close());
    this.overlay.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
    this.groupButtons.forEach((button) => {
      button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
    });
    this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
  }

  open() {
    this.sidebar.classList.add("is-open");
    this.overlay.classList.add("is-active");
  }

  close() {
    this.sidebar.classList.remove("is-open");
    this.overlay.classList.remove("is-active");
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

5. Cambio de secciones del body con data-section

Navegación e interfaz base

ID 5

Descripción

Este bloque organiza la navegación principal, el cambio de secciones y los estados visuales de la interfaz. Es fundamental para páginas de una sola vista, dashboards o portafolios.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	view-section, data-section, page-shell	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="nav-link" data-section="componentes">Componentes</button>
<section class="view-section" id="componentes" data-view-section>...</section>
```

styles.css - CSS asociado validado

```
.page-shell {
  width: min(1120px, calc(100% - 32px));
  margin: 0 auto;
  padding: 120px 0 80px;
}

.view-section {
  display: none;
  animation: fadeIn 0.35s ease;
}

.view-section.is-active {
  display: block;
}

@media (max-width: 520px) {
  .page-shell {
    width: min(100% - 24px, 1120px);
    padding-top: 104px;
  }

  .glass-navbar {
    width: calc(100% - 20px);
    top: 10px;
    border-radius: 22px;
  }

  .brand__text {
    display: none;
  }

  .glass-panel {
    padding: 18px;
  }
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class RouterSections {
  constructor(toast) {
    this.sections = $$('[data-view-section]');
    this.buttons = $$('[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.showSection(button.dataset.section));
    });
  }

  showSection(sectionId) {
    const target = document.getElementById(sectionId);
    if (!target) return;

    this.sections.forEach((section) => section.classList.remove("is-active"));
    target.classList.add("is-active");
  }
}
```

```

        this.buttons.forEach((button) => {
            button.classList.toggle("is-active", button.dataset.section === sectionId);
        });

        window.scrollTo({ top: 0, behavior: "smooth" });
        this.toast?.show(`Seccion activa: ${sectionId}`);
    }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class RouterSections {
    constructor(toast) {
        this.sections = $$('.[data-view-section]');
        this.buttons = $$('.[data-section]');
        this.toast = toast;
        this.init();
    }

    init() {
        this.buttons.forEach((button) => {
            button.addEventListener("click", () => this.showSection(button.dataset.section));
        });
    }

    showSection(sectionId) {
        const target = document.getElementById(sectionId);
        if (!target) return;

        this.sections.forEach((section) => section.classList.remove("is-active"));
        target.classList.add("is-active");

        this.buttons.forEach((button) => {
            button.classList.toggle("is-active", button.dataset.section === sectionId);
        });

        window.scrollTo({ top: 0, behavior: "smooth" });
        this.toast?.show(`Seccion activa: ${sectionId}`);
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

6. Link activo en navbar y sidebar

Navegación e interfaz base

ID 6

Descripción

Este bloque organiza la navegación principal, el cambio de secciones y los estados visuales de la interfaz. Es fundamental para páginas de una sola vista, dashboards o portafolios.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	is-active, nav-link, side-link	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="nav-link is-active" data-section="inicio">Inicio</button>
<button class="side-link is-active" data-section="inicio">Panel principal</button>
```

```

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-
shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
  background: var(--glass-strong);
  transform: translateY(-1px);
}

.nav-link.is-active,
.side-link.is-active,
.chip.is-active,
.pagination button.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
  border-color: transparent;
}

.side-link,
.side-group__button,
.side-group__content button {
  width: 100%;
  text-align: left;
}

.overlay.is-active {
  opacity: 1;
  pointer-events: auto;
}

.view-section.is-active {
  display: block;
}

.tabs__buttons button.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
}

```

```

.tab-panel.is-active {
  display: block;
}

.carousel__slide.is-active {
  display: grid;
}

.unit-button.is-active[data-unit="celsius"] {
  background: linear-gradient(135deg, #ef4444, #991b1b);
}

.unit-button.is-active[data-unit="fahrenheit"] {
  background: linear-gradient(135deg, #3b82f6, #1e3a8a);
}

.unit-button.is-active[data-unit="kelvin"] {
  background: linear-gradient(135deg, #22c55e, #14532d);
}

.stepper__steps span.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class RouterSections {
  constructor(toast) {
    this.sections = $$('[data-view-section]');
    this.buttons = $$('[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.showSection(button.dataset.section));
    });
  }

  showSection(sectionId) {
    const target = document.getElementById(sectionId);
    if (!target) return;

    this.sections.forEach((section) => section.classList.remove("is-active"));
    target.classList.add("is-active");

    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });

    window.scrollTo({ top: 0, behavior: "smooth" });
    this.toast?.show(`Sección activa: ${sectionId}`);
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.

```

```
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class RouterSections {
  constructor(toast) {
    this.sections = $$('[data-view-section]');
    this.buttons = $$('[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.showSection(button.dataset.section));
    });
  }

  showSection(sectionId) {
    const target = document.getElementById(sectionId);
    if (!target) return;

    this.sections.forEach((section) => section.classList.remove("is-active"));
    target.classList.add("is-active");

    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });

    window.scrollTo({ top: 0, behavior: "smooth" });
    this.toast?.show(`Sección activa: ${sectionId}`);
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

7. Dropdown superior

Navegación e interfaz base

ID 7

Descripción

Este bloque organiza la navegación principal, el cambio de secciones y los estados visuales de la interfaz. Es fundamental para páginas de una sola vista, dashboards o portafolios.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	dropdown, dropdown__menu	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="dropdown" data-dropdown>
  <button class="nav-link dropdown__button" data-dropdown-button type="button">Mas ▼</button>
  <div class="dropdown__menu" data-dropdown-menu>
    <button data-section="login">Login demo</button>
    <button data-section="storage">Storage</button>
    <button data-section="guia">Guía</button>
  </div>
```

```
</div>
</nav>
```

styles.css - CSS asociado validado

```
.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-
shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
  background: var(--glass-strong);
  transform: translateY(-1px);
}

.dropdown {
  position: relative;
}

.dropdown__menu {
  position: absolute;
  top: calc(100% + 10px);
  right: 0;
  min-width: 180px;
  display: grid;
  gap: 8px;
  padding: 10px;
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  background: rgba(15, 23, 42, 0.86);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  backdrop-filter: blur(18px);
  box-shadow: var(--shadow);
  transition: opacity var(--transition), transform var(--transition);
}

.dropdown.is-open .dropdown__menu {
  opacity: 1;
  transform: translateY(0);
}
```

```
pointer-events: auto;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class Dropdowns {
  constructor() {
    this.dropdowns = $$('[data-dropdown]');
    this.init();
  }

  init() {
    this.dropdowns.forEach((dropdown) => {
      const button = $('[data-dropdown-button]', dropdown);
      button.addEventListener("click", (event) => {
        event.stopPropagation();
        dropdown.classList.toggle("is-open");
      });
    });
    document.addEventListener("click", () => this.dropdowns.forEach((d) => d.classList.remove("is-open")));
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class Dropdowns {
  constructor() {
    this.dropdowns = $$('[data-dropdown]');
    this.init();
  }

  init() {
    this.dropdowns.forEach((dropdown) => {
      const button = $('[data-dropdown-button]', dropdown);
      button.addEventListener("click", (event) => {
        event.stopPropagation();
        dropdown.classList.toggle("is-open");
      });
    });
    document.addEventListener("click", () => this.dropdowns.forEach((d) => d.classList.remove("is-open")));
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.

2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

8. Tema claro/oscuro

Estado visual y experiencia de usuario

ID 8

Descripción

Este bloque mejora la experiencia del usuario con cambios visuales, scroll, temas y animaciones. La clave es conectar eventos del navegador con clases CSS de estado.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	themeToggle, data-theme, icon-button	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="icon-button" id="themeToggle" type="button" aria-label="Cambiar tema"></button>
```

styles.css - CSS asociado validado

```
[data-theme="light"] {  
  --bg: #e0f2fe;
```

```

--bg-2: #f8fafc;
--text: #0f172a;
--muted: rgba(15, 23, 42, 0.72);
--glass: rgba(255, 255, 255, 0.68);
--glass-strong: rgba(255, 255, 255, 0.86);
--border: rgba(15, 23, 42, 0.12);
--shadow: 0 25px 60px rgba(15, 23, 42, 0.18);
}

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-
shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
  background: var(--glass-strong);
  transform: translateY(-1px);
}

.icon-button {
  width: 44px;
  height: 44px;
  display: inline-grid;
  place-items: center;
  padding: 0;
  font-size: 20px;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class ThemeManager {
  constructor(buttonId = "themeToggle") {
    this.button = document.getElementById(buttonId);
    this.theme = localStorage.getItem("kit-theme") || "dark";
    this.init();
  }
}

```

```

init() {
  this.applyTheme();
  this.button.addEventListener("click", () => {
    this.theme = this.theme === "dark" ? "light" : "dark";
    localStorage.setItem("kit-theme", this.theme);
    this.applyTheme();
  });
}

applyTheme() {
  document.documentElement.dataset.theme = this.theme;
  this.button.textContent = this.theme === "dark" ? "☾" : "☀";
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ThemeManager {
  constructor(buttonId = "themeToggle") {
    this.button = document.getElementById(buttonId);
    this.theme = localStorage.getItem("kit-theme") || "dark";
    this.init();
  }

  init() {
    this.applyTheme();
    this.button.addEventListener("click", () => {
      this.theme = this.theme === "dark" ? "light" : "dark";
      localStorage.setItem("kit-theme", this.theme);
      this.applyTheme();
    });
  }

  applyTheme() {
    document.documentElement.dataset.theme = this.theme;
    this.button.textContent = this.theme === "dark" ? "☾" : "☀";
  }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.

- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

9. Guardar tema en localStorage

Estado visual y experiencia de usuario

ID 9

Descripción

Este bloque mejora la experiencia del usuario con cambios visuales, scroll, temas y animaciones. La clave es conectar eventos del navegador con clases CSS de estado.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	themeToggle, data-theme	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="icon-button" id="themeToggle" type="button" aria-label="Cambiar tema"></button>
```

styles.css - CSS asociado validado

```
[data-theme="light"] {  
  --bg: #e0f2fe;
```

```

--bg-2: #f8fafc;
--text: #0f172a;
--muted: rgba(15, 23, 42, 0.72);
--glass: rgba(255, 255, 255, 0.68);
--glass-strong: rgba(255, 255, 255, 0.86);
--border: rgba(15, 23, 42, 0.12);
--shadow: 0 25px 60px rgba(15, 23, 42, 0.18);
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class ThemeManager {
  constructor(buttonId = "themeToggle") {
    this.button = document.getElementById(buttonId);
    this.theme = localStorage.getItem("kit-theme") || "dark";
    this.init();
  }

  init() {
    this.applyTheme();
    this.button.addEventListener("click", () => {
      this.theme = this.theme === "dark" ? "light" : "dark";
      localStorage.setItem("kit-theme", this.theme);
      this.applyTheme();
    });
  }

  applyTheme() {
    document.documentElement.dataset.theme = this.theme;
    this.button.textContent = this.theme === "dark" ? "☾" : "☀";
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {

```

```
initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class ThemeManager {
  constructor(buttonId = "themeToggle") {
    this.button = document.getElementById(buttonId);
    this.theme = localStorage.getItem("kit-theme") || "dark";
    this.init();
  }

  init() {
    this.applyTheme();
    this.button.addEventListener("click", () => {
      this.theme = this.theme === "dark" ? "light" : "dark";
      localStorage.setItem("kit-theme", this.theme);
      this.applyTheme();
    });
  }

  applyTheme() {
    document.documentElement.dataset.theme = this.theme;
    this.button.textContent = this.theme === "dark" ? "☾" : "☀";
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.

- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

10. Barra de progreso de scroll

Estado visual y experiencia de usuario

ID 10

Descripción

Este bloque mejora la experiencia del usuario con cambios visuales, scroll, temas y animaciones. La clave es conectar eventos del navegador con clases CSS de estado.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	scroll-progress	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="scroll-progress" id="scrollProgress"></div>
```

styles.css - CSS asociado validado

```
.scroll-progress {  
  position: fixed;
```

```

top: 0;
left: 0;
width: 0%;
height: 4px;
z-index: 2000;
background: linear-gradient(90deg, var(--primary), var(--secondary));
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class ScrollTools {
  constructor() {
    this.progress = $("#scrollProgress");
    this.backButton = $("#backToTop");
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update());
    this.backButton.addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth" }));
    this.update();
  }

  update() {
    const total = document.documentElement.scrollHeight - window.innerHeight;
    const progress = total > 0 ? (window.scrollY / total) * 100 : 0;
    this.progress.style.width = `${progress}%`;
    this.backButton.classList.toggle("is-visible", window.scrollY > 400);
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ScrollTools {
  constructor() {
    this.progress = $("#scrollProgress");
    this.backButton = $("#backToTop");
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update());
    this.backButton.addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth" }));
    this.update();
  }
}

```

```
update() {  
  const total = document.documentElement.scrollHeight - window.innerHeight;  
  const progress = total > 0 ? (window.scrollY / total) * 100 : 0;  
  this.progress.style.width = `${progress}%`;  
  this.backButton.classList.toggle("is-visible", window.scrollY > 400);  
}  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

11. Botón volver arriba

Estado visual y experiencia de usuario

ID 11

Descripción

Este bloque mejora la experiencia del usuario con cambios visuales, scroll, temas y animaciones. La clave es conectar eventos del navegador con clases CSS de estado.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	back-to-top	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="back-to-top" id="backToTop" type="button" aria-label="Volver arriba">↑</button>
```

styles.css - CSS asociado validado

```
.back-to-top {  
  position: fixed;
```

```

    right: 20px;
    bottom: 20px;
    z-index: 900;
    width: 48px;
    height: 48px;
    border: 1px solid var(--border);
    border-radius: 50%;
    color: #ffffff;
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    opacity: 0;
    pointer-events: none;
    transition: opacity var(--transition), transform var(--transition);
}

.back-to-top.is-visible {
    opacity: 1;
    pointer-events: auto;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class ScrollTools {
    constructor() {
        this.progress = $("#scrollProgress");
        this.backButton = $("#backToTop");
        this.init();
    }

    init() {
        window.addEventListener("scroll", () => this.update());
        this.backButton.addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth" }));
        this.update();
    }

    update() {
        const total = document.documentElement.scrollHeight - window.innerHeight;
        const progress = total > 0 ? (window.scrollY / total) * 100 : 0;
        this.progress.style.width = `${progress}%`;
        this.backButton.classList.toggle("is-visible", window.scrollY > 400);
    }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ScrollTools {
    constructor() {

```

```

    this.progress = $("#scrollProgress");
    this.backButton = $("#backToTop");
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update());
    this.backButton.addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth" }));
    this.update();
  }

  update() {
    const total = document.documentElement.scrollHeight - window.innerHeight;
    const progress = total > 0 ? (window.scrollY / total) * 100 : 0;
    this.progress.style.width = `${progress}%`;
    this.backButton.classList.toggle("is-visible", window.scrollY > 400);
  }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

12. Animación reveal on scroll

Estado visual y experiencia de usuario

ID 12

Descripción

Este bloque mejora la experiencia del usuario con cambios visuales, scroll, temas y animaciones. La clave es conectar eventos del navegador con clases CSS de estado.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	reveal	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="hero glass-panel reveal">...</div>
<div class="stats-grid reveal">...</div>
```

styles.css - CSS asociado validado

```
.reveal {
  opacity: 0;
  transform: translateY(18px);
  transition: opacity 0.6s ease, transform 0.6s ease;
}

.reveal.is-visible {
  opacity: 1;
  transform: translateY(0);
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class RevealOnScroll {
  constructor(selector = ".reveal") {
    this.elements = $$(selector);
    this.observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) entry.target.classList.add("is-visible");
      });
    }, { threshold: 0.15 });
    this.elements.forEach((element) => this.observer.observe(element));
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class RevealOnScroll {
  constructor(selector = ".reveal") {
    this.elements = $$(selector);
    this.observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) entry.target.classList.add("is-visible");
      });
    }, { threshold: 0.15 });
    this.elements.forEach((element) => this.observer.observe(element));
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.

3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

13. Contador animado

Estado visual y experiencia de usuario

ID 13

Descripción

Este bloque mejora la experiencia del usuario con cambios visuales, scroll, temas y animaciones. La clave es conectar eventos del navegador con clases CSS de estado.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	stats-grid, stat-card	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="stats-grid reveal">
  <article class="stat-card glass-panel">
    <strong data-counter data-target="52">0</strong>
    <span>funcionalidades incluidas</span>
  </article>
  <article class="stat-card glass-panel">
    <strong>2</strong>
```

```

    <span>versiones JS: modular y normal</span>
  </article>
  <article class="stat-card glass-panel">
    <strong>1</strong>
    <span>pagina demo unificada</span>
  </article>

```

styles.css - CSS asociado validado

```

.stats-grid,
.feature-grid,
.component-grid,
.card-grid,
.two-column {
  display: grid;
  gap: 18px;
}

.stats-grid {
  grid-template-columns: repeat(3, 1fr);
  margin-top: 18px;
}

.stat-card strong {
  display: block;
  font-size: 42px;
}

@media (max-width: 900px) {
  .navbar-links {
    display: none;
  }

  .stats-grid,
  .component-grid,
  .two-column,
  .card-grid,
  .feature-grid,
  .data-toolbar {
    grid-template-columns: 1fr;
  }

  h1 {
    font-size: 36px;
  }
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class CounterAnimation {
  constructor(selector = "[data-counter]") {
    this.counters = $$(selector);
    this.init();
  }

  init() {
    this.counters.forEach((counter) => {
      const target = Number(counter.dataset.target || 0);
      let current = 0;
      const step = Math.max(1, Math.ceil(target / 40));

```

```

        const interval = setInterval(() => {
            current += step;
            if (current >= target) {
                current = target;
                clearInterval(interval);
            }
            counter.textContent = current;
        }, 30);
    });
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class CounterAnimation {
    constructor(selector = "[data-counter]") {
        this.counters = $$(selector);
        this.init();
    }

    init() {
        this.counters.forEach((counter) => {
            const target = Number(counter.dataset.target || 0);
            let current = 0;
            const step = Math.max(1, Math.ceil(target / 40));
            const interval = setInterval(() => {
                current += step;
                if (current >= target) {
                    current = target;
                    clearInterval(interval);
                }
                counter.textContent = current;
            }, 30);
        });
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.

- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

14. Login demo

Formularios, entradas y almacenamiento

ID 14

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	form-card, loginForm, sessionStatus, badge-row	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<section class="view-section" id="login" data-view-section>
  <div class="section-heading">
    <span class="eyebrow">Login reutilizable</span>
    <h2>Login demo con validacion, mostrar contraseña y sesion local</h2>
    <p>Es un ejemplo frontend. No reemplaza autenticacion real con backend.</p>
  </div>
```

```

<div class="two-column">
  <form class="glass-panel form-card" id="loginForm" novalidate>
    <label for="loginEmail">Correo</label>
    <input id="loginEmail" name="email" type="email" placeholder="usuario@correo.com" required>
    <small class="field-error" data-error="loginEmail"></small>

    <label for="loginPassword">Contraseña</label>
    <div class="input-action">
      <input id="loginPassword" name="password" type="password" placeholder="Mínimo 6
caracteres" required minlength="6">
      <button type="button" data-toggle-password="loginPassword">Ver</button>
    </div>
    <small class="field-error" data-error="loginPassword"></small>

    <label class="check-line">
      <input type="checkbox" id="rememberUser">
      Recordar usuario en este navegador
    </label>

    <button class="primary-button" type="submit">Ingresar</button>
    <button class="secondary-button" id="logoutButton" type="button">Cerrar sesión</button>
  </form>

  <article class="glass-panel">
    <h3>Estado de sesión</h3>
    <p id="sessionStatus">Aun no hay sesión iniciada.</p>
    <div class="badge-row">
      <span class="badge">Validación email</span>
      <span class="badge">Password toggle</span>
      <span class="badge">localStorage</span>
    </div>
  </article>
</div>
</section>

```

styles.css - CSS asociado validado

```

.navbar-links,
.navbar-actions,
.hero-actions,
.badge-row,
.carousel__actions,
.utilities-row,
.unit-buttons,
.chips,
.pagination {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.form-card {
  display: grid;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
  }
}

```

```

        this.status = $("#sessionStatus");
        this.logout = $("#logoutButton");
        this.remember = $("#rememberUser");
        this.toast = toast;
        this.init();
    }

    init() {
        this.renderStatus();
        this.form.addEventListener("submit", (event) => this.login(event));
        this.logout.addEventListener("click", () => this.logoutUser());
    }

    login(event) {
        event.preventDefault();
        const email = $("#loginEmail").value.trim();
        const password = $("#loginPassword").value.trim();
        if (!email.includes("@") || password.length < 6) {
            this.toast.show("Revisa correo y contraseña.");
            return;
        }
        const user = { email, date: new Date().toLocaleString() };
        if (this.remember.checked) saveJSON("kit-user", user);
        sessionStorage.setItem("kit-session", JSON.stringify(user));
        this.renderStatus(user);
        this.toast.show("Sesion iniciada correctamente.");
    }

    logoutUser() {
        sessionStorage.removeItem("kit-session");
        localStorage.removeItem("kit-user");
        this.renderStatus(null);
        this.toast.show("Sesion cerrada.");
    }

    renderStatus(user = null) {
        const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session") ||
"null");
        this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun
no hay sesion iniciada.";
    }
}

export class FormValidation {
    constructor(formId, toast) {
        this.form = document.getElementById(formId);
        this.toast = toast;
        if (this.form) this.init();
    }

    init() {
        this.form.addEventListener("submit", (event) => {
            event.preventDefault();
            const valid = this.validate();
            this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
        });
    }

    validate() {
        let isValid = true;
        $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
            const error = document.querySelector(`[data-error="${input.id}"]`);
            let message = "";
            if (!input.value.trim()) message = "Este campo es obligatorio.";
            if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no
valido.";

```

```

        if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo ${input.minLength} caracteres.`;
        if (error) error.textContent = message;
        input.style.borderColor = message ? "#ef4444" : "#22c55e";
        if (message) isValid = false;
    });
    return isValid;
}
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class LoginDemo {
    constructor(toast) {
        this.form = $("#loginForm");
        this.status = $("#sessionStatus");
        this.logout = $("#logoutButton");
        this.remember = $("#rememberUser");
        this.toast = toast;
        this.init();
    }

    init() {
        this.renderStatus();
        this.form.addEventListener("submit", (event) => this.login(event));
        this.logout.addEventListener("click", () => this.logoutUser());
    }

    login(event) {
        event.preventDefault();
        const email = $("#loginEmail").value.trim();
        const password = $("#loginPassword").value.trim();
        if (!email.includes("@") || password.length < 6) {
            this.toast.show("Revisa correo y contraseña.");
            return;
        }
        const user = { email, date: new Date().toLocaleString() };
        if (this.remember.checked) saveJSON("kit-user", user);
        sessionStorage.setItem("kit-session", JSON.stringify(user));
        this.renderStatus(user);
        this.toast.show("Sesion iniciada correctamente.");
    }

    logoutUser() {
        sessionStorage.removeItem("kit-session");
        localStorage.removeItem("kit-user");
        this.renderStatus(null);
        this.toast.show("Sesion cerrada.");
    }

    renderStatus(user = null) {

```

```

        const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session")) ||
        "null");
        this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun
no hay sesion iniciada.";
    }
}

class FormValidation {
    constructor(formId, toast) {
        this.form = document.getElementById(formId);
        this.toast = toast;
        if (this.form) this.init();
    }

    init() {
        this.form.addEventListener("submit", (event) => {
            event.preventDefault();
            const valid = this.validate();
            this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
        });
    }

    validate() {
        let isValid = true;
        $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
            const error = document.querySelector(`[data-error="${input.id}"]`);
            let message = "";
            if (!input.value.trim()) message = "Este campo es obligatorio.";
            if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no
valido.";
            if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo $
${input.minLength} caracteres.`;
            if (error) error.textContent = message;
            input.style.borderColor = message ? "#ef4444" : "#22c55e";
            if (message) isValid = false;
        });
        return isValid;
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

15. Logout demo

Formularios, entradas y almacenamiento

ID 15

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	logoutButton, sessionStatus	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="secondary-button" id="logoutButton" type="button">Cerrar sesión</button>
<p id="sessionStatus">Aun no hay sesión iniciada.</p>
```

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
    this.remember = $("#rememberUser");
    this.toast = toast;
    this.init();
  }

  init() {
    this.renderStatus();
    this.form.addEventListener("submit", (event) => this.login(event));
    this.logout.addEventListener("click", () => this.logoutUser());
  }

  login(event) {
    event.preventDefault();
    const email = $("#loginEmail").value.trim();
    const password = $("#loginPassword").value.trim();
    if (!email.includes("@") || password.length < 6) {
      this.toast.show("Revisa correo y contraseña.");
      return;
    }
    const user = { email, date: new Date().toLocaleString() };
    if (this.remember.checked) saveJSON("kit-user", user);
    sessionStorage.setItem("kit-session", JSON.stringify(user));
    this.renderStatus(user);
    this.toast.show("Sesion iniciada correctamente.");
  }

  logoutUser() {
    sessionStorage.removeItem("kit-session");
    localStorage.removeItem("kit-user");
    this.renderStatus(null);
    this.toast.show("Sesion cerrada.");
  }

  renderStatus(user = null) {
    const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session")) ||
    "null");
    this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun
no hay sesion iniciada.";
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
```

```
initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
    this.remember = $("#rememberUser");
    this.toast = toast;
    this.init();
  }

  init() {
    this.renderStatus();
    this.form.addEventListener("submit", (event) => this.login(event));
    this.logout.addEventListener("click", () => this.logoutUser());
  }

  login(event) {
    event.preventDefault();
    const email = $("#loginEmail").value.trim();
    const password = $("#loginPassword").value.trim();
    if (!email.includes("@") || password.length < 6) {
      this.toast.show("Revisa correo y contraseña.");
      return;
    }
    const user = { email, date: new Date().toLocaleString() };
    if (this.remember.checked) saveJSON("kit-user", user);
    sessionStorage.setItem("kit-session", JSON.stringify(user));
    this.renderStatus(user);
    this.toast.show("Sesion iniciada correctamente.");
  }

  logoutUser() {
    sessionStorage.removeItem("kit-session");
    localStorage.removeItem("kit-user");
    this.renderStatus(null);
    this.toast.show("Sesion cerrada.");
  }

  renderStatus(user = null) {
    const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session") || "null");
    this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun no hay sesion iniciada.";
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

16. Recordar usuario

Formularios, entradas y almacenamiento

ID 16

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	rememberUser, check-line	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<label class="check-line">
  <input type="checkbox" id="rememberUser">
  Recordar usuario en este navegador
</label>
```

styles.css - CSS asociado validado

```
.check-line {
  display: flex;
  align-items: center;
  gap: 10px;
  font-weight: 500;
}

.check-line input {
  width: auto;
  min-height: auto;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
    this.remember = $("#rememberUser");
    this.toast = toast;
    this.init();
  }

  init() {
    this.renderStatus();
    this.form.addEventListener("submit", (event) => this.login(event));
    this.logout.addEventListener("click", () => this.logoutUser());
  }

  login(event) {
    event.preventDefault();
    const email = $("#loginEmail").value.trim();
    const password = $("#loginPassword").value.trim();
    if (!email.includes("@") || password.length < 6) {
      this.toast.show("Revisa correo y contraseña.");
      return;
    }
  }
}
```

```

    const user = { email, date: new Date().toLocaleString() };
    if (this.remember.checked) saveJSON("kit-user", user);
    sessionStorage.setItem("kit-session", JSON.stringify(user));
    this.renderStatus(user);
    this.toast.show("Sesion iniciada correctamente.");
  }

  logoutUser() {
    sessionStorage.removeItem("kit-session");
    localStorage.removeItem("kit-user");
    this.renderStatus(null);
    this.toast.show("Sesion cerrada.");
  }

  renderStatus(user = null) {
    const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session") ||
    "null");
    this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun
    no hay sesion iniciada.";
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
    this.remember = $("#rememberUser");
  }
}

```

```

        this.toast = toast;
        this.init();
    }

    init() {
        this.renderStatus();
        this.form.addEventListener("submit", (event) => this.login(event));
        this.logout.addEventListener("click", () => this.logoutUser());
    }

    login(event) {
        event.preventDefault();
        const email = $("#loginEmail").value.trim();
        const password = $("#loginPassword").value.trim();
        if (!email.includes("@") || password.length < 6) {
            this.toast.show("Revisa correo y contraseña.");
            return;
        }
        const user = { email, date: new Date().toLocaleString() };
        if (this.remember.checked) saveJSON("kit-user", user);
        sessionStorage.setItem("kit-session", JSON.stringify(user));
        this.renderStatus(user);
        this.toast.show("Sesion iniciada correctamente.");
    }

    logoutUser() {
        sessionStorage.removeItem("kit-session");
        localStorage.removeItem("kit-user");
        this.renderStatus(null);
        this.toast.show("Sesion cerrada.");
    }

    renderStatus(user = null) {
        const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session") ||
"null");
        this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun
no hay sesion iniciada.";
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

17. Mostrar/ocultar contraseña

Formularios, entradas y almacenamiento

ID 17

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	input-action, toggle-password	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="input-action">
  <input id="loginPassword" name="password" type="password" placeholder="Mínimo 6
  caracteres" required minlength="6">
  <button type="button" data-toggle-password="loginPassword">Ver</button>
</div>
```

styles.css - CSS asociado validado

```
.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-shadow var(--transition);
}

.input-action {
  display: flex;
  gap: 10px;
}

.input-action input {
  flex: 1;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class PasswordToggle {
  constructor() {
    this.buttons = $('[data-toggle-password]');
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        const input = document.getElementById(button.dataset.togglePassword);
        input.type = input.type === "password" ? "text" : "password";
        button.textContent = input.type === "password" ? "Ver" : "Ocultar";
      });
    });
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
```

```
initApp();  
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];  
  
class PasswordToggle {  
  constructor() {  
    this.buttons = $$('[data-toggle-password]');  
    this.init();  
  }  
  
  init() {  
    this.buttons.forEach((button) => {  
      button.addEventListener("click", () => {  
        const input = document.getElementById(button.dataset.togglePassword);  
        input.type = input.type === "password" ? "text" : "password";  
        button.textContent = input.type === "password" ? "Ver" : "Ocultar";  
      });  
    });  
  }  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

18. Validación de formulario

Formularios, entradas y almacenamiento

ID 18

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	form-card, field-error, contactForm	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<form class="glass-panel form-card" id="contactForm" novalidate>
  <label for="contactName">Nombre</label>
  <input id="contactName" name="nombre" type="text" placeholder="Tu nombre" required
minlength="3">
  <small class="field-error" data-error="contactName"></small>

  <label for="contactEmail">Correo</label>
```

```

    <input id="contactEmail" name="email" type="email" placeholder="correo@ejemplo.com"
required>

    <small class="field-error" data-error="contactEmail"></small>

    <label for="messageText">Mensaje</label>
    <textarea id="messageText" rows="4" maxlength="160" placeholder="Escribe un mensaje"></
textarea>

    <small><span id="charCounter">0</span>/160 caracteres</small>

    <label for="rangeInput">Nivel de interes</label>
    <input id="rangeInput" type="range" min="0" max="100" value="50">
    <p>Valor: <strong id="rangeValue">50</strong></p>

    <label for="fileInput">Archivo</label>
    <input id="fileInput" type="file">
    <p id="fileInfo" class="muted">No hay archivo seleccionado.</p>

    <button class="primary-button" type="submit">Enviar formulario</button>
    <button class="secondary-button" id="clearDraft" type="button">Limpiar borrador</button>
</form>

```

styles.css - CSS asociado validado

```

.form-card {
  display: grid;
}

.field-error {
  min-height: 18px;
  color: #fecaca;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class FormValidation {
  constructor(formId, toast) {
    this.form = document.getElementById(formId);
    this.toast = toast;
    if (this.form) this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();
      const valid = this.validate();
      this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
    });
  }

  validate() {
    let isValid = true;
    $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
      const error = document.querySelector(`[data-error="${input.id}"]`);
      let message = "";
      if (!input.value.trim()) message = "Este campo es obligatorio.";
      if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no
valido.";
      if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo $
{input.minLength} caracteres.`;
      if (error) error.textContent = message;
      input.style.borderColor = message ? "#ef4444" : "#22c55e";
    });
  }
}

```

```

        if (message) isValid = false;
    });
    return isValid;
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class FormValidation {
    constructor(formId, toast) {
        this.form = document.getElementById(formId);
        this.toast = toast;
        if (this.form) this.init();
    }

    init() {
        this.form.addEventListener("submit", (event) => {
            event.preventDefault();
            const valid = this.validate();
            this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
        });
    }

    validate() {
        let isValid = true;
        $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
            const error = document.querySelector(`[data-error="${input.id}"]`);
            let message = "";
            if (!input.value.trim()) message = "Este campo es obligatorio.";
            if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no valido.";
            if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo ${input.minLength} caracteres.`;
            if (error) error.textContent = message;
            input.style.borderColor = message ? "#ef4444" : "#22c55e";
            if (message) isValid = false;
        });
        return isValid;
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

19. Mensajes de error por campo

Formularios, entradas y almacenamiento

ID 19

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	field-error	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<small class="field-error" data-error="contactName"></small>
<small class="field-error" data-error="contactEmail"></small>
```

styles.css - CSS asociado validado

```
.field-error {
  min-height: 18px;
  color: #fecaca;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class FormValidation {
  constructor(formId, toast) {
    this.form = document.getElementById(formId);
    this.toast = toast;
    if (this.form) this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();
      const valid = this.validate();
      this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
    });
  }

  validate() {
    let isValid = true;
    $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
      const error = document.querySelector(`[data-error="${input.id}"]`);
      let message = "";
      if (!input.value.trim()) message = "Este campo es obligatorio.";
      if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no valido.";
      if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo ${input.minLength} caracteres.`;
      if (error) error.textContent = message;
      input.style.borderColor = message ? "#ef4444" : "#22c55e";
      if (message) isValid = false;
    });
    return isValid;
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class FormValidation {
  constructor(formId, toast) {
```

```

    this.form = document.getElementById(formId);
    this.toast = toast;
    if (this.form) this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();
      const valid = this.validate();
      this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
    });
  }

  validate() {
    let isValid = true;
    $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
      const error = document.querySelector(`[data-error="${input.id}"]`);
      let message = "";
      if (!input.value.trim()) message = "Este campo es obligatorio.";
      if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no
valido.";
      if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo $
${input.minLength} caracteres.`;
      if (error) error.textContent = message;
      input.style.borderColor = message ? "#ef4444" : "#22c55e";
      if (message) isValid = false;
    });
    return isValid;
  }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

20. Contador de caracteres

Formularios, entradas y almacenamiento

ID 20

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	messageText, charCounter	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<textarea id="messageText" rows="4" maxlength="160" placeholder="Escribe un mensaje"></textarea>
<small><span id="charCounter">0</span>/160 caracteres</small>
```

styles.css - CSS asociado validado

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class CharacterCounter {
  constructor(inputId, counterId) {
    this.input = document.getElementById(inputId);
    this.counter = document.getElementById(counterId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.counter.textContent = this.input.value.length);
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class CharacterCounter {
  constructor(inputId, counterId) {
    this.input = document.getElementById(inputId);
    this.counter = document.getElementById(counterId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.counter.textContent = this.input.value.length);
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.

- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

21. Range slider con valor visible

Formularios, entradas y almacenamiento

ID 21

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	rangeInput, rangeValue	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<input id="rangeInput" type="range" min="0" max="100" value="50">
<p>Valor: <strong id="rangeValue">50</strong></p>
```

styles.css - CSS asociado validado

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class RangePreview {
  constructor(inputId, valueId) {
    this.input = document.getElementById(inputId);
    this.value = document.getElementById(valueId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.value.textContent = this.input.value);
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class RangePreview {
  constructor(inputId, valueId) {
    this.input = document.getElementById(inputId);
    this.value = document.getElementById(valueId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.value.textContent = this.input.value);
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.

- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

22. Información de archivo seleccionado

Formularios, entradas y almacenamiento

ID 22

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	fileInput, fileInfo	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<input id="fileInput" type="file">
<p id="fileInfo" class="muted">No hay archivo seleccionado.</p>
```

styles.css - CSS asociado validado

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class FileInfo {
  constructor(inputId, infoId) {
    this.input = document.getElementById(inputId);
    this.info = document.getElementById(infoId);
    this.init();
  }

  init() {
    this.input.addEventListener("change", () => {
      const file = this.input.files[0];
      this.info.textContent = file ? `${file.name} - ${(file.size / 1024).toFixed(1)} KB` : "No hay archivo seleccionado.";
    });
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class FileInfo {
  constructor(inputId, infoId) {
    this.input = document.getElementById(inputId);
    this.info = document.getElementById(infoId);
    this.init();
  }

  init() {
    this.input.addEventListener("change", () => {
      const file = this.input.files[0];
      this.info.textContent = file ? `${file.name} - ${(file.size / 1024).toFixed(1)} KB` : "No hay archivo seleccionado.";
    });
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.

4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

23. Autoguardado de borrador

Formularios, entradas y almacenamiento

ID 23

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	clearDraft, messageText	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<textarea id="messageText" rows="4" maxlength="160" placeholder="Escribe un mensaje"></textarea>
<button class="secondary-button" id="clearDraft" type="button">Limpiar borrador</button>
```

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class AutoSaveDraft {
  constructor(inputId, clearId, key, toast) {
    this.input = document.getElementById(inputId);
    this.clearButton = document.getElementById(clearId);
    this.key = key;
    this.toast = toast;
    this.init();
  }

  init() {
    this.input.value = localStorage.getItem(this.key) || "";
    this.input.dispatchEvent(new Event("input"));
    this.input.addEventListener("input", debounce(() => localStorage.setItem(this.key, this.input.value),
250));
    this.clearButton.addEventListener("click", () => {
      localStorage.removeItem(this.key);
      this.input.value = "";
      this.input.dispatchEvent(new Event("input"));
      this.toast.show("Borrador eliminado.");
    });
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class AutoSaveDraft {
  constructor(inputId, clearId, key, toast) {
    this.input = document.getElementById(inputId);
    this.clearButton = document.getElementById(clearId);
    this.key = key;
    this.toast = toast;
    this.init();
  }

  init() {
    this.input.value = localStorage.getItem(this.key) || "";
    this.input.dispatchEvent(new Event("input"));
    this.input.addEventListener("input", debounce(() => localStorage.setItem(this.key, this.input.value),
250));
    this.clearButton.addEventListener("click", () => {
      localStorage.removeItem(this.key);
      this.input.value = "";
      this.input.dispatchEvent(new Event("input"));
      this.toast.show("Borrador eliminado.");
    });
  }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.

- La versión modular y normal no se cargan juntas.

24. Stepper de pasos

Formularios, entradas y almacenamiento

ID 24

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	stepper	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="stepper" data-stepper>
  <div class="stepper__steps">
    <span class="is-active">1</span>
    <span>2</span>
    <span>3</span>
  </div>
  <p class="stepper__text">Paso 1: Datos personales</p>
```

```
<button class="secondary-button" data-step-prev>Anterior</button>
<button class="primary-button" data-step-next>Siguiente</button>
```

styles.css - CSS asociado validado

```
.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-shadow var(--transition);
}

.stepper__steps {
  display: flex;
  gap: 12px;
  margin-bottom: 12px;
}

.stepper__steps span {
  width: 36px;
  height: 36px;
  display: grid;
  place-items: center;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid var(--border);
}

.stepper__steps span.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class Stepper {
  constructor(selector = "[data-stepper]") {
    this.stepper = $(selector);
    this.index = 0;
    this.texts = ["Paso 1: Datos personales", "Paso 2: Preferencias", "Paso 3: Confirmacion"];
    this.init();
  }

  init() {
    $('[data-step-next]', this.stepper).addEventListener("click", () => this.move(1));
  }
}
```

```

    $('[data-step-prev]', this.stepper).addEventListener("click", () => this.move(-1));
    this.render();
  }

  move(direction) {
    this.index = Math.min(2, Math.max(0, this.index + direction));
    this.render();
  }

  render() {
    $(".stepper__steps span", this.stepper).forEach((step, i) => step.classList.toggle("is-active", i <=
this.index));
    $(".stepper__text", this.stepper).textContent = this.texts[this.index];
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class Stepper {
  constructor(selector = "[data-stepper]") {
    this.stepper = $(selector);
    this.index = 0;
    this.texts = ["Paso 1: Datos personales", "Paso 2: Preferencias", "Paso 3: Confirmacion"];
    this.init();
  }

  init() {
    $('[data-step-next]', this.stepper).addEventListener("click", () => this.move(1));
    $('[data-step-prev]', this.stepper).addEventListener("click", () => this.move(-1));
    this.render();
  }

  move(direction) {
    this.index = Math.min(2, Math.max(0, this.index + direction));
    this.render();
  }

  render() {
    $(".stepper__steps span", this.stepper).forEach((step, i) => step.classList.toggle("is-active", i <=
this.index));
    $(".stepper__text", this.stepper).textContent = this.texts[this.index];
  }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.

5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

25. Skeleton loader

Formularios, entradas y almacenamiento

ID 25

Descripción

Este bloque trabaja con inputs, formularios y datos temporales del navegador. Es importante respetar ids y atributos required, minlength, maxlength o data-* porque JavaScript los usa para validar.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	skeleton	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="secondary-button" id="loadSkeleton">Simular carga</button>
<div id="skeletonArea"></div>
```

styles.css - CSS asociado validado

```
.skeleton {
  height: 18px;
  margin-top: 12px;
  border-radius: 999px;
  background: linear-gradient(90deg, rgba(255,255,255,0.08), rgba(255,255,255,0.24), rgba(255,255,255,0.08));
  background-size: 200% 100%;
  animation: loading 1.2s infinite;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class SkeletonLoader {
  constructor(buttonId, areaId, toast) {
    this.button = document.getElementById(buttonId);
    this.area = document.getElementById(areaId);
    this.toast = toast;
    this.init();
  }

  init() {
    this.button.addEventListener("click", () => {
      this.area.innerHTML = `<div class="skeleton"></div><div class="skeleton"></div><div class="skeleton"></div>`;
      setTimeout(() => {
        this.area.innerHTML = `<p class="muted">Contenido cargado correctamente.</p>`;
        this.toast.show("Carga simulada finalizada.");
      }, 1200);
    });
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class SkeletonLoader {
  constructor(buttonId, areaId, toast) {
    this.button = document.getElementById(buttonId);
    this.area = document.getElementById(areaId);
    this.toast = toast;
    this.init();
  }

  init() {
    this.button.addEventListener("click", () => {
      this.area.innerHTML = `<div class="skeleton"></div><div class="skeleton"></div><div class="skeleton"></div>`;
    });
  }
}
```

```
class="skeleton"></div>`;
    setTimeout(() => {
        this.area.innerHTML = `<p class="muted">Contenido cargado correctamente.</p>`;
        this.toast.show("Carga simulada finalizada.");
    }, 1200);
});
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

26. Select personalizado editable

Componentes interactivos reutilizables

ID 26

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	custom-select	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="custom-select" data-custom-select>
  <input type="hidden" id="selectProducto" value="apa">
  <button type="button" class="custom-select__button">
    <span class="custom-select__value">APA artesanal</span>
    <span class="custom-select__arrow">></span>
  </button>
  <ul class="custom-select__options">
```

```

        <li class="custom-select__option is-selected" data-value="apa">APA artesanal</li>
        <li class="custom-select__option" data-value="stout">Stout premium</li>
        <li class="custom-select__option" data-value="sour">Sour de fruta</li>
        <li class="custom-select__option" data-value="barley">Barley Wine</li>
    </ul>
</div>
<p class="muted">Valor seleccionado

```

styles.css - CSS asociado validado

```

.custom-select {
    position: relative;
    width: 100%;
    z-index: 20;
}

.custom-select__button {
    width: 100%;
    min-height: 50px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 12px 16px;
    color: var(--text);
    background: rgba(255, 255, 255, 0.16);
    border: 1px solid var(--border);
    border-radius: var(--radius-sm);
    backdrop-filter: blur(8px);
}

.custom-select__arrow {
    font-size: 22px;
    transition: transform var(--transition);
}

.custom-select__options {
    position: absolute;
    top: calc(100% + 8px);
    left: 0;
    width: 100%;
    margin: 0;
    padding: 8px;
    list-style: none;
    opacity: 0;
    transform: translateY(-8px);
    pointer-events: none;
    background: rgba(15, 23, 42, 0.88);
    border: 1px solid rgba(255, 255, 255, 0.25);
    border-radius: var(--radius-md);
    backdrop-filter: blur(18px);
    box-shadow: var(--shadow);
    transition: opacity var(--transition), transform var(--transition);
}

.custom-select.is-open .custom-select__options {
    opacity: 1;
    transform: translateY(0);
    pointer-events: auto;
}

.custom-select.is-open .custom-select__arrow {
    transform: rotate(180deg);
}

.custom-select__option {

```

```
padding: 12px 14px;
border-radius: var(--radius-sm);
color: #ffffff;
cursor: pointer;
}

.custom-select__option:hover,
.custom-select__option.is-selected {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class CustomSelect {
  constructor(selector = "[data-custom-select]", onChange = null) {
    this.selects = $(selector);
    this.onChange = onChange;
    this.init();
  }

  init() {
    this.selects.forEach((select) => {
      const button = $(".custom-select__button", select);
      const valueText = $(".custom-select__value", select);
      const hiddenInput = $("input[type='hidden']", select);
      const options = $(".custom-select__option", select);

      button.addEventListener("click", (event) => {
        event.stopPropagation();
        this.closeAll(select);
        select.classList.toggle("is-open");
      });

      options.forEach((option) => {
        option.addEventListener("click", () => {
          hiddenInput.value = option.dataset.value;
          valueText.textContent = option.textContent;
          options.forEach((item) => item.classList.remove("is-selected"));
          option.classList.add("is-selected");
          select.classList.remove("is-open");
          this.onChange?.(hiddenInput.value, select);
          document.dispatchEvent(new CustomEvent("custom-select:change", { detail: { value:
hiddenInput.value } }));
        });
      });

      document.addEventListener("click", () => this.closeAll());
    });

    closeAll(except = null) {
      this.selects.forEach((select) => {
        if (select !== except) select.classList.remove("is-open");
      });
    }
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
```

```
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class CustomSelect {
  constructor(selector = "[data-custom-select]", onChange = null) {
    this.selects = $$(selector);
    this.onChange = onChange;
    this.init();
  }

  init() {
    this.selects.forEach((select) => {
      const button = $(".custom-select__button", select);
      const valueText = $(".custom-select__value", select);
      const hiddenInput = $("input[type='hidden']", select);
      const options = $$(".custom-select__option", select);

      button.addEventListener("click", (event) => {
        event.stopPropagation();
        this.closeAll(select);
        select.classList.toggle("is-open");
      });

      options.forEach((option) => {
        option.addEventListener("click", () => {
          hiddenInput.value = option.dataset.value;
          valueText.textContent = option.textContent;
          options.forEach((item) => item.classList.remove("is-selected"));
          option.classList.add("is-selected");
          select.classList.remove("is-open");
          this.onChange?.(hiddenInput.value, select);
          document.dispatchEvent(new CustomEvent("custom-select:change", { detail: { value:
hiddenInput.value } }));
        });
      });
    });

    document.addEventListener("click", () => this.closeAll());
  }

  closeAll(except = null) {
    this.selects.forEach((select) => {
      if (select !== except) select.classList.remove("is-open");
    });
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

27. Tabs

Componentes interactivos reutilizables

ID 27

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	tabs, tab-panel	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="tabs" data-tabs>
  <div class="tabs__buttons">
    <button class="is-active" data-tab="tab1">HTML</button>
    <button data-tab="tab2">CSS</button>
    <button data-tab="tab3">JS</button>
  </div>
  <div class="tab-panel is-active" id="tab1">HTML estructura el contenido.</div>
```

```
<div class="tab-panel" id="tab2">CSS diseña colores, tamaños, layout y responsive.</div>
<div class="tab-panel" id="tab3">JS agrega interaccion y logica.</div>
```

styles.css - CSS asociado validado

```
.tabs__buttons {
  display: flex;
  gap: 8px;
  margin-bottom: 12px;
}

.tabs__buttons button {
  border: 1px solid var(--border);
  border-radius: 999px;
  padding: 8px 12px;
  color: var(--text);
  background: rgba(255, 255, 255, 0.12);
}

.tabs__buttons button.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
}

.tab-panel {
  display: none;
  padding: 14px;
  border-radius: var(--radius-sm);
  background: rgba(255, 255, 255, 0.08);
}

.tab-panel.is-active {
  display: block;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class Tabs {
  constructor(selector = "[data-tabs]") {
    this.components = $$(selector);
    this.init();
  }

  init() {
    this.components.forEach((tabs) => {
      const buttons = $$(".data-tab", tabs);
      const panels = $$(".tab-panel", tabs);
      buttons.forEach((button) => {
        button.addEventListener("click", () => {
          buttons.forEach((item) => item.classList.remove("is-active"));
          panels.forEach((panel) => panel.classList.remove("is-active"));
          button.classList.add("is-active");
          document.getElementById(button.dataset.tab)?.classList.add("is-active");
        });
      });
    });
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class Tabs {
  constructor(selector = "[data-tabs]") {
    this.components = $$(selector);
    this.init();
  }

  init() {
    this.components.forEach((tabs) => {
      const buttons = $$(".data-tab", tabs);
      const panels = $$(".tab-panel", tabs);
      buttons.forEach((button) => {
        button.addEventListener("click", () => {
          buttons.forEach((item) => item.classList.remove("is-active"));
          panels.forEach((panel) => panel.classList.remove("is-active"));
          button.classList.add("is-active");
          document.getElementById(button.dataset.tab)?.classList.add("is-active");
        });
      });
    });
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

28. Acordeón

Componentes interactivos reutilizables

ID 28

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	accordion	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="accordion" data-accordion>
  <button class="accordion_button">¿Para que sirve un componente?</button>
  <div class="accordion_content">Para reutilizar una solución en varias páginas sin
  empezar desde cero.</div>
  <button class="accordion_button">¿Puedo cambiar colores?</button>
  <div class="accordion_content">Si. Cambia las variables CSS dentro de <code>:root</
code>.</div>
```

```

        <button class="accordion__button">¿Funciona con JS modular?</button>
        <div class="accordion__content">Si. El archivo principal importa clases desde
</code>funciones.js</code>.</div>

```

styles.css - CSS asociado validado

```

.accordion__button {
  width: 100%;
  margin-top: 8px;
  padding: 12px;
  text-align: left;
  color: var(--text);
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
}

.accordion__content {
  max-height: 0;
  overflow: hidden;
  color: var(--muted);
  transition: max-height var(--transition), padding var(--transition);
}

.accordion__content.is-open {
  max-height: 140px;
  padding: 12px;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class Accordion {
  constructor(selector = "[data-accordion]") {
    this.components = $$ (selector);
    this.init();
  }

  init() {
    this.components.forEach((accordion) => {
      const buttons = $$(".accordion__button", accordion);
      buttons.forEach((button) => {
        button.addEventListener("click", () => button.nextElementSibling.classList.toggle("is-open"));
      });
    });
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class Accordion {
  constructor(selector = "[data-accordion]") {
    this.components = $$(selector);
    this.init();
  }

  init() {
    this.components.forEach((accordion) => {
      const buttons = $$(".accordion__button", accordion);
      buttons.forEach((button) => {
        button.addEventListener("click", () => button.nextElementSibling.classList.toggle("is-open"));
      });
    });
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

29. Modal

Componentes interactivos reutilizables

ID 29

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	modal	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="primary-button" data-open-modal="demoModal">Abrir modal</button>
<dialog class="modal" id="demoModal">
  <div class="modal__content">
    <button class="modal__close" data-close-modal type="button">x</button>
    <h3>Modal reutilizable</h3>
    <p>Este modal se puede usar para alertas, confirmaciones, detalles de producto o formularios cortos.</p>
```

```

        <button class="primary-button" data-close-modal type="button">Entendido</button>
      </div>
</dialog>

```

styles.css - CSS asociado validado

```

.modal {
  width: min(520px, calc(100% - 32px));
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 0;
  color: var(--text);
  background: rgba(15, 23, 42, 0.92);
  backdrop-filter: blur(20px);
  box-shadow: var(--shadow);
}

.modal::backdrop {
  background: rgba(0, 0, 0, 0.55);
}

.modal__content {
  position: relative;
  padding: 26px;
}

.modal__close {
  position: absolute;
  top: 14px;
  right: 14px;
  width: 36px;
  height: 36px;
  border-radius: 50%;
  border: 1px solid var(--border);
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class ModalManager {
  constructor() {
    this.init();
  }

  init() {
    $$('[data-open-modal]').forEach((button) => {
      button.addEventListener("click", () =>
document.getElementById(button.dataset.openModal)?.showModal());
    });
    $$('[data-close-modal]').forEach((button) => {
      button.addEventListener("click", () => button.closest("dialog)?.close());
    });
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.

```

```
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ModalManager {
  constructor() {
    this.init();
  }

  init() {
    $('[data-open-modal]').forEach((button) => {
      button.addEventListener("click", () =>
document.getElementById(button.dataset.openModal)?.showModal());
    });
    $('[data-close-modal]').forEach((button) => {
      button.addEventListener("click", () => button.closest("dialog)?.close());
    });
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

30. Toast notifications

Componentes interactivos reutilizables

ID 30

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	toast	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="secondary-button" data-toast="Este es un toast reutilizable">Mostrar toast</button>
<div class="toast-container" id="toastContainer"></div>
```

styles.css - CSS asociado validado

```
.toast-container {
  position: fixed;
  right: 20px;
  bottom: 20px;
  z-index: 2000;
  display: grid;
  gap: 10px;
}

.toast {
  padding: 14px 16px;
  border-radius: var(--radius-md);
  color: #ffffff;
  background: rgba(15, 23, 42, 0.92);
  border: 1px solid rgba(255, 255, 255, 0.2);
  box-shadow: var(--shadow);
  animation: slideToast 0.25s ease;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
```

```
toast.className = "toast";  
toast.textContent = message;  
this.container.appendChild(toast);  
setTimeout(() => toast.remove(), time);  
}  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

31. Carrusel

Componentes interactivos reutilizables

ID 31

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	carousel	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="carousel" data-carousel>
  <div class="carousel__track">
    <div class="carousel__slide is-active">Slide 1 - Hero</div>
    <div class="carousel__slide">Slide 2 - Producto</div>
    <div class="carousel__slide">Slide 3 - Contacto</div>
  </div>
  <div class="carousel__actions">
```

```

        <button data-carousel-prev>Anterior</button>
        <button data-carousel-next>Siguiente</button>
    </div>

```

styles.css - CSS asociado validado

```

.navbar-links,
.navbar-actions,
.hero-actions,
.badge-row,
.carousel__actions,
.utilities-row,
.unit-buttons,
.chips,
.pagination {
    display: flex;
    align-items: center;
    gap: 10px;
    flex-wrap: wrap;
}

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
    border: 1px solid var(--border);
    color: var(--text);
    background: rgba(255, 255, 255, 0.1);
    border-radius: 999px;
    padding: 10px 14px;
    transition: transform var(--transition), background var(--transition), border-color var(--transition), box-shadow var(--transition);
}

.carousel__slide {
    display: none;
    min-height: 120px;
    place-items: center;
    border-radius: var(--radius-md);
    background: linear-gradient(135deg, rgba(37, 99, 235, 0.6), rgba(124, 58, 237, 0.6));
    color: #ffffff;
    font-weight: 900;
}

.carousel__slide.is-active {
    display: grid;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class Carousel {

```

```

    constructor(selector = "[data-carousel]") {
        this.carousels = $$(selector);
        this.init();
    }

    init() {
        this.carousels.forEach((carousel) => {
            const slides = $$(".carousel__slide", carousel);
            const next = $("[data-carousel-next]", carousel);
            const prev = $("[data-carousel-prev]", carousel);
            let index = 0;
            const render = () => slides.forEach((slide, i) => slide.classList.toggle("is-active", i === index));
            next.addEventListener("click", () => { index = (index + 1) % slides.length; render(); });
            prev.addEventListener("click", () => { index = (index - 1 + slides.length) % slides.length;
render(); });
            });
        }
    }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class Carousel {
    constructor(selector = "[data-carousel]") {
        this.carousels = $$(selector);
        this.init();
    }

    init() {
        this.carousels.forEach((carousel) => {
            const slides = $$(".carousel__slide", carousel);
            const next = $("[data-carousel-next]", carousel);
            const prev = $("[data-carousel-prev]", carousel);
            let index = 0;
            const render = () => slides.forEach((slide, i) => slide.classList.toggle("is-active", i === index));
            next.addEventListener("click", () => { index = (index + 1) % slides.length; render(); });
            prev.addEventListener("click", () => { index = (index - 1 + slides.length) % slides.length;
render(); });
            });
        }
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

32. Tooltip

Componentes interactivos reutilizables

ID 32

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	tooltip	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="icon-button tooltip" data-tooltip="Texto de ayuda que aparece al pasar el mouse"></button>
```

styles.css - CSS asociado validado

```
.tooltip {  
  position: relative;
```

```

}

.tooltip::after {
  content: attr(data-tooltip);
  position: absolute;
  bottom: calc(100% + 8px);
  right: 0;
  width: 210px;
  padding: 10px;
  border-radius: var(--radius-sm);
  color: #ffffff;
  background: rgba(15, 23, 42, 0.92);
  opacity: 0;
  pointer-events: none;
  transform: translateY(6px);
  transition: opacity var(--transition), transform var(--transition);
}

.tooltip:hover::after {
  opacity: 1;
  transform: translateY(0);
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

33. Badge de notificación

Componentes interactivos reutilizables

ID 33

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	notification-badge	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<span class="notification-badge" id="notificationBadge">3</span>
```

styles.css - CSS asociado validado

```
.badge,  
.notification-badge {
```

```

display: inline-flex;
align-items: center;
justify-content: center;
border-radius: 999px;
padding: 7px 10px;
color: #ffffff;
background: rgba(37, 99, 235, 0.9);
font-size: 13px;
font-weight: 800;
}

.notification-badge {
width: 32px;
height: 32px;
padding: 0;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

34. Copiar al portapapeles

Componentes interactivos reutilizables

ID 34

Descripción

Este bloque contiene patrones reutilizables de interfaz: tabs, acordeones, modal, toast, carrusel y utilidades de copia. Son componentes que se pueden llevar a casi cualquier pagina.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	couponCode, data-copy	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<p>Codigo promocional: <code id="couponCode">WEB-2026</code></p>
<button class="secondary-button" data-copy="#couponCode">Copiar codigo</button>
```

styles.css - CSS asociado validado

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class CopyToClipboard {
  constructor(toast) {
    this.buttons = $$('[data-copy]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", async () => {
        const target = document.querySelector(button.dataset.copy);
        await navigator.clipboard.writeText(target.textContent);
        this.toast.show("Texto copiado.");
      });
    });
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class CopyToClipboard {
  constructor(toast) {
    this.buttons = $$('[data-copy]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", async () => {
        const target = document.querySelector(button.dataset.copy);
        await navigator.clipboard.writeText(target.textContent);
        this.toast.show("Texto copiado.");
      });
    });
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

35. Cards dinámicas desde array

Datos, búsqueda, tablas y almacenamiento

ID 35

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	card-grid, feature-grid, product-card	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="feature-grid reveal" id="featureList"></div>
<div class="card-grid" id="catalogGrid"></div>
```

styles.css - CSS asociado validado

```
.stats-grid,
.feature-grid,
.component-grid,
.card-grid,
.two-column {
  display: grid;
  gap: 18px;
}

.feature-grid {
  grid-template-columns: repeat(4, 1fr);
  margin-top: 18px;
}

.card-grid {
  grid-template-columns: repeat(3, 1fr);
}

.product-card {
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 18px;
  background: rgba(255, 255, 255, 0.1);
}

.product-card__meta {
  display: flex;
  justify-content: space-between;
  gap: 8px;
  color: var(--muted);
}

@media (max-width: 900px) {
  .navbar-links {
    display: none;
  }

  .stats-grid,
  .component-grid,
  .two-column,
  .card-grid,
  .feature-grid,
  .data-toolbar {
    grid-template-columns: 1fr;
  }

  h1 {
    font-size: 36px;
  }
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}
```

```

}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
      .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
  }

  render() {
    const items = this.getFilteredData();
    const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
    this.page = Math.min(this.page, totalPages);
  }
}

```

```

const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
this.grid.innerHTML = pageItems.map((item) => `
  <article class="product-card">
    <h3>${item.name}</h3>
    <p class="muted">Categoria: ${item.category}</p>
    <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
{item.rating}</span></div>
  </article>
`).join("") || `<p class="muted">No hay resultados.</p>`;
this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");
$$( 'button', this.pagination ).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
}
}

export class FeatureList {
  constructor() {
    this.features = [
      "Navbar fija glass", "Sidebar oclutable", "Submenus laterales", "Router de secciones",
      "Dropdown", "Tema claro/oscuro", "Scroll progress", "Back to top",
      "Reveal on scroll", "Contador animado", "Login demo", "Logout demo",
      "Recordar usuario", "Password toggle", "Validacion formularios", "Mensajes de error",
      "Character counter", "Range preview", "File info", "Autosave draft",
      "Stepper", "Skeleton loader", "Select personalizado", "Tabs",
      "Acordeon", "Modal", "Toast", "Carrusel",
      "Tooltip", "Badge", "Copy clipboard", "Cards dinamicas",
      "Busqueda", "Filtros chips", "Ordenamiento", "Paginacion",
      "Tabla ordenable", "Filtro de tabla", "LocalStorage notas", "Export JSON",
      "Todo list", "Drag and drop", "Confirmacion visual", "Conversor C/F/K",
      "Cambio fondo dinamico", "Print page", "Reset demo", "Debounce",
      "Helpers reutilizables", "Version modular", "Version normal", "Responsive design"
    ];
    this.container = $("#featureList");
    this.render();
  }

  render() {
    this.container.innerHTML = this.features.map((feature, index) => `
      <article class="feature-card">
        <strong>${index + 1}. ${feature}</strong>
        <span class="muted">Reutilizable en proyectos web.</span>
      </article>
    `).join("");
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;

```

```

    return (...args) => {
      clearTimeout(timeoutId);
      timeoutId = setTimeout(() => callback(...args), delay);
    };
  }

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
      .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
  }
}

```

```

render() {
  const items = this.getFilteredData();
  const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
  this.page = Math.min(this.page, totalPages);
  const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
  this.grid.innerHTML = pageItems.map((item) => `
    <article class="product-card">
      <h3>${item.name}</h3>
      <p class="muted">Categoria: ${item.category}</p>
      <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
{item.rating}</span></div>
    </article>
  `).join("") || `<p class="muted">No hay resultados.</p>`;
  this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");
  $$('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
}
}

class FeatureList {
  constructor() {
    this.features = [
      "Navbar fija glass", "Sidebar ocultable", "Submenus laterales", "Router de secciones",
      "Dropdown", "Tema claro/oscuro", "Scroll progress", "Back to top",
      "Reveal on scroll", "Contador animado", "Login demo", "Logout demo",
      "Recordar usuario", "Password toggle", "Validacion formularios", "Mensajes de error",
      "Character counter", "Range preview", "File info", "Autosave draft",
      "Stepper", "Skeleton loader", "Select personalizado", "Tabs",
      "Acordeon", "Modal", "Toast", "Carrusel",
      "Tooltip", "Badge", "Copy clipboard", "Cards dinamicas",
      "Busqueda", "Filtros chips", "Ordenamiento", "Paginacion",
      "Tabla ordenable", "Filtro de tabla", "LocalStorage notas", "Export JSON",
      "Todo list", "Drag and drop", "Confirmacion visual", "Conversor C/F/K",
      "Cambio fondo dinamico", "Print page", "Reset demo", "Debounce",
      "Helpers reutilizables", "Version modular", "Version normal", "Responsive design"
    ];
    this.container = $("#featureList");
    this.render();
  }

  render() {
    this.container.innerHTML = this.features.map((feature, index) => `
      <article class="feature-card">
        <strong>${index + 1}. ${feature}</strong>
        <span class="muted">Reutilizable en proyectos web.</span>
      </article>
    `).join("");
  }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.

- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

36. Búsqueda con debounce

Datos, búsqueda, tablas y almacenamiento

ID 36

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	catalogSearch	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<input id="catalogSearch" type="search" placeholder="Buscar producto...">
```

styles.css - CSS asociado validado

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $$("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $$("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }
}

```

```

    getFilteredData() {
      return this.data
        .filter((item) => this.category === "all" || item.category === this.category)
        .filter((item) => item.name.toLowerCase().includes(this.search))
        .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
    }

    render() {
      const items = this.getFilteredData();
      const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
      this.page = Math.min(this.page, totalPages);
      const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
      this.grid.innerHTML = pageItems.map((item) => `
        <article class="product-card">
          <h3>${item.name}</h3>
          <p class="muted">Categoria: ${item.category}</p>
          <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
{item.rating}</span></div>
        </article>
      `).join("") || `<p class="muted">No hay resultados.</p>`;
      this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");
      $$('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
    }
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

```

```

class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
      .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
  }

  render() {
    const items = this.getFilteredData();
    const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
    this.page = Math.min(this.page, totalPages);
    const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
    this.grid.innerHTML = pageItems.map((item) => `
      <article class="product-card">
        <h3>${item.name}</h3>
        <p class="muted">Categoria: ${item.category}</p>
        <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
${item.rating}</span></div>
      </article>
    `).join("") || `<p class="muted">No hay resultados.</p>`;
    this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1 ===
this.page ? "is-active" : ""}" data-page="${i + 1}">${i + 1}</button>`).join("");
    $('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
  }
}

```

```
}  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

37. Filtros con chips

Datos, búsqueda, tablas y almacenamiento

ID 37

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	chips, chip	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="chips" id="catalogChips">
  <button class="chip is-active" data-category="all">Todos</button>
  <button class="chip" data-category="cerveza">Cerveza</button>
  <button class="chip" data-category="comida">Comida</button>
```

```
<button class="chip" data-category="postre">Postre</button>
</div>
```

styles.css - CSS asociado validado

```
.navbar-links,
.navbar-actions,
.hero-actions,
.badge-row,
.carousel__actions,
.utilities-row,
.unit-buttons,
.chips,
.pagination {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
  background: var(--glass-strong);
  transform: translateY(-1px);
}

.nav-link.is-active,
.side-link.is-active,
.chip.is-active,
.pagination button.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
  border-color: transparent;
}

.chips {
```

```

    grid-column: 1 / -1;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
    let timeoutId;
    return (...args) => {
        clearTimeout(timeoutId);
        timeoutId = setTimeout(() => callback(...args), delay);
    };
}

export function saveJSON(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
    try {
        const value = localStorage.getItem(key);
        return value ? JSON.parse(value) : fallback;
    } catch {
        return fallback;
    }
}

export class CatalogCards {
    constructor() {
        this.data = [
            { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
            { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
            { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
            { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
            { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
            { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
            { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
            { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
        ];
        this.page = 1;
        this.perPage = 6;
        this.category = "all";
        this.search = "";
        this.sort = "name";
        this.grid = $("#catalogGrid");
        this.pagination = $("#catalogPagination");
        this.init();
    }

    init() {
        $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
        $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
        $("#catalogChips .chip").forEach((chip) => {
            chip.addEventListener("click", () => {
                $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
                chip.classList.add("is-active");
                this.category = chip.dataset.category;
                this.page = 1;
                this.render();
            });
        });
    }
}

```

```

    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
      .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
  }

  render() {
    const items = this.getFilteredData();
    const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
    this.page = Math.min(this.page, totalPages);
    const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
    this.grid.innerHTML = pageItems.map((item) => `
      <article class="product-card">
        <h3>${item.name}</h3>
        <p class="muted">Categoria: ${item.category}</p>
        <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
{item.rating}</span></div>
      </article>
    `).join("") || `<p class="muted">No hay resultados.</p>`;
    this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");
    $('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {}
}

```

```

        return fallback;
    }
}

class CatalogCards {
    constructor() {
        this.data = [
            { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
            { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
            { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
            { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
            { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
            { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
            { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
            { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
        ];
        this.page = 1;
        this.perPage = 6;
        this.category = "all";
        this.search = "";
        this.sort = "name";
        this.grid = $("#catalogGrid");
        this.pagination = $("#catalogPagination");
        this.init();
    }

    init() {
        $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
        $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
        $("#catalogChips .chip").forEach((chip) => {
            chip.addEventListener("click", () => {
                $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
                chip.classList.add("is-active");
                this.category = chip.dataset.category;
                this.page = 1;
                this.render();
            });
        });
        this.render();
    }

    getFilteredData() {
        return this.data
            .filter((item) => this.category === "all" || item.category === this.category)
            .filter((item) => item.name.toLowerCase().includes(this.search))
            .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
    }

    render() {
        const items = this.getFilteredData();
        const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
        this.page = Math.min(this.page, totalPages);
        const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
        this.grid.innerHTML = pageItems.map((item) => `
            <article class="product-card">
                <h3>${item.name}</h3>
                <p class="muted">Categoria: ${item.category}</p>
                <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
${item.rating}</span></div>
            </article>
        `).join("") || `<p class="muted">No hay resultados.</p>`;
        this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");
    }
}

```

```
    $$('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =  
    Number(button.dataset.page); this.render(); }));  
  }  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

38. Ordenamiento de cards

Datos, búsqueda, tablas y almacenamiento

ID 38

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	catalogSort	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<select id="catalogSort">
  <option value="name">Ordenar por nombre</option>
  <option value="price">Ordenar por precio</option>
  <option value="rating">Ordenar por calificación</option>
</select>
```

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
  }
}
```

```

    });
  });
  this.render();
}

getFilteredData() {
  return this.data
    .filter((item) => this.category === "all" || item.category === this.category)
    .filter((item) => item.name.toLowerCase().includes(this.search))
    .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
}

render() {
  const items = this.getFilteredData();
  const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
  this.page = Math.min(this.page, totalPages);
  const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
  this.grid.innerHTML = pageItems.map((item) => `
    <article class="product-card">
      <h3>${item.name}</h3>
      <p class="muted">Categoria: ${item.category}</p>
      <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
{item.rating}</span></div>
    </article>
  `).join("") || `<p class="muted">No hay resultados.</p>`;
  this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");
  $$('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  }
}

```

```

    } catch {
      return fallback;
    }
  }

class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
      .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
  }

  render() {
    const items = this.getFilteredData();
    const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
    this.page = Math.min(this.page, totalPages);
    const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
    this.grid.innerHTML = pageItems.map((item) => `
      <article class="product-card">
        <h3>${item.name}</h3>
        <p class="muted">Categor  a: ${item.category}</p>
        <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
${item.rating}</span></div>
      </article>
    `).join("") || `<p class="muted">No hay resultados.</p>`;
    this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===

```

```
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");  
    $$('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =  
    Number(button.dataset.page); this.render(); }));  
  }  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

39. Paginación de cards

Datos, búsqueda, tablas y almacenamiento

ID 39

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	pagination	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="pagination" id="catalogPagination"></div>
```

styles.css - CSS asociado validado

```
.navbar-links,  
.navbar-actions,
```

```

.hero-actions,
.badge-row,
.carousel__actions,
.utilities-row,
.unit-buttons,
.chips,
.pagination {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
  background: var(--glass-strong);
  transform: translateY(-1px);
}

.nav-link.is-active,
.side-link.is-active,
.chip.is-active,
.pagination button.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
  border-color: transparent;
}

.pagination {
  margin: 18px 0;
  justify-content: center;
}

```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);
```

```

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
  }
}

```

```

        .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
    }

    render() {
        const items = this.getFilteredData();
        const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
        this.page = Math.min(this.page, totalPages);
        const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
        this.grid.innerHTML = pageItems.map((item) => `
            <article class="product-card">
                <h3>${item.name}</h3>
                <p class="muted">Categoria: ${item.category}</p>
                <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
{item.rating}</span></div>
            </article>
        `).join("") || `<p class="muted">No hay resultados.</p>`;
        this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1} ===
this.page ? "is-active" : """ data-page="${i + 1}">${i + 1}</button>`).join("");
        $$('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
    }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
    let timeoutId;
    return (...args) => {
        clearTimeout(timeoutId);
        timeoutId = setTimeout(() => callback(...args), delay);
    };
}

function saveJSON(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
    try {
        const value = localStorage.getItem(key);
        return value ? JSON.parse(value) : fallback;
    } catch {
        return fallback;
    }
}

class CatalogCards {
    constructor() {
        this.data = [
            { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },

```

```

        { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
        { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
        { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
        { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
        { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
        { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
        { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
}

init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
        chip.addEventListener("click", () => {
            $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
            chip.classList.add("is-active");
            this.category = chip.dataset.category;
            this.page = 1;
            this.render();
        });
    });
    this.render();
}

getFilteredData() {
    return this.data
        .filter((item) => this.category === "all" || item.category === this.category)
        .filter((item) => item.name.toLowerCase().includes(this.search))
        .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
}

render() {
    const items = this.getFilteredData();
    const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
    this.page = Math.min(this.page, totalPages);
    const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
    this.grid.innerHTML = pageItems.map((item) => `
        <article class="product-card">
            <h3>${item.name}</h3>
            <p class="muted">Categoria: ${item.category}</p>
            <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
${item.rating}</span></div>
        </article>
    `).join("") || `<p class="muted">No hay resultados.</p>`;
    this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1 ===
this.page ? "is-active" : ""}" data-page="${i + 1}">${i + 1}</button>`).join("");
    $('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
}
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

40. Tabla ordenable

Datos, búsqueda, tablas y almacenamiento ID 40

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginacion, tablas y almacenamiento local. La logica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	data-table	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<table class="data-table" id="demoTable">
  <thead>
    <tr>
      <th data-sort-table="name">Nombre</th>
      <th data-sort-table="role">Rol</th>
      <th data-sort-table="score">Puntaje</th>
    </tr>
```

```

        </thead>
        <tbody>
          <tr><td>Carlos</td><td>Frontend</td><td>92</td></tr>
          <tr><td>Laura</td><td>UX</td><td>88</td></tr>
          <tr><td>Valeria</td><td>Backend</td><td>95</td></tr>
          <tr><td>Juan</td><td>QA</td><td>84</td></tr>
        </tbody>
      </table>

```

styles.css - CSS asociado validado

```

.data-table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 14px;
}

.data-table th,
.data-table td {
  padding: 12px;
  border-bottom: 1px solid var(--border);
  text-align: left;
}

.data-table th {
  cursor: pointer;
  color: #93c5fd;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class SortableTable {
  constructor(tableId, searchId) {
    this.table = document.getElementById(tableId);
    this.search = document.getElementById(searchId);
    this.direction = 1;
    this.init();
  }

  init() {
    $(''[data-sort-table]', this.table).forEach((header, index) => {
      header.addEventListener("click", () => this.sort(index));
    });
    this.search.addEventListener("input", () => this.filter());
  }

  sort(index) {
    const rows = $('tbody tr', this.table);
    rows.sort((a, b) => a.children[index].textContent.localeCompare(b.children[index].textContent,
    undefined, { numeric: true }) * this.direction);
    this.direction *= -1;
    rows.forEach((row) => this.table.tBodies[0].appendChild(row));
  }

  filter() {
    const term = this.search.value.toLowerCase();
    $('tbody tr', this.table).forEach((row) => row.style.display =
    row.textContent.toLowerCase().includes(term) ? "" : "none");
  }
}

```

```
}  
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.  
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.  
import { initApp } from "../funciones.js";  
  
document.addEventListener("DOMContentLoaded", () => {  
  initApp();  
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];  
  
class SortableTable {  
  constructor(tableId, searchId) {  
    this.table = document.getElementById(tableId);  
    this.search = document.getElementById(searchId);  
    this.direction = 1;  
    this.init();  
  }  
  
  init() {  
    $$('#[data-sort-table]', this.table).forEach((header, index) => {  
      header.addEventListener("click", () => this.sort(index));  
    });  
    this.search.addEventListener("input", () => this.filter());  
  }  
  
  sort(index) {  
    const rows = $$('#tbody tr', this.table);  
    rows.sort((a, b) => a.children[index].textContent.localeCompare(b.children[index].textContent,  
undefined, { numeric: true }) * this.direction);  
    this.direction *= -1;  
    rows.forEach((row) => this.table.tBodies[0].appendChild(row));  
  }  
  
  filter() {  
    const term = this.search.value.toLowerCase();  
    $$('#tbody tr', this.table).forEach((row) => row.style.display =  
row.textContent.toLowerCase().includes(term) ? "" : "none");  
  }  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.

- La versión modular y normal no se cargan juntas.

41. Filtro de tabla

Datos, búsqueda, tablas y almacenamiento

ID 41

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	tableSearch, data-table	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<input id="tableSearch" type="search" placeholder="Filtrar tabla...">
<table class="data-table" id="demoTable">
  <thead>
    <tr>
      <th data-sort-table="name">Nombre</th>
      <th data-sort-table="role">Rol</th>
      <th data-sort-table="score">Puntaje</th>
```

```

        </tr>
      </thead>
      <tbody>
        <tr><td>Carlos</td><td>Frontend</td><td>92</td></tr>
        <tr><td>Laura</td><td>UX</td><td>88</td></tr>
        <tr><td>Valeria</td><td>Backend</td><td>95</td></tr>
        <tr><td>Juan</td><td>QA</td><td>84</td></tr>
      </tbody>
    </table>

```

styles.css - CSS asociado validado

```

.data-table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 14px;
}

.data-table th,
.data-table td {
  padding: 12px;
  border-bottom: 1px solid var(--border);
  text-align: left;
}

.data-table th {
  cursor: pointer;
  color: #93c5fd;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class SortableTable {
  constructor(tableId, searchId) {
    this.table = document.getElementById(tableId);
    this.search = document.getElementById(searchId);
    this.direction = 1;
    this.init();
  }

  init() {
    $$('#[data-sort-table]', this.table).forEach((header, index) => {
      header.addEventListener("click", () => this.sort(index));
    });
    this.search.addEventListener("input", () => this.filter());
  }

  sort(index) {
    const rows = $$("#tbody tr", this.table);
    rows.sort((a, b) => a.children[index].textContent.localeCompare(b.children[index].textContent,
undefined, { numeric: true }) * this.direction);
    this.direction *= -1;
    rows.forEach((row) => this.table.tBodies[0].appendChild(row));
  }

  filter() {
    const term = this.search.value.toLowerCase();
    $$("#tbody tr", this.table).forEach((row) => row.style.display =
row.textContent.toLowerCase().includes(term) ? "" : "none");
  }
}

```

```
}  
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.  
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.  
import { initApp } from "../funciones.js";  
  
document.addEventListener("DOMContentLoaded", () => {  
  initApp();  
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];  
  
class SortableTable {  
  constructor(tableId, searchId) {  
    this.table = document.getElementById(tableId);  
    this.search = document.getElementById(searchId);  
    this.direction = 1;  
    this.init();  
  }  
  
  init() {  
    $$('#[data-sort-table]', this.table).forEach((header, index) => {  
      header.addEventListener("click", () => this.sort(index));  
    });  
    this.search.addEventListener("input", () => this.filter());  
  }  
  
  sort(index) {  
    const rows = $$('#tbody tr', this.table);  
    rows.sort((a, b) => a.children[index].textContent.localeCompare(b.children[index].textContent,  
undefined, { numeric: true }) * this.direction);  
    this.direction *= -1;  
    rows.forEach((row) => this.table.tBodies[0].appendChild(row));  
  }  
  
  filter() {  
    const term = this.search.value.toLowerCase();  
    $$('#tbody tr', this.table).forEach((row) => row.style.display =  
row.textContent.toLowerCase().includes(term) ? "" : "none");  
  }  
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.

- La versión modular y normal no se cargan juntas.

42. Notas con localStorage

Datos, búsqueda, tablas y almacenamiento

ID 42

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	item-list, noteInput, noteList	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<textarea id="noteInput" rows="4" placeholder="Escribe una nota..."></textarea>
<button class="primary-button" id="saveNote">Guardar nota</button>
<ul class="item-list" id="noteList"></ul>
```

styles.css - CSS asociado validado

```
.item-list {
  padding: 0;
  list-style: none;
  display: grid;
  gap: 10px;
}

.item-list li {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  padding: 12px;
  border-radius: var(--radius-sm);
  background: rgba(255, 255, 255, 0.1);
  border: 1px solid var(--border);
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class NotesManager {
  constructor(toast) {
    this.input = $("#noteInput");
    this.saveButton = $("#saveNote");
    this.list = $("#noteList");
    this.exportButton = $("#exportNotes");
    this.toast = toast;
    this.notes = readJSON("kit-notes", []);
    this.init();
  }

  init() {
    this.saveButton.addEventListener("click", () => this.add());
    this.exportButton.addEventListener("click", () => this.export());
    this.render();
  }

  add() {
```

```

    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una nota primero.");
    this.notes.push({ id: Date.now(), text });
    saveJSON("kit-notes", this.notes);
    this.input.value = "";
    this.render();
    this.toast.show("Nota guardada.");
  }

  remove(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON("kit-notes", this.notes);
    this.render();
  }

  render() {
    this.list.innerHTML = this.notes.map((note) => `<li><span>${note.text}</span><button class="danger-button" data-id="${note.id}">Borrar</button></li>`).join("");
    $$('button', this.list).forEach((button) => button.addEventListener("click", () => this.remove(Number(button.dataset.id))));
  }

  export() {
    const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type: "application/json" });
    const link = document.createElement("a");
    link.href = URL.createObjectURL(blob);
    link.download = "notas.json";
    link.click();
    URL.revokeObjectURL(link.href);
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {}
}

```

```

        return fallback;
    }
}

class NotesManager {
    constructor(toast) {
        this.input = $("#noteInput");
        this.saveButton = $("#saveNote");
        this.list = $("#noteList");
        this.exportButton = $("#exportNotes");
        this.toast = toast;
        this.notes = readJSON("kit-notes", []);
        this.init();
    }

    init() {
        this.saveButton.addEventListener("click", () => this.add());
        this.exportButton.addEventListener("click", () => this.export());
        this.render();
    }

    add() {
        const text = this.input.value.trim();
        if (!text) return this.toast.show("Escribe una nota primero.");
        this.notes.push({ id: Date.now(), text });
        saveJSON("kit-notes", this.notes);
        this.input.value = "";
        this.render();
        this.toast.show("Nota guardada.");
    }

    remove(id) {
        this.notes = this.notes.filter((note) => note.id !== id);
        saveJSON("kit-notes", this.notes);
        this.render();
    }

    render() {
        this.list.innerHTML = this.notes.map((note) => `<li><span>${note.text}</span><button class="danger-button" data-id="${note.id}">Borrar</button></li>`).join("");
        $$('button', this.list).forEach((button) => button.addEventListener("click", () => this.remove(Number(button.dataset.id))));
    }

    export() {
        const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type: "application/json" });
        const link = document.createElement("a");
        link.href = URL.createObjectURL(blob);
        link.download = "notas.json";
        link.click();
        URL.revokeObjectURL(link.href);
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.

- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

43. Exportar notas a JSON

Datos, búsqueda, tablas y almacenamiento

ID 43

Descripción

Este bloque maneja datos en arrays, filtros, ordenamiento, paginación, tablas y almacenamiento local. La lógica depende de renderizar datos en el DOM.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	exportNotes, noteList	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="secondary-button" id="exportNotes">Exportar notas JSON</button>
<ul class="item-list" id="noteList"></ul>
```

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class NotesManager {
  constructor(toast) {
    this.input = $("#noteInput");
    this.saveButton = $("#saveNote");
    this.list = $("#noteList");
    this.exportButton = $("#exportNotes");
    this.toast = toast;
    this.notes = readJSON("kit-notes", []);
    this.init();
  }

  init() {
    this.saveButton.addEventListener("click", () => this.add());
    this.exportButton.addEventListener("click", () => this.export());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una nota primero.");
    this.notes.push({ id: Date.now(), text });
    saveJSON("kit-notes", this.notes);
    this.input.value = "";
    this.render();
    this.toast.show("Nota guardada.");
  }

  remove(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON("kit-notes", this.notes);
    this.render();
  }

  render() {
```

```

        this.list.innerHTML = this.notes.map((note) => `<li><span>${note.text}</span><button class="danger-
button" data-id="${note.id}">Borrar</button></li>`).join("");
        $('button', this.list).forEach((button) => button.addEventListener("click", () =>
this.remove(Number(button.dataset.id))));
    }

    export() {
        const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type: "application/json" });
        const link = document.createElement("a");
        link.href = URL.createObjectURL(blob);
        link.download = "notas.json";
        link.click();
        URL.revokeObjectURL(link.href);
    }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
    let timeoutId;
    return (...args) => {
        clearTimeout(timeoutId);
        timeoutId = setTimeout(() => callback(...args), delay);
    };
}

function saveJSON(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
    try {
        const value = localStorage.getItem(key);
        return value ? JSON.parse(value) : fallback;
    } catch {
        return fallback;
    }
}

class NotesManager {
    constructor(toast) {
        this.input = $("#noteInput");
        this.saveButton = $("#saveNote");
        this.list = $("#noteList");
        this.exportButton = $("#exportNotes");
        this.toast = toast;
        this.notes = readJSON("kit-notes", []);
        this.init();
    }

    init() {

```

```

        this.saveButton.addEventListener("click", () => this.add());
        this.exportButton.addEventListener("click", () => this.export());
        this.render();
    }

    add() {
        const text = this.input.value.trim();
        if (!text) return this.toast.show("Escribe una nota primero.");
        this.notes.push({ id: Date.now(), text });
        saveJSON("kit-notes", this.notes);
        this.input.value = "";
        this.render();
        this.toast.show("Nota guardada.");
    }

    remove(id) {
        this.notes = this.notes.filter((note) => note.id !== id);
        saveJSON("kit-notes", this.notes);
        this.render();
    }

    render() {
        this.list.innerHTML = this.notes.map((note) => `<li><span>${note.text}</span><button class="danger-button" data-id="${note.id}">Borrar</button></li>`).join("");
        $$('button', this.list).forEach((button) => button.addEventListener("click", () => this.remove(Number(button.dataset.id))));
    }

    export() {
        const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type: "application/json" });
        const link = document.createElement("a");
        link.href = URL.createObjectURL(blob);
        link.download = "notas.json";
        link.click();
        URL.revokeObjectURL(link.href);
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

44. Lista de tareas

Utilidades avanzadas y herramientas DOM

ID 44

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	todoInput, todoList, item-list	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="input-action">
  <input id="todoInput" placeholder="Nueva tarea">
  <button id="addTodo" type="button">Agregar</button>
</div>
<ul class="item-list draggable-list" id="todoList"></ul>
```

styles.css - CSS asociado validado

```
.item-list {
  padding: 0;
  list-style: none;
  display: grid;
  gap: 10px;
}

.item-list li {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  padding: 12px;
  border-radius: var(--radius-sm);
  background: rgba(255, 255, 255, 0.1);
  border: 1px solid var(--border);
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class TodoManager {
  constructor(toast) {
    this.input = $("#todoInput");
    this.addButton = $("#addTodo");
    this.clearButton = $("#clearTodos");
    this.list = $("#todoList");
    this.toast = toast;
    this.todos = readJSON("kit-todos", []);
    this.dragged = null;
    this.init();
  }

  init() {
    this.addButton.addEventListener("click", () => this.add());
    this.clearButton.addEventListener("click", () => this.clear());
    this.render();
  }
}
```

```

add() {
  const text = this.input.value.trim();
  if (!text) return this.toast.show("Escribe una tarea.");
  this.todos.push({ id: Date.now(), text, done: false });
  this.input.value = "";
  this.saveRender();
}

toggle(id) {
  this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
  this.saveRender();
}

remove(id) {
  this.todos = this.todos.filter((todo) => todo.id !== id);
  this.saveRender();
}

clear() {
  this.todos = [];
  this.saveRender();
  this.toast.show("Tareas borradas.");
}

saveRender() {
  saveJSON("kit-todos", this.todos);
  this.render();
}

render() {
  this.list.innerHTML = this.todos.map((todo) => `
    <li draggable="true" data-id="${todo.id}">
      <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>${
  {todo.done ? "✅" : "❌"} ${todo.text}</span></label>
      <button class="danger-button">Borrar</button>
    </li>
  `).join("");
  $$('li', this.list).forEach((li) => {
    const id = Number(li.dataset.id);
    $('input', li).addEventListener("change", () => this.toggle(id));
    $('button', li).addEventListener("click", () => this.remove(id));
    li.addEventListener("dragstart", () => this.dragged = li);
    li.addEventListener("dragover", (event) => event.preventDefault());
    li.addEventListener("drop", () => {
      if (this.dragged && this.dragged !== li) {
        this.list.insertBefore(this.dragged, li);
        this.todos = $$('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
Number(item.dataset.id)));
        saveJSON("kit-todos", this.todos);
      }
    });
  });
}
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class TodoManager {
  constructor(toast) {
    this.input = $("#todoInput");
    this.addButton = $("#addTodo");
    this.clearButton = $("#clearTodos");
    this.list = $("#todoList");
    this.toast = toast;
    this.todos = readJSON("kit-todos", []);
    this.dragged = null;
    this.init();
  }

  init() {
    this.addButton.addEventListener("click", () => this.add());
    this.clearButton.addEventListener("click", () => this.clear());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una tarea.");
    this.todos.push({ id: Date.now(), text, done: false });
    this.input.value = "";
    this.saveRender();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.saveRender();
  }

  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.saveRender();
  }

  clear() {
    this.todos = [];
    this.saveRender();
  }
}

```

```

        this.toast.show("Tareas borradas.");
    }

    saveRender() {
        saveJSON("kit-todos", this.todos);
        this.render();
    }

    render() {
        this.list.innerHTML = this.todos.map((todo) => `
            <li draggable="true" data-id="${todo.id}">
                <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>${
                    {todo.done ? "✅" : "❌"} ${todo.text}</span></label>
                <button class="danger-button">Borrar</button>
            </li>
        `).join("");
        $$('li', this.list).forEach((li) => {
            const id = Number(li.dataset.id);
            $('input', li).addEventListener("change", () => this.toggle(id));
            $('button', li).addEventListener("click", () => this.remove(id));
            li.addEventListener("dragstart", () => this.dragged = li);
            li.addEventListener("dragover", (event) => event.preventDefault());
            li.addEventListener("drop", () => {
                if (this.dragged && this.dragged !== li) {
                    this.list.insertBefore(this.dragged, li);
                    this.todos = $$('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
                        Number(item.dataset.id)));
                    saveJSON("kit-todos", this.todos);
                }
            });
        });
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

45. Drag and drop en tareas

Utilidades avanzadas y herramientas DOM

ID 45

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	draggable-list, drag	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<ul class="item-list draggable-list" id="todoList"></ul>
```

```
.draggable-list li {
  cursor: grab;
}
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class TodoManager {
  constructor(toast) {
    this.input = $("#todoInput");
    this.addButton = $("#addTodo");
    this.clearButton = $("#clearTodos");
    this.list = $("#todoList");
    this.toast = toast;
    this.todos = readJSON("kit-todos", []);
    this.dragged = null;
    this.init();
  }

  init() {
    this.addButton.addEventListener("click", () => this.add());
    this.clearButton.addEventListener("click", () => this.clear());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una tarea.");
    this.todos.push({ id: Date.now(), text, done: false });
    this.input.value = "";
    this.saveRender();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.saveRender();
  }

  remove(id) {
```

```

    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.saveRender();
  }

  clear() {
    this.todos = [];
    this.saveRender();
    this.toast.show("Tareas borradas.");
  }

  saveRender() {
    saveJSON("kit-todos", this.todos);
    this.render();
  }

  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li draggable="true" data-id="${todo.id}">
        <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>${
        {todo.done ? "✅" : "❌"} ${todo.text}</span></label>
        <button class="danger-button">Borrar</button>
      </li>
    `).join("");
    $$('li', this.list).forEach((li) => {
      const id = Number(li.dataset.id);
      $('input', li).addEventListener("change", () => this.toggle(id));
      $('button', li).addEventListener("click", () => this.remove(id));
      li.addEventListener("dragstart", () => this.dragged = li);
      li.addEventListener("dragover", (event) => event.preventDefault());
      li.addEventListener("drop", () => {
        if (this.dragged && this.dragged !== li) {
          this.list.insertBefore(this.dragged, li);
          this.todos = $$('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
          Number(item.dataset.id)));
          saveJSON("kit-todos", this.todos);
        }
      });
    });
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

```

```

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class TodoManager {
  constructor(toast) {
    this.input = $("#todoInput");
    this.addButton = $("#addTodo");
    this.clearButton = $("#clearTodos");
    this.list = $("#todoList");
    this.toast = toast;
    this.todos = readJSON("kit-todos", []);
    this.dragged = null;
    this.init();
  }

  init() {
    this.addButton.addEventListener("click", () => this.add());
    this.clearButton.addEventListener("click", () => this.clear());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una tarea.");
    this.todos.push({ id: Date.now(), text, done: false });
    this.input.value = "";
    this.saveRender();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.saveRender();
  }

  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.saveRender();
  }

  clear() {
    this.todos = [];
    this.saveRender();
    this.toast.show("Tareas borradas.");
  }

  saveRender() {
    saveJSON("kit-todos", this.todos);
    this.render();
  }

  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li draggable="true" data-id="${todo.id}">
        <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>${
        todo.done ? "✅" : "❌" } ${todo.text}</span></label>
        <button class="danger-button">Borrar</button>
    `);
  }
}

```

```

    </li>
  `).join("");
  $('li', this.list).forEach((li) => {
    const id = Number(li.dataset.id);
    $('input', li).addEventListener("change", () => this.toggle(id));
    $('button', li).addEventListener("click", () => this.remove(id));
    li.addEventListener("dragstart", () => this.dragged = li);
    li.addEventListener("dragover", (event) => event.preventDefault());
    li.addEventListener("drop", () => {
      if (this.dragged && this.dragged !== li) {
        this.list.insertBefore(this.dragged, li);
        this.todos = $('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
Number(item.dataset.id)));
        saveJSON("kit-todos", this.todos);
      }
    });
  });
}
}
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

46. Borrar tareas

Utilidades avanzadas y herramientas DOM

ID 46

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	clearTodos, danger-button	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="danger-button" id="clearTodos">Borrar tareas</button>
```

styles.css - CSS asociado validado

```
.nav-link,  
.icon-button,
```

```

.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
  border: 1px solid var(--border);
  color: var(--text);
  background: rgba(255, 255, 255, 0.1);
  border-radius: 999px;
  padding: 10px 14px;
  transition: transform var(--transition), background var(--transition), border-color var(--transition), box-
shadow var(--transition);
}

.primary-button:hover,
.danger-button:hover {
  transform: translateY(-2px);
}

.danger-button {
  background: rgba(239, 68, 68, 0.9);
  color: #ffffff;
  border-color: transparent;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class TodoManager {
  constructor(toast) {
    this.input = $("#todoInput");
    this.addButton = $("#addTodo");
    this.clearButton = $("#clearTodos");
    this.list = $("#todoList");
    this.toast = toast;
  }
}

```

```

    this.todos = readJSON("kit-todos", []);
    this.dragged = null;
    this.init();
  }

  init() {
    this.addButton.addEventListener("click", () => this.add());
    this.clearButton.addEventListener("click", () => this.clear());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una tarea.");
    this.todos.push({ id: Date.now(), text, done: false });
    this.input.value = "";
    this.saveRender();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.saveRender();
  }

  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.saveRender();
  }

  clear() {
    this.todos = [];
    this.saveRender();
    this.toast.show("Tareas borradas.");
  }

  saveRender() {
    saveJSON("kit-todos", this.todos);
    this.render();
  }

  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li draggable="true" data-id="${todo.id}">
        <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>${
    {todo.done ? "✅" : "❑"} ${todo.text}</span></label>
        <button class="danger-button">Borrar</button>
      </li>
    `).join("");
    $$('li', this.list).forEach((li) => {
      const id = Number(li.dataset.id);
      $('input', li).addEventListener("change", () => this.toggle(id));
      $('button', li).addEventListener("click", () => this.remove(id));
      li.addEventListener("dragstart", () => this.dragged = li);
      li.addEventListener("dragover", (event) => event.preventDefault());
      li.addEventListener("drop", () => {
        if (this.dragged && this.dragged !== li) {
          this.list.insertBefore(this.dragged, li);
          this.todos = $$('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
Number(item.dataset.id)));
          saveJSON("kit-todos", this.todos);
        }
      });
    });
  }
}

```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class TodoManager {
  constructor(toast) {
    this.input = $("#todoInput");
    this.addButton = $("#addTodo");
    this.clearButton = $("#clearTodos");
    this.list = $("#todoList");
    this.toast = toast;
    this.todos = readJSON("kit-todos", []);
    this.dragged = null;
    this.init();
  }

  init() {
    this.addButton.addEventListener("click", () => this.add());
    this.clearButton.addEventListener("click", () => this.clear());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una tarea.");
    this.todos.push({ id: Date.now(), text, done: false });
    this.input.value = "";
    this.saveRender();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
  }
}
```

```

        this.saveRender();
    }

    remove(id) {
        this.todos = this.todos.filter((todo) => todo.id !== id);
        this.saveRender();
    }

    clear() {
        this.todos = [];
        this.saveRender();
        this.toast.show("Tareas borradas.");
    }

    saveRender() {
        saveJSON("kit-todos", this.todos);
        this.render();
    }

    render() {
        this.list.innerHTML = this.todos.map((todo) => `
            <li draggable="true" data-id="${todo.id}">
                <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>${
                    {todo.done ? "✅" : "❑"} ${todo.text}</span></label>
                <button class="danger-button">Borrar</button>
            </li>
        `).join("");
        $$('li', this.list).forEach((li) => {
            const id = Number(li.dataset.id);
            $('input', li).addEventListener("change", () => this.toggle(id));
            $('button', li).addEventListener("click", () => this.remove(id));
            li.addEventListener("dragstart", () => this.dragged = li);
            li.addEventListener("dragover", (event) => event.preventDefault());
            li.addEventListener("drop", () => {
                if (this.dragged && this.dragged !== li) {
                    this.list.insertBefore(this.dragged, li);
                    this.todos = $$('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
                        Number(item.dataset.id)));
                    saveJSON("kit-todos", this.todos);
                }
            });
        });
    }
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

47. Conversor con botones C/F/K

Utilidades avanzadas y herramientas DOM

ID 47

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	temperature-tool, unit-button, result-box, unit-celsius, unit-fahrenheit, unit-kelvin	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="glass-panel temperature-tool" id="temperatureTool">
  <div class="unit-buttons">
    <button class="unit-button is-active" data-unit="celsius">C</button>
```

```

        <button class="unit-button" data-unit="fahrenheit">F</button>
        <button class="unit-button" data-unit="kelvin">K</button>
    </div>

    <label for="temperatureInput">Temperatura</label>
    <input id="temperatureInput" type="number" placeholder="Ingresa un valor">
    <button class="primary-button" id="calculateTemperature">Calcular</button>
    <p id="temperatureMessage" class="muted"></p>
    <div class="result-box" id="temperatureResult">Selecciona una unidad y calcula.</div>

```

styles.css - CSS asociado validado

```

body.unit-celsius {
    background: radial-gradient(circle at top left, rgba(248, 113, 113, 0.6), transparent 35%), linear-
gradient(135deg, #7f1d1d, #111827);
}

body.unit-fahrenheit {
    background: radial-gradient(circle at top left, rgba(96, 165, 250, 0.65), transparent 35%), linear-
gradient(135deg, #1e3a8a, #111827);
}

body.unit-kelvin {
    background: radial-gradient(circle at top left, rgba(74, 222, 128, 0.6), transparent 35%), linear-
gradient(135deg, #14532d, #111827);
}

.navbar-links,
.navbar-actions,
.hero-actions,
.badge-row,
.carousel_actions,
.utilities-row,
.unit-buttons,
.chips,
.pagination {
    display: flex;
    align-items: center;
    gap: 10px;
    flex-wrap: wrap;
}

.temperature-tool {
    max-width: 560px;
}

.unit-button {
    width: 64px;
    height: 64px;
    border: 1px solid var(--border);
    border-radius: 20px;
    color: #ffffff;
    background: rgba(255, 255, 255, 0.12);
    font-size: 22px;
    font-weight: 900;
}

.unit-button.is-active[data-unit="celsius"] {
    background: linear-gradient(135deg, #ef4444, #991b1b);
}

.unit-button.is-active[data-unit="fahrenheit"] {
    background: linear-gradient(135deg, #3b82f6, #1e3a8a);
}

```

```

.unit-button.is-active[data-unit="kelvin"] {
  background: linear-gradient(135deg, #22c55e, #14532d);
}

.result-box {
  margin-top: 18px;
  padding: 18px;
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid var(--border);
  line-height: 1.7;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }

  selectUnit(unit) {
    this.selectedUnit = unit;
    this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    document.body.classList.add(`unit-${unit}`);
    this.message.textContent = `Unidad seleccionada: ${unit}`;
  }

  calculate() {
    const rawValue = this.input.value.trim();

    if (rawValue === "") {
      this.message.textContent = "El campo esta vacio. Ingresa una temperatura para convertir.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    const value = Number(rawValue);
    if (!Number.isFinite(value)) {
      this.message.textContent = "Ingresa un numero valido.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    let celsius;

```

```

    if (this.selectedUnit === "celsius") celsius = value;
    if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
    if (this.selectedUnit === "kelvin") celsius = value - 273.15;

    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;

    this.result.innerHTML = `
      <strong>Resultado desde ${this.selectedUnit}</strong><br>
      Celsius: ${celsius.toFixed(2)} °C<br>
      Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
      Kelvin: ${kelvin.toFixed(2)} K
    `;
    this.toast?.show("Temperatura calculada.");
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }

  selectUnit(unit) {
    this.selectedUnit = unit;
    this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    document.body.classList.add(`unit-${unit}`);
    this.message.textContent = `Unidad seleccionada: ${unit}`;
  }

  calculate() {
    const rawValue = this.input.value.trim();

    if (rawValue === "") {

```

```

        this.message.textContent = "El campo esta vacio. Ingresar una temperatura para convertir.";
        this.result.textContent = "No se pudo calcular.";
        return;
    }

    const value = Number(rawValue);
    if (!Number.isFinite(value)) {
        this.message.textContent = "Ingresa un numero valido.";
        this.result.textContent = "No se pudo calcular.";
        return;
    }

    let celsius;
    if (this.selectedUnit === "celsius") celsius = value;
    if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
    if (this.selectedUnit === "kelvin") celsius = value - 273.15;

    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;

    this.result.innerHTML = `
        <strong>Resultado desde ${this.selectedUnit}</strong><br>
        Celsius: ${celsius.toFixed(2)} °C<br>
        Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
        Kelvin: ${kelvin.toFixed(2)} K
    `;
    this.toast?.show("Temperatura calculada.");
}
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

48. Cambio de fondo según unidad

Utilidades avanzadas y herramientas DOM

ID 48

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	unit-celsius, unit-fahrenheit, unit-kelvin, temperature-tool	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="glass-panel temperature-tool" id="temperatureTool">
  <div class="unit-buttons">
    <button class="unit-button is-active" data-unit="celsius">C</button>
    <button class="unit-button" data-unit="fahrenheit">F</button>
    <button class="unit-button" data-unit="kelvin">K</button>
  </div>
```

```

<label for="temperatureInput">Temperatura</label>
<input id="temperatureInput" type="number" placeholder="Ingresa un valor">
<button class="primary-button" id="calculateTemperature">Calcular</button>
<p id="temperatureMessage" class="muted"></p>
<div class="result-box" id="temperatureResult">Selecciona una unidad y calcula.</div>

```

styles.css - CSS asociado validado

```

body.unit-celsius {
  background: radial-gradient(circle at top left, rgba(248, 113, 113, 0.6), transparent 35%), linear-
gradient(135deg, #7f1d1d, #111827);
}

body.unit-fahrenheit {
  background: radial-gradient(circle at top left, rgba(96, 165, 250, 0.65), transparent 35%), linear-
gradient(135deg, #1e3a8a, #111827);
}

body.unit-kelvin {
  background: radial-gradient(circle at top left, rgba(74, 222, 128, 0.6), transparent 35%), linear-
gradient(135deg, #14532d, #111827);
}

.temperature-tool {
  max-width: 560px;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }

  selectUnit(unit) {
    this.selectedUnit = unit;
    this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    document.body.classList.add(`unit-${unit}`);
    this.message.textContent = `Unidad seleccionada: ${unit}`;
  }

  calculate() {
    const rawValue = this.input.value.trim();

```

```

    if (rawValue === "") {
      this.message.textContent = "El campo esta vacio. Ingresar una temperatura para convertir.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    const value = Number(rawValue);
    if (!Number.isFinite(value)) {
      this.message.textContent = "Ingresar un numero valido.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    let celsius;
    if (this.selectedUnit === "celsius") celsius = value;
    if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
    if (this.selectedUnit === "kelvin") celsius = value - 273.15;

    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;

    this.result.innerHTML = `
      <strong>Resultado desde ${this.selectedUnit}</strong><br>
      Celsius: ${celsius.toFixed(2)} °C<br>
      Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
      Kelvin: ${kelvin.toFixed(2)} K
    `;
    this.toast?.show("Temperatura calculada.");
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }
}

```

```

}

selectUnit(unit) {
  this.selectedUnit = unit;
  this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
  document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
  document.body.classList.add(`unit-${unit}`);
  this.message.textContent = `Unidad seleccionada: ${unit}`;
}

calculate() {
  const rawValue = this.input.value.trim();

  if (rawValue === "") {
    this.message.textContent = "El campo esta vacio. Ingresar una temperatura para convertir.";
    this.result.textContent = "No se pudo calcular.";
    return;
  }

  const value = Number(rawValue);
  if (!Number.isFinite(value)) {
    this.message.textContent = "Ingresar un numero valido.";
    this.result.textContent = "No se pudo calcular.";
    return;
  }

  let celsius;
  if (this.selectedUnit === "celsius") celsius = value;
  if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
  if (this.selectedUnit === "kelvin") celsius = value - 273.15;

  const fahrenheit = (celsius * 9 / 5) + 32;
  const kelvin = celsius + 273.15;

  this.result.innerHTML = `
    <strong>Resultado desde ${this.selectedUnit}</strong><br>
    Celsius: ${celsius.toFixed(2)} °C<br>
    Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
    Kelvin: ${kelvin.toFixed(2)} K
  `;
  this.toast?.show("Temperatura calculada.");
}
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

49. Cálculo dinámico de temperatura

Utilidades avanzadas y herramientas DOM

ID 49

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	temperature-tool, temperatureResult, calculateTemperature	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="glass-panel temperature-tool" id="temperatureTool">
  <div class="unit-buttons">
    <button class="unit-button is-active" data-unit="celsius">C</button>
    <button class="unit-button" data-unit="fahrenheit">F</button>
```

```

        <button class="unit-button" data-unit="kelvin">K</button>
      </div>

      <label for="temperatureInput">Temperatura</label>
      <input id="temperatureInput" type="number" placeholder="Ingresa un valor">
      <button class="primary-button" id="calculateTemperature">Calcular</button>
      <p id="temperatureMessage" class="muted"></p>
      <div class="result-box" id="temperatureResult">Selecciona una unidad y calcula.</div>

```

styles.css - CSS asociado validado

```

.temperature-tool {
  max-width: 560px;
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }

  selectUnit(unit) {
    this.selectedUnit = unit;
    this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    document.body.classList.add(`unit-${unit}`);
    this.message.textContent = `Unidad seleccionada: ${unit}`;
  }

  calculate() {
    const rawValue = this.input.value.trim();

    if (rawValue === "") {
      this.message.textContent = "El campo esta vacio. Ingresa una temperatura para convertir.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    const value = Number(rawValue);
    if (!Number.isFinite(value)) {
      this.message.textContent = "Ingresa un numero valido.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }
  }
}

```

```

    let celsius;
    if (this.selectedUnit === "celsius") celsius = value;
    if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
    if (this.selectedUnit === "kelvin") celsius = value - 273.15;

    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;

    this.result.innerHTML = `
      <strong>Resultado desde ${this.selectedUnit}</strong><br>
      Celsius: ${celsius.toFixed(2)} °C<br>
      Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
      Kelvin: ${kelvin.toFixed(2)} K
    `;
    this.toast?.show("Temperatura calculada.");
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }

  selectUnit(unit) {
    this.selectedUnit = unit;
    this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    document.body.classList.add(`unit-${unit}`);
    this.message.textContent = `Unidad seleccionada: ${unit}`;
  }

  calculate() {
    const rawValue = this.input.value.trim();

```

```

    if (rawValue === "") {
        this.message.textContent = "El campo esta vacio. Ingrese una temperatura para convertir.";
        this.result.textContent = "No se pudo calcular.";
        return;
    }

    const value = Number(rawValue);
    if (!Number.isFinite(value)) {
        this.message.textContent = "Ingresa un numero valido.";
        this.result.textContent = "No se pudo calcular.";
        return;
    }

    let celsius;
    if (this.selectedUnit === "celsius") celsius = value;
    if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
    if (this.selectedUnit === "kelvin") celsius = value - 273.15;

    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;

    this.result.innerHTML = `
        <strong>Resultado desde ${this.selectedUnit}</strong><br>
        Celsius: ${celsius.toFixed(2)} °C<br>
        Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
        Kelvin: ${kelvin.toFixed(2)} K
    `;
    this.toast?.show("Temperatura calculada.");
}
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

50. Imprimir página

Utilidades avanzadas y herramientas DOM

ID 50

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	printPage	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="secondary-button" id="printPage">Imprimir pagina</button>
```

styles.css - CSS asociado validado

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class UtilityActions {
  constructor(toast) {
    this.toast = toast;
    this.init();
  }

  init() {
    $('[data-toast]').forEach((button) => button.addEventListener("click", () =>
this.toast.show(button.dataset.toast)));
    $("#printPage").addEventListener("click", () => window.print());
    $("#resetDemo").addEventListener("click", () => {
      document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
      this.toast.show("Demo visual reiniciada.");
    });
    document.addEventListener("custom-select:change", (event) => {
      const target = $("#selectedProductText");
      if (target) target.textContent = event.detail.value;
    });
  }
}
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class UtilityActions {
  constructor(toast) {
    this.toast = toast;
    this.init();
  }

  init() {
    $('[data-toast]').forEach((button) => button.addEventListener("click", () =>
this.toast.show(button.dataset.toast)));
    $("#printPage").addEventListener("click", () => window.print());
    $("#resetDemo").addEventListener("click", () => {
      document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
      this.toast.show("Demo visual reiniciada.");
    });
    document.addEventListener("custom-select:change", (event) => {
      const target = $("#selectedProductText");
      if (target) target.textContent = event.detail.value;
    });
  }
}
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

51. Reset visual de demo

Utilidades avanzadas y herramientas DOM

ID 51

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	resetDemo, unit-celsius, unit-fahrenheit, unit-kelvin	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<button class="secondary-button" id="resetDemo">Reiniciar demo visual</button>
```

styles.css - CSS asociado validado

```
body.unit-celsius {  
  background: radial-gradient(circle at top left, rgba(248, 113, 113, 0.6), transparent 35%), linear-
```

```

gradient(135deg, #7f1d1d, #111827);
}

body.unit-fahrenheit {
  background: radial-gradient(circle at top left, rgba(96, 165, 250, 0.65), transparent 35%), linear-
gradient(135deg, #1e3a8a, #111827);
}

body.unit-kelvin {
  background: radial-gradient(circle at top left, rgba(74, 222, 128, 0.6), transparent 35%), linear-
gradient(135deg, #14532d, #111827);
}

```

funciones.js - versión modular

```

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class UtilityActions {
  constructor(toast) {
    this.toast = toast;
    this.init();
  }

  init() {
    $('[data-toast]').forEach((button) => button.addEventListener("click", () =>
this.toast.show(button.dataset.toast)));
    $("#printPage").addEventListener("click", () => window.print());
    $("#resetDemo").addEventListener("click", () => {
      document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
      this.toast.show("Demo visual reiniciada.");
    });
    document.addEventListener("custom-select:change", (event) => {
      const target = $("#selectedProductText");
      if (target) target.textContent = event.detail.value;
    });
  }
}

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);

const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class UtilityActions {
  constructor(toast) {
    this.toast = toast;
    this.init();
  }

  init() {
    $('[data-toast]').forEach((button) => button.addEventListener("click", () =>

```

```

this.toast.show(button.dataset.toast));
$("#printPage").addEventListener("click", () => window.print());
$("#resetDemo").addEventListener("click", () => {
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    this.toast.show("Demo visual reiniciada.");
});
document.addEventListener("custom-select:change", (event) => {
    const target = $("#selectedProductText");
    if (target) target.textContent = event.detail.value;
});
}
}

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

52. Helpers reutilizables para DOM, storage y debounce

Utilidades avanzadas y herramientas DOM

ID 52

Descripción

Este bloque incluye herramientas practicas como tareas, drag and drop, conversor de temperatura, impresion, reset y helpers reutilizables.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	No CSS propio	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<!-- No requiere HTML propio. Son helpers de JavaScript usados por otros componentes. -->
```

styles.css - CSS asociado validado

```
/* Este componente usa estilos globales del proyecto. Consulta el Anexo A para styles.css completo. */
```

funciones.js - versión modular

```
export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export const $ = (selector, parent = document) => parent.querySelector(selector);

export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}
```

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

53. Tarjeta de presentación Bootstrap como actividad guiada

Actividades guiadas y componentes finales

ID 53

Descripción

Actividad guiada para construir una tarjeta de presentación personal con estructura tipo Bootstrap, estilo propio y vista alterna de tecnologías.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	profile-lab, profile-lab__flip, profileLabFlip, profile-lab__face profile-lab__face--front, profile-lab__avatar, profile-lab__role	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<article class="profile-lab">
  <div class="profile-lab__flip" id="profileLabFlip">
    <section class="profile-lab__face profile-lab__face--front">
      <div class="profile-lab__avatar">CM</div>
      <h4>Carlos Madero</h4>
      <p class="profile-lab__role">Full Stack Java Developer</p>
      <button id="showProfileTech">Ver tecnologías</button>
    </section>
    <section class="profile-lab__face profile-lab__face--back">
      <h4>Tecnologías</h4>
      <button id="hideProfileTech">Volver</button>
    </section>
  </div>
</article>
```

styles.css - CSS asociado validado

```
.profile-lab {
  max-width: 340px;
  perspective: 1200px;
}
.profile-lab__flip {
  position: relative;
  min-height: 320px;
  transform-style: preserve-3d;
  transition: transform 0.8s ease;
}
.profile-lab__flip.is-flipped {
  transform: rotateY(180deg);
}
.profile-lab__face {
  position: absolute;
  inset: 0;
  display: grid;
  gap: 12px;
  align-content: start;
  padding: 24px;
  border: 1px solid #cbd5e1;
  border-radius: 20px;
  background: #ffffff;
  backface-visibility: hidden;
}
.profile-lab__face--back {
  transform: rotateY(180deg);
}
.profile-lab__avatar {
  width: 64px;
  height: 64px;
  display: grid;
  place-items: center;
  border-radius: 50%;
  background: #2563eb;
  color: #ffffff;
  font-weight: 700;
}
```

funciones.js - versión modular

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class ProfilePresentationActivity {
  constructor() {
    this.container = $("#profileLabFlip");
    this.showButton = $("#showProfileTech");
    this.hideButton = $("#hideProfileTech");
    if (this.container && this.showButton && this.hideButton) {
      this.init();
    }
  }

  init() {
    this.showButton.addEventListener("click", () => this.container.classList.add("is-flipped"));
    this.hideButton.addEventListener("click", () => this.container.classList.remove("is-flipped"));
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new ProfilePresentationActivity();
});
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "./funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ProfilePresentationActivity {
  constructor() {
    this.container = $("#profileLabFlip");
    this.showButton = $("#showProfileTech");
    this.hideButton = $("#hideProfileTech");
    if (this.container && this.showButton && this.hideButton) {
      this.init();
    }
  }

  init() {
    this.showButton.addEventListener("click", () => this.container.classList.add("is-flipped"));
    this.hideButton.addEventListener("click", () => this.container.classList.remove("is-flipped"));
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new ProfilePresentationActivity();
});
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

54. Carrito de compras DOM como actividad guiada

Actividades guiadas y componentes finales

ID 54

Descripción

Actividad guiada para practicar DOM y eventos con una mini tienda: agregar productos, actualizar badge, calcular total, eliminar ítems y vaciar carrito.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	shop-lab, shop-lab__topbar, shop-lab__badge, shopBadge, shop-lab__grid, shop-product	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="shop-lab">
  <div class="shop-lab__topbar">
    <strong>Mini tienda</strong>
    <span class="shop-lab__badge" id="shopBadge">0</span>
  </div>
  <div class="shop-lab__grid">
    <article class="shop-product">
      <h4>Pack IPA</h4>
      <p>$18,000</p>
      <button class="shop-add-button" type="button" data-name="Pack IPA" data-price="18000">Agregar</
button>
    </article>
    <article class="shop-product">
      <h4>Cheesecake</h4>
      <p>$11,000</p>
      <button class="shop-add-button" type="button" data-name="Cheesecake" data-price="11000">Agregar</
button>
    </article>
  </div>
  <div class="shop-cart">
    <div class="shop-cart__head">
      <h4>Carrito</h4>
      <button id="clearShopCart" type="button">Vaciar</button>
    </div>
    <p id="shopEmptyMessage">Tu carrito esta vacio.</p>
    <ul id="shopCartList"></ul>
    <strong id="shopCartTotal">$0</strong>
  </div>
</div>
<div id="toastContainer"></div>
```

styles.css - CSS asociado validado

```
.shop-lab {
  display: grid;
  gap: 18px;
}
.shop-lab__topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.shop-lab__badge {
  min-width: 34px;
  padding: 6px 10px;
  border-radius: 999px;
  background: #2563eb;
  color: #ffffff;
  text-align: center;
}
.shop-lab__grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 14px;
}
.shop-product,
.shop-cart {
  padding: 16px;
  border: 1px solid #cbd5e1;
  border-radius: 16px;
}
```

```

.shop-cart__item {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  padding: 12px 0;
  border-bottom: 1px solid #e2e8f0;
}
.danger-button {
  border: 0;
  border-radius: 10px;
  padding: 8px 12px;
  background: #ef4444;
  color: #ffffff;
  cursor: pointer;
}
.muted {
  color: rgba(15, 23, 42, 0.72);
}

```

funciones.js - versión modular

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

export class ShoppingCartActivity {
  constructor(toast) {
    this.toast = toast;
    this.buttons = $$(".shop-add-button");
    this.list = $("#shopCartList");
    this.badge = $("#shopBadge");
    this.total = $("#shopCartTotal");
    this.emptyMessage = $("#shopEmptyMessage");
    this.clearButton = $("#clearShopCart");
    this.itemsCount = 0;
    this.totalAmount = 0;
    if (this.buttons.length && this.list && this.badge && this.total && this.emptyMessage &&
this.clearButton) {
      this.init();
    }
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.addItem(button.dataset.name, Number(button.dataset.price));
      });
    });
    this.clearButton.addEventListener("click", () => this.clear());
    this.renderSummary();
  }
}

```

```

addItem(name, price) {
  const item = document.createElement("li");
  item.className = "shop-cart__item";
  item.innerHTML = `
    <div>
      <strong>${name}</strong>
      <p class="muted">${this.formatPrice(price)}</p>
    </div>
    <button type="button" class="danger-button">x</button>
  `;

  $("button", item)?.addEventListener("click", () => this.removeItem(item, price));
  this.list.appendChild(item);
  this.itemsCount += 1;
  this.totalAmount += price;
  this.renderSummary();
  this.toast?.show(`${name} agregado al carrito.`);
}

removeItem(item, price) {
  item.remove();
  this.itemsCount = Math.max(0, this.itemsCount - 1);
  this.totalAmount = Math.max(0, this.totalAmount - price);
  this.renderSummary();
}

clear() {
  $$("li", this.list).forEach((item) => item.remove());
  this.itemsCount = 0;
  this.totalAmount = 0;
  this.renderSummary();
  this.toast?.show("Carrito vaciado.");
}

renderSummary() {
  this.badge.textContent = String(this.itemsCount);
  this.total.textContent = this.formatPrice(this.totalAmount);
  this.emptyMessage.style.display = this.itemsCount ? "none" : "block";
}

formatPrice(value) {
  return `$$${value.toLocaleString("es-CO")}`;
}
}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new ShoppingCartActivity(toast);
});

```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

```

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

class ShoppingCartActivity {
  constructor(toast) {
    this.toast = toast;
    this.buttons = $(".shop-add-button");
    this.list = $("#shopCartList");
    this.badge = $("#shopBadge");
    this.total = $("#shopCartTotal");
    this.emptyMessage = $("#shopEmptyMessage");
    this.clearButton = $("#clearShopCart");
    this.itemsCount = 0;
    this.totalAmount = 0;
    if (this.buttons.length && this.list && this.badge && this.total && this.emptyMessage &&
this.clearButton) {
      this.init();
    }
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.addItem(button.dataset.name, Number(button.dataset.price));
      });
    });
    this.clearButton.addEventListener("click", () => this.clear());
    this.renderSummary();
  }

  addItem(name, price) {
    const item = document.createElement("li");
    item.className = "shop-cart__item";
    item.innerHTML = `
      <div>
        <strong>${name}</strong>
        <p class="muted">${this.formatPrice(price)}</p>
      </div>
      <button type="button" class="danger-button">x</button>
    `;

    $(".button", item)?.addEventListener("click", () => this.removeItem(item, price));
    this.list.appendChild(item);
    this.itemsCount += 1;
    this.totalAmount += price;
    this.renderSummary();
    this.toast?.show(`${name} agregado al carrito.`);
  }

  removeItem(item, price) {
    item.remove();
    this.itemsCount = Math.max(0, this.itemsCount - 1);
    this.totalAmount = Math.max(0, this.totalAmount - price);
    this.renderSummary();
  }
}

```

```

clear() {
  $$("li", this.list).forEach((item) => item.remove());
  this.itemsCount = 0;
  this.totalAmount = 0;
  this.renderSummary();
  this.toast?.show("Carrito vaciado.");
}

renderSummary() {
  this.badge.textContent = String(this.itemsCount);
  this.total.textContent = this.formatPrice(this.totalAmount);
  this.emptyMessage.style.display = this.itemsCount ? "none" : "block";
}

formatPrice(value) {
  return `$$${value.toLocaleString("es-CO")}`;
}
}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new ShoppingCartActivity(toast);
});

```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

55. Footer reutilizable como pie de página profesional

Actividades guiadas y componentes finales

ID 55

Descripción

Pie de pagina profesional con marca, enlaces de navegacion, contacto, redes sociales y barra de copyright.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	site-footer, siteFooter, site-footer__grid, site-footer__brand, brand__mark, muted	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<footer class="site-footer" id="siteFooter">
  <div class="site-footer__grid">
```

```

<div class="site-footer__brand">
  <span class="brand__mark">CW</span>
  <strong>Componentes Web</strong>
  <p class="muted">Banco de componentes reutilizables.</p>
</div>
<div class="site-footer__links">
  <strong>Secciones</strong>
  <nav>
    <a href="#inicio">Inicio</a>
    <a href="#componentes">Componentes</a>
    <a href="#guia">Guia</a>
  </nav>
</div>
<div class="site-footer__contact">
  <strong>Contacto</strong>
  <a href="mailto:email@ejemplo.com">Email</a>
  <a href="https://github.com">GitHub</a>
</div>
</div>
<div class="site-footer__bottom">
  <span>© 2026 Autor. Todos los derechos reservados.</span>
  <span>57 funcionalidades documentadas</span>
</div>
</footer>

```

styles.css - CSS asociado validado

```

.site-footer {
  margin-top: 40px;
  padding: 48px 32px 24px;
  background: #0f172a;
  color: #ffffff;
}
.site-footer__grid {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;
  gap: 32px;
}
.site-footer__links a,
.site-footer__contact a {
  display: block;
  color: rgba(255, 255, 255, 0.72);
  text-decoration: none;
  margin-top: 8px;
}
.site-footer__bottom {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  margin-top: 24px;
  padding-top: 24px;
  border-top: 1px solid rgba(255, 255, 255, 0.18);
}
@media (max-width: 720px) {
  .site-footer__grid,
  .site-footer__bottom {
    grid-template-columns: 1fr;
    flex-direction: column;
  }
}

```

funciones.js - versión modular

```
// No requiere JS. Los enlaces internos usan hash o se conectan con RouterSections.
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.  
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.  
import { initApp } from "../funciones.js";  
  
document.addEventListener("DOMContentLoaded", () => {  
  initApp();  
});
```

script-normal.js - versión sin módulos

```
// No requiere JS. Los enlaces internos usan hash o se conectan con RouterSections.
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

56. Tarjetas de precios Bootstrap como actividad guiada completa

Actividades guiadas y componentes finales ID 56

Descripción

Actividad guiada para maquetar una seccion de precios responsiva con tarjetas, badges y botones estilo Bootstrap.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	pricing-lab, pricing-lab__card, pricing-lab__badge, pricing-lab__price, pricing-lab__list, secondary-button	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="pricing-lab">
  <article class="pricing-lab__card">
```

```

    <span class="pricing-lab__badge">Basico</span>
    <h4>Starter</h4>
    <p class="pricing-lab__price">$19<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>1 proyecto</li>
      <li>Soporte por correo</li>
      <li>Componentes base</li>
    </ul>
    <button type="button" class="secondary-button">Elegir plan</button>
  </article>
  <article class="pricing-lab__card pricing-lab__card--featured">
    <span class="pricing-lab__badge">Popular</span>
    <h4>Pro</h4>
    <p class="pricing-lab__price">$49<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>10 proyectos</li>
      <li>Soporte prioritario</li>
      <li>Layouts premium</li>
    </ul>
    <button type="button" class="primary-button">Elegir plan</button>
  </article>
  <article class="pricing-lab__card">
    <span class="pricing-lab__badge">Equipo</span>
    <h4>Scale</h4>
    <p class="pricing-lab__price">$99<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>Proyectos ilimitados</li>
      <li>Equipo colaborativo</li>
      <li>Soporte dedicado</li>
    </ul>
    <button type="button" class="secondary-button">Elegir plan</button>
  </article>
</div>

```

styles.css - CSS asociado validado

```

.pricing-lab {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}
.pricing-lab__card {
  display: grid;
  gap: 14px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.06);
}
.pricing-lab__card--featured {
  background: linear-gradient(180deg, rgba(37, 99, 235, 0.2), rgba(255, 255, 255, 0.08));
  border-color: rgba(37, 99, 235, 0.5);
  box-shadow: 0 18px 36px rgba(37, 99, 235, 0.16);
}
[data-theme="light"] .pricing-lab__card {
  background: rgba(255, 255, 255, 0.82);
}
.pricing-lab__badge {
  display: inline-flex;
  width: fit-content;
  padding: 6px 10px;
  border-radius: 999px;
  background: rgba(37, 99, 235, 0.16);
  color: #bfdbfef;
  font-size: 12px;
}

```

```

    font-weight: 700;
  }
  [data-theme="light"] .pricing-lab__badge {
    color: #1d4ed8;
    background: rgba(37, 99, 235, 0.1);
  }
  .pricing-lab__price {
    margin: 0;
    font-size: 2rem;
    font-weight: 800;
    color: var(--text);
  }
  .pricing-lab__price span {
    font-size: 0.9rem;
    font-weight: 600;
    color: var(--muted);
  }
  .pricing-lab__list {
    display: grid;
    gap: 8px;
    margin: 0;
    padding-left: 18px;
    color: var(--muted);
  }
}

```

funciones.js - versión modular

```
// No requiere JS. Esta actividad es de maquetacion de componentes.
```

script.js - arranque modular

```

// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});

```

script-normal.js - versión sin módulos

```
// No requiere JS. Esta actividad es de maquetacion de componentes.
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

57. Barra de desplazamiento personalizada

Descripción

Estilo CSS moderno para personalizar barras de desplazamiento en navegadores Webkit y Firefox.

Dónde se usa normalmente: landing pages, dashboards, portafolios, formularios, tiendas, blogs, documentación técnica o paneles administrativos, según el componente.

Tabla de elementos, variables y propiedades

Elemento / Variable	Tipo / Selector	Función / Propósito	¿Se puede modificar?	Qué cambiar si lo adaptas
Contenedor o bloque HTML	custom-scrollable-container, custom-scrollable-content	Agrupar la funcionalidad visible.	Sí	Si cambias clases o ids, actualiza CSS y JS.
Clase de estado	is-active / is-open / hidden / clase equivalente	Activa o desactiva estados visuales.	Sí, con cuidado	Debe coincidir con classList en JavaScript.
Eventos JS	addEventListener	Detecta clic, input, submit, change, scroll o drag.	No eliminar	Sin eventos el componente queda estático.
CSS asociado	styles.css	Controla diseño, responsive y estados.	Sí	Mantén llaves completas y selectores alineados.
JS asociado	funciones.js / script-normal.js	Controla la interacción.	Sí	Usa modular o normal, no ambas.

Explicación detallada del código

HTML

El HTML define la estructura que JavaScript selecciona mediante ids, clases o atributos data-*. Copia el bloque completo para no perder botones, contenedores, mensajes o listas requeridas.

CSS

El CSS entregado está completo para esta funcionalidad o remite al anexo completo. Estado CSS: **balanceado**. Las clases de estado deben existir en CSS para que los cambios de JavaScript sean visibles.

JavaScript

La lógica selecciona elementos del DOM, registra eventos y actualiza texto, clases, estilos, localStorage o listas. Estado JS: **balanceado**.

Código fuente completo e integrado

index.html

```
<div class="custom-scrollable-container">
  <h3>Contenido desplazable</h3>
  <p>Desplaza este panel verticalmente para ver el estilo de la barra de movimiento personalizada en acción.</p>
</div>
<div class="custom-scrollable-content">
  <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
  <p>Vivamus id urna quis eros sodales accumsan quis ac mi.</p>
  <p>Proin sit amet elementum nulla. Aliquam convallis tellus sed leo congue feugiat.</p>
</div>
```

```
<p>Donec feugiat tellus et elementum hendrerit.</p>
</div>
</div>
```

styles.css - CSS asociado validado

```
.custom-scrollable-container {
  padding: 16px;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.04);
}
.custom-scrollable-content {
  max-height: 150px;
  overflow-y: auto;
  padding-right: 10px;
  scrollbar-width: thin;
  scrollbar-color: rgba(255, 255, 255, 0.18) rgba(255, 255, 255, 0.02);
}
.custom-scrollable-content::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}
.custom-scrollable-content::-webkit-scrollbar-track {
  background: rgba(255, 255, 255, 0.02);
  border-radius: 999px;
}
.custom-scrollable-content::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.18);
  border-radius: 999px;
}
.custom-scrollable-content::-webkit-scrollbar-thumb:hover {
  background: linear-gradient(180deg, var(--primary), var(--secondary));
}
```

funciones.js - versión modular

```
// No requiere JS. Estilo de scroll puro con CSS.
```

script.js - arranque modular

```
// En el proyecto real, script.js importa initApp() desde funciones.js.
// Para un componente aislado, importa su clase y crea una instancia en DOMContentLoaded.
import { initApp } from "../funciones.js";

document.addEventListener("DOMContentLoaded", () => {
  initApp();
});
```

script-normal.js - versión sin módulos

```
// No requiere JS. Estilo de scroll puro con CSS.
```

Cómo reciclar esta funcionalidad

1. Copia el HTML dentro del body.
2. Copia el CSS asociado o usa el styles.css completo del anexo.
3. Copia la clase JS indicada o usa el archivo funciones.js completo.
4. Conecta el script modular o normal.
5. Prueba en Live Server y revisa la consola.

Checklist de prueba

- El HTML aparece en pantalla.
- El CSS no está cortado y tiene llaves balanceadas.
- Los botones o eventos responden.
- No hay errores de selectores nulos en consola.
- La versión modular y normal no se cargan juntas.

Anexo A - styles.css completo del proyecto

Este anexo garantiza que ningún estudiante tenga CSS incompleto. Si un componente usa variables o clases globales, están aquí.

```
:root {
  --primary: #2563eb;
  --primary-dark: #1d4ed8;
  --secondary: #7c3aed;
  --success: #22c55e;
  --danger: #ef4444;
  --warning: #f59e0b;
  --bg: #0f172a;
  --bg-2: #1e293b;
  --text: #ffffff;
  --muted: rgba(255, 255, 255, 0.72);
  --glass: rgba(255, 255, 255, 0.14);
  --glass-strong: rgba(255, 255, 255, 0.2);
  --border: rgba(255, 255, 255, 0.22);
  --shadow: 0 25px 60px rgba(0, 0, 0, 0.35);
  --radius-lg: 24px;
  --radius-md: 16px;
  --radius-sm: 10px;
  --navbar-height: 76px;
  --transition: 0.22s ease;
}

[data-theme="light"] {
  --bg: #e0f2fe;
  --bg-2: #f8fafc;
  --text: #0f172a;
  --muted: rgba(15, 23, 42, 0.72);
  --glass: rgba(255, 255, 255, 0.68);
  --glass-strong: rgba(255, 255, 255, 0.86);
  --border: rgba(15, 23, 42, 0.12);
  --shadow: 0 25px 60px rgba(15, 23, 42, 0.18);
}

* {
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: Arial, Helvetica, sans-serif;
  color: var(--text);
  background:
    radial-gradient(circle at top left, rgba(56, 189, 248, 0.5), transparent 34%),
    radial-gradient(circle at bottom right, rgba(124, 58, 237, 0.5), transparent 34%),
    linear-gradient(135deg, var(--bg), var(--bg-2));
  transition: background var(--transition), color var(--transition);
}

body.unit-celsius {
  background: radial-gradient(circle at top left, rgba(248, 113, 113, 0.6), transparent 35%), linear-
gradient(135deg, #7f1d1d, #111827);
}

body.unit-fahrenheit {
```

```

    background: radial-gradient(circle at top left, rgba(96, 165, 250, 0.65), transparent 35%), linear-
gradient(135deg, #1e3a8a, #111827);
}

body.unit-kelvin {
    background: radial-gradient(circle at top left, rgba(74, 222, 128, 0.6), transparent 35%), linear-
gradient(135deg, #14532d, #111827);
}

button,
input,
select,
textarea {
    font: inherit;
}

button {
    cursor: pointer;
}

code,
pre {
    font-family: Consolas, Monaco, monospace;
}

pre {
    overflow-x: auto;
    padding: 16px;
    border-radius: var(--radius-md);
    background: rgba(15, 23, 42, 0.7);
    color: #e5e7eb;
}

.scroll-progress {
    position: fixed;
    top: 0;
    left: 0;
    width: 0%;
    height: 4px;
    z-index: 2000;
    background: linear-gradient(90deg, var(--primary), var(--secondary));
}

.glass-navbar {
    position: fixed;
    top: 16px;
    left: 50%;
    transform: translateX(-50%);
    z-index: 1000;
    width: min(1120px, calc(100% - 32px));
    min-height: var(--navbar-height);
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 16px;
    padding: 12px 16px;
    background: var(--glass);
    border: 1px solid var(--border);
    border-radius: 999px;
    backdrop-filter: blur(18px);
    -webkit-backdrop-filter: blur(18px);
    box-shadow: var(--shadow);
}

.brand {
    display: inline-flex;

```

```

    align-items: center;
    gap: 10px;
    color: var(--text);
    text-decoration: none;
    font-weight: 800;
}

.brand__mark {
    width: 42px;
    height: 42px;
    display: grid;
    place-items: center;
    border-radius: 50%;
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    color: #fff;
}

.navbar-links,
.navbar-actions,
.hero-actions,
.badge-row,
.carousel__actions,
.utilities-row,
.unit-buttons,
.chips,
.pagination {
    display: flex;
    align-items: center;
    gap: 10px;
    flex-wrap: wrap;
}

.nav-link,
.icon-button,
.side-link,
.side-group__button,
.dropdown__menu button,
.primary-button,
.secondary-button,
.danger-button,
.chip,
.pagination button,
.carousel__actions button,
.stepper button,
.input-action button {
    border: 1px solid var(--border);
    color: var(--text);
    background: rgba(255, 255, 255, 0.1);
    border-radius: 999px;
    padding: 10px 14px;
    transition: transform var(--transition), background var(--transition), border-color var(--transition), box-shadow var(--transition);
}

.nav-link:hover,
.icon-button:hover,
.side-link:hover,
.side-group__button:hover,
.dropdown__menu button:hover,
.secondary-button:hover,
.chip:hover,
.pagination button:hover {
    background: var(--glass-strong);
    transform: translateY(-1px);
}

```

```

.nav-link.is-active,
.side-link.is-active,
.chip.is-active,
.pagination button.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
  border-color: transparent;
}

.icon-button {
  width: 44px;
  height: 44px;
  display: inline-grid;
  place-items: center;
  padding: 0;
  font-size: 20px;
}

.dropdown {
  position: relative;
}

.dropdown__menu {
  position: absolute;
  top: calc(100% + 10px);
  right: 0;
  min-width: 180px;
  display: grid;
  gap: 8px;
  padding: 10px;
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  background: rgba(15, 23, 42, 0.86);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  backdrop-filter: blur(18px);
  box-shadow: var(--shadow);
  transition: opacity var(--transition), transform var(--transition);
}

.dropdown.is-open .dropdown__menu {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1500;
  width: min(340px, 86vw);
  height: 100vh;
  padding: 20px;
  background: rgba(15, 23, 42, 0.9);
  color: #ffffff;
  border-right: 1px solid rgba(255, 255, 255, 0.18);
  backdrop-filter: blur(20px);
  transform: translateX(-105%);
  transition: transform var(--transition);
}

.sidebar.is-open {
  transform: translateX(0);
}

```

```

.sidebar_header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 20px;
}

.sidebar_content,
.side-group_content {
  display: grid;
  gap: 10px;
}

.side-link,
.side-group_button,
.side-group_content button {
  width: 100%;
  text-align: left;
}

.side-group_content {
  max-height: 0;
  overflow: hidden;
  transition: max-height var(--transition);
  padding-left: 12px;
}

.side-group.is-open .side-group_content {
  max-height: 220px;
}

.side-group_content button {
  border: 0;
  border-radius: var(--radius-sm);
  padding: 10px 12px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.08);
}

.overlay {
  position: fixed;
  inset: 0;
  z-index: 1400;
  background: rgba(0, 0, 0, 0.45);
  opacity: 0;
  pointer-events: none;
  transition: opacity var(--transition);
}

.overlay.is-active {
  opacity: 1;
  pointer-events: auto;
}

.page-shell {
  width: min(1120px, calc(100% - 32px));
  margin: 0 auto;
  padding: 120px 0 80px;
}

.view-section {
  display: none;
  animation: fadeIn 0.35s ease;
}

```

```

.view-section.is-active {
  display: block;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

.glass-panel {
  background: var(--glass);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 26px;
  box-shadow: var(--shadow);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
}

.hero {
  min-height: 360px;
  display: grid;
  align-content: center;
}

.eyebrow {
  display: inline-block;
  margin-bottom: 10px;
  color: #93c5fd;
  font-weight: 800;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  font-size: 13px;
}

h1,
h2,
h3,
p {
  margin-top: 0;
}

h1 {
  max-width: 850px;
  font-size: clamp(34px, 7vw, 64px);
  line-height: 1.05;
  margin-bottom: 18px;
}

h2 {
  font-size: clamp(28px, 5vw, 42px);
  margin-bottom: 12px;
}

p,
.muted {
  color: var(--muted);
  line-height: 1.7;
}

.primary-button {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
  border-color: transparent;
  box-shadow: 0 16px 30px rgba(37, 99, 235, 0.32);
}

```

```

.primary-button:hover,
.danger-button:hover {
  transform: translateY(-2px);
}

.secondary-button {
  background: rgba(255, 255, 255, 0.12);
}

.danger-button {
  background: rgba(239, 68, 68, 0.9);
  color: #ffffff;
  border-color: transparent;
}

.stats-grid,
.feature-grid,
.component-grid,
.card-grid,
.two-column {
  display: grid;
  gap: 18px;
}

.stats-grid {
  grid-template-columns: repeat(3, 1fr);
  margin-top: 18px;
}

.stat-card strong {
  display: block;
  font-size: 42px;
}

.feature-grid {
  grid-template-columns: repeat(4, 1fr);
  margin-top: 18px;
}

.feature-card {
  padding: 18px;
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.1);
  border: 1px solid var(--border);
}

.feature-card strong {
  display: block;
  margin-bottom: 8px;
}

.section-heading {
  margin-bottom: 22px;
}

.component-grid {
  grid-template-columns: repeat(2, 1fr);
}

.two-column {
  grid-template-columns: 1fr 1fr;
}

input,
select,

```

```

textarea {
  width: 100%;
  min-height: 48px;
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  padding: 12px 14px;
  color: var(--text);
  background: rgba(255, 255, 255, 0.12);
  outline: none;
  transition: box-shadow var(--transition), border-color var(--transition), background var(--transition);
}

textarea {
  resize: vertical;
}

input::placeholder,
textarea::placeholder {
  color: var(--muted);
}

input:focus,
select:focus,
textarea:focus {
  border-color: rgba(147, 197, 253, 0.9);
  box-shadow: 0 0 4px rgba(147, 197, 253, 0.18);
}

label {
  display: block;
  margin: 12px 0 8px;
  font-weight: 800;
}

.form-card {
  display: grid;
}

.field-error {
  min-height: 18px;
  color: #fecaca;
}

.input-action {
  display: flex;
  gap: 10px;
}

.input-action input {
  flex: 1;
}

.check-line {
  display: flex;
  align-items: center;
  gap: 10px;
  font-weight: 500;
}

.check-line input {
  width: auto;
  min-height: auto;
}

.badge,
.notification-badge {

```

```

display: inline-flex;
align-items: center;
justify-content: center;
border-radius: 999px;
padding: 7px 10px;
color: #ffffff;
background: rgba(37, 99, 235, 0.9);
font-size: 13px;
font-weight: 800;
}

.notification-badge {
width: 32px;
height: 32px;
padding: 0;
}

.custom-select {
position: relative;
width: 100%;
z-index: 20;
}

.custom-select__button {
width: 100%;
min-height: 50px;
display: flex;
align-items: center;
justify-content: space-between;
padding: 12px 16px;
color: var(--text);
background: rgba(255, 255, 255, 0.16);
border: 1px solid var(--border);
border-radius: var(--radius-sm);
backdrop-filter: blur(8px);
}

.custom-select__arrow {
font-size: 22px;
transition: transform var(--transition);
}

.custom-select__options {
position: absolute;
top: calc(100% + 8px);
left: 0;
width: 100%;
margin: 0;
padding: 8px;
list-style: none;
opacity: 0;
transform: translateY(-8px);
pointer-events: none;
background: rgba(15, 23, 42, 0.88);
border: 1px solid rgba(255, 255, 255, 0.25);
border-radius: var(--radius-md);
backdrop-filter: blur(18px);
box-shadow: var(--shadow);
transition: opacity var(--transition), transform var(--transition);
}

.custom-select.is-open .custom-select__options {
opacity: 1;
transform: translateY(0);
pointer-events: auto;
}

```

```

.custom-select.is-open .custom-select__arrow {
  transform: rotate(180deg);
}

.custom-select__option {
  padding: 12px 14px;
  border-radius: var(--radius-sm);
  color: #ffffff;
  cursor: pointer;
}

.custom-select__option:hover,
.custom-select__option.is-selected {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
}

.tabs__buttons {
  display: flex;
  gap: 8px;
  margin-bottom: 12px;
}

.tabs__buttons button {
  border: 1px solid var(--border);
  border-radius: 999px;
  padding: 8px 12px;
  color: var(--text);
  background: rgba(255, 255, 255, 0.12);
}

.tabs__buttons button.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
}

.tab-panel {
  display: none;
  padding: 14px;
  border-radius: var(--radius-sm);
  background: rgba(255, 255, 255, 0.08);
}

.tab-panel.is-active {
  display: block;
}

.accordion__button {
  width: 100%;
  margin-top: 8px;
  padding: 12px;
  text-align: left;
  color: var(--text);
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
}

.accordion__content {
  max-height: 0;
  overflow: hidden;
  color: var(--muted);
  transition: max-height var(--transition), padding var(--transition);
}

.accordion__content.is-open {

```

```

    max-height: 140px;
    padding: 12px;
}

.carousel__slide {
    display: none;
    min-height: 120px;
    place-items: center;
    border-radius: var(--radius-md);
    background: linear-gradient(135deg, rgba(37, 99, 235, 0.6), rgba(124, 58, 237, 0.6));
    color: #ffffff;
    font-weight: 900;
}

.carousel__slide.is-active {
    display: grid;
}

.modal {
    width: min(520px, calc(100% - 32px));
    border: 1px solid var(--border);
    border-radius: var(--radius-lg);
    padding: 0;
    color: var(--text);
    background: rgba(15, 23, 42, 0.92);
    backdrop-filter: blur(20px);
    box-shadow: var(--shadow);
}

.modal::backdrop {
    background: rgba(0, 0, 0, 0.55);
}

.modal__content {
    position: relative;
    padding: 26px;
}

.modal__close {
    position: absolute;
    top: 14px;
    right: 14px;
    width: 36px;
    height: 36px;
    border-radius: 50%;
    border: 1px solid var(--border);
    color: #ffffff;
    background: rgba(255, 255, 255, 0.12);
}

.toast-container {
    position: fixed;
    right: 20px;
    bottom: 20px;
    z-index: 2000;
    display: grid;
    gap: 10px;
}

.toast {
    padding: 14px 16px;
    border-radius: var(--radius-md);
    color: #ffffff;
    background: rgba(15, 23, 42, 0.92);
    border: 1px solid rgba(255, 255, 255, 0.2);
    box-shadow: var(--shadow);
}

```

```

    animation: slideToast 0.25s ease;
}

@keyframes slideToast {
  from { opacity: 0; transform: translateX(20px); }
  to { opacity: 1; transform: translateX(0); }
}

.tooltip {
  position: relative;
}

.tooltip::after {
  content: attr(data-tooltip);
  position: absolute;
  bottom: calc(100% + 8px);
  right: 0;
  width: 210px;
  padding: 10px;
  border-radius: var(--radius-sm);
  color: #ffffff;
  background: rgba(15, 23, 42, 0.92);
  opacity: 0;
  pointer-events: none;
  transform: translateY(6px);
  transition: opacity var(--transition), transform var(--transition);
}

.tooltip:hover::after {
  opacity: 1;
  transform: translateY(0);
}

.data-toolbar {
  display: grid;
  grid-template-columns: 1fr 220px;
  gap: 14px;
  margin-bottom: 18px;
}

.chips {
  grid-column: 1 / -1;
}

.card-grid {
  grid-template-columns: repeat(3, 1fr);
}

.product-card {
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 18px;
  background: rgba(255, 255, 255, 0.1);
}

.product-card__meta {
  display: flex;
  justify-content: space-between;
  gap: 8px;
  color: var(--muted);
}

.pagination {
  margin: 18px 0;
  justify-content: center;
}

```

```

.table-wrap {
  overflow-x: auto;
}

.data-table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 14px;
}

.data-table th,
.data-table td {
  padding: 12px;
  border-bottom: 1px solid var(--border);
  text-align: left;
}

.data-table th {
  cursor: pointer;
  color: #93c5fd;
}

.item-list {
  padding: 0;
  list-style: none;
  display: grid;
  gap: 10px;
}

.item-list li {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  padding: 12px;
  border-radius: var(--radius-sm);
  background: rgba(255, 255, 255, 0.1);
  border: 1px solid var(--border);
}

.draggable-list li {
  cursor: grab;
}

.temperature-tool {
  max-width: 560px;
}

.unit-button {
  width: 64px;
  height: 64px;
  border: 1px solid var(--border);
  border-radius: 20px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
  font-size: 22px;
  font-weight: 900;
}

.unit-button.is-active[data-unit="celsius"] {
  background: linear-gradient(135deg, #ef4444, #991b1b);
}

.unit-button.is-active[data-unit="fahrenheit"] {
  background: linear-gradient(135deg, #3b82f6, #1e3a8a);
}

```

```

}

.unit-button.is-active[data-unit="kelvin"] {
  background: linear-gradient(135deg, #22c55e, #14532d);
}

.result-box {
  margin-top: 18px;
  padding: 18px;
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid var(--border);
  line-height: 1.7;
}

.stepper__steps {
  display: flex;
  gap: 12px;
  margin-bottom: 12px;
}

.stepper__steps span {
  width: 36px;
  height: 36px;
  display: grid;
  place-items: center;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid var(--border);
}

.stepper__steps span.is-active {
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
}

.skeleton {
  height: 18px;
  margin-top: 12px;
  border-radius: 999px;
  background: linear-gradient(90deg, rgba(255,255,255,0.08), rgba(255,255,255,0.24), rgba(255,255,255,0.08));
  background-size: 200% 100%;
  animation: loading 1.2s infinite;
}

@keyframes loading {
  to { background-position: -200% 0; }
}

.reveal {
  opacity: 0;
  transform: translateY(18px);
  transition: opacity 0.6s ease, transform 0.6s ease;
}

.reveal.is-visible {
  opacity: 1;
  transform: translateY(0);
}

.back-to-top {
  position: fixed;
  right: 20px;
  bottom: 20px;
  z-index: 900;
  width: 48px;

```

```

height: 48px;
border: 1px solid var(--border);
border-radius: 50%;
color: #ffffff;
background: linear-gradient(135deg, var(--primary), var(--secondary));
opacity: 0;
pointer-events: none;
transition: opacity var(--transition), transform var(--transition);
}

.back-to-top.is-visible {
  opacity: 1;
  pointer-events: auto;
}

@media (max-width: 900px) {
  .navbar-links {
    display: none;
  }

  .stats-grid,
  .component-grid,
  .two-column,
  .card-grid,
  .feature-grid,
  .data-toolbar {
    grid-template-columns: 1fr;
  }

  h1 {
    font-size: 36px;
  }
}

@media (max-width: 520px) {
  .page-shell {
    width: min(100% - 24px, 1120px);
    padding-top: 104px;
  }

  .glass-navbar {
    width: calc(100% - 20px);
    top: 10px;
    border-radius: 22px;
  }

  .brand__text {
    display: none;
  }

  .glass-panel {
    padding: 18px;
  }
}

```

Anexo B - funciones.js completo corregido

Incluye la versión modular completa. La función de temperatura se documenta con validación de campo vacío.

```
export const $ = (selector, parent = document) => parent.querySelector(selector);
export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

export class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

export class RouterSections {
  constructor(toast) {
    this.sections = $$('[data-view-section]');
    this.buttons = $$('[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.showSection(button.dataset.section));
    });
  }

  showSection(sectionId) {
    const target = document.getElementById(sectionId);
    if (!target) return;

    this.sections.forEach((section) => section.classList.remove("is-active"));
    target.classList.add("is-active");

    this.buttons.forEach((button) => {

```

```

        button.classList.toggle("is-active", button.dataset.section === sectionId);
    });

    window.scrollTo({ top: 0, behavior: "smooth" });
    this.toast?.show(`Seccion activa: ${sectionId}`);
}
}

export class ThemeManager {
    constructor(buttonId = "themeToggle") {
        this.button = document.getElementById(buttonId);
        this.theme = localStorage.getItem("kit-theme") || "dark";
        this.init();
    }

    init() {
        this.applyTheme();
        this.button.addEventListener("click", () => {
            this.theme = this.theme === "dark" ? "light" : "dark";
            localStorage.setItem("kit-theme", this.theme);
            this.applyTheme();
        });
    }

    applyTheme() {
        document.documentElement.dataset.theme = this.theme;
        this.button.textContent = this.theme === "dark" ? "☾" : "☀";
    }
}

export class SidebarMenu {
    constructor() {
        this.sidebar = $("#sidebar");
        this.overlay = $("#overlay");
        this.openButton = $("#openSidebar");
        this.closeButton = $("#closeSidebar");
        this.groupButtons = $(".side-group__button");
        this.sectionButtons = $(".#sidebar [data-section]");
        this.init();
    }

    init() {
        this.openButton.addEventListener("click", () => this.open());
        this.closeButton.addEventListener("click", () => this.close());
        this.overlay.addEventListener("click", () => this.close());
        document.addEventListener("keydown", (event) => {
            if (event.key === "Escape") this.close();
        });
        this.groupButtons.forEach((button) => {
            button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
        });
        this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
    }

    open() {
        this.sidebar.classList.add("is-open");
        this.overlay.classList.add("is-active");
    }

    close() {
        this.sidebar.classList.remove("is-open");
        this.overlay.classList.remove("is-active");
    }
}

export class Dropdowns {

```

```

    constructor() {
        this.dropdowns = $$('[data-dropdown]');
        this.init();
    }

    init() {
        this.dropdowns.forEach((dropdown) => {
            const button = $('[data-dropdown-button]', dropdown);
            button.addEventListener("click", (event) => {
                event.stopPropagation();
                dropdown.classList.toggle("is-open");
            });
        });
        document.addEventListener("click", () => this.dropdowns.forEach((d) => d.classList.remove("is-open")));
    }
}

export class CustomSelect {
    constructor(selector = "[data-custom-select]", onChange = null) {
        this.selects = $(selector);
        this.onChange = onChange;
        this.init();
    }

    init() {
        this.selects.forEach((select) => {
            const button = $(".custom-select__button", select);
            const valueText = $(".custom-select__value", select);
            const hiddenInput = $("input[type='hidden']", select);
            const options = $(".custom-select__option", select);

            button.addEventListener("click", (event) => {
                event.stopPropagation();
                this.closeAll(select);
                select.classList.toggle("is-open");
            });

            options.forEach((option) => {
                option.addEventListener("click", () => {
                    hiddenInput.value = option.dataset.value;
                    valueText.textContent = option.textContent;
                    options.forEach((item) => item.classList.remove("is-selected"));
                    option.classList.add("is-selected");
                    select.classList.remove("is-open");
                    this.onChange?.(hiddenInput.value, select);
                    document.dispatchEvent(new CustomEvent("custom-select:change", { detail: { value:
hiddenInput.value } }));
                });
            });

            document.addEventListener("click", () => this.closeAll());
        });
    }

    closeAll(except = null) {
        this.selects.forEach((select) => {
            if (select !== except) select.classList.remove("is-open");
        });
    }
}

export class ModalManager {
    constructor() {
        this.init();
    }
}

```

```

init() {
  $$('[data-open-modal]').forEach((button) => {
    button.addEventListener("click", () =>
document.getElementById(button.dataset.openModal)?.showModal());
  });
  $$('[data-close-modal]').forEach((button) => {
    button.addEventListener("click", () => button.closest("dialog)?.close());
  });
}
}

export class Tabs {
  constructor(selector = "[data-tabs]") {
    this.components = $$(selector);
    this.init();
  }

  init() {
    this.components.forEach((tabs) => {
      const buttons = $$("[data-tab]", tabs);
      const panels = $$(".tab-panel", tabs);
      buttons.forEach((button) => {
        button.addEventListener("click", () => {
          buttons.forEach((item) => item.classList.remove("is-active"));
          panels.forEach((panel) => panel.classList.remove("is-active"));
          button.classList.add("is-active");
          document.getElementById(button.dataset.tab)?.classList.add("is-active");
        });
      });
    });
  }
}

export class Accordion {
  constructor(selector = "[data-accordion]") {
    this.components = $$(selector);
    this.init();
  }

  init() {
    this.components.forEach((accordion) => {
      const buttons = $$(".accordion__button", accordion);
      buttons.forEach((button) => {
        button.addEventListener("click", () => button.nextElementSibling.classList.toggle("is-open"));
      });
    });
  }
}

export class Carousel {
  constructor(selector = "[data-carousel]") {
    this.carousels = $$(selector);
    this.init();
  }

  init() {
    this.carousels.forEach((carousel) => {
      const slides = $$(".carousel__slide", carousel);
      const next = $("[data-carousel-next]", carousel);
      const prev = $("[data-carousel-prev]", carousel);
      let index = 0;
      const render = () => slides.forEach((slide, i) => slide.classList.toggle("is-active", i === index));
      next.addEventListener("click", () => { index = (index + 1) % slides.length; render(); });
      prev.addEventListener("click", () => { index = (index - 1 + slides.length) % slides.length;
render(); });
    });
  }
}

```

```

    }
  }

export class ScrollTools {
  constructor() {
    this.progress = $("#scrollProgress");
    this.backButton = $("#backToTop");
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update());
    this.backButton.addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth" }));
    this.update();
  }

  update() {
    const total = document.documentElement.scrollHeight - window.innerHeight;
    const progress = total > 0 ? (window.scrollY / total) * 100 : 0;
    this.progress.style.width = `${progress}%`;
    this.backButton.classList.toggle("is-visible", window.scrollY > 400);
  }
}

export class RevealOnScroll {
  constructor(selector = ".reveal") {
    this.elements = $(selector);
    this.observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) entry.target.classList.add("is-visible");
      });
    }, { threshold: 0.15 });
    this.elements.forEach((element) => this.observer.observe(element));
  }
}

export class CounterAnimation {
  constructor(selector = "[data-counter]") {
    this.counters = $(selector);
    this.init();
  }

  init() {
    this.counters.forEach((counter) => {
      const target = Number(counter.dataset.target || 0);
      let current = 0;
      const step = Math.max(1, Math.ceil(target / 40));
      const interval = setInterval(() => {
        current += step;
        if (current >= target) {
          current = target;
          clearInterval(interval);
        }
        counter.textContent = current;
      }, 30);
    });
  }
}

export class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
    this.remember = $("#rememberUser");
    this.toast = toast;
  }
}

```

```

        this.init();
    }

    init() {
        this.renderStatus();
        this.form.addEventListener("submit", (event) => this.login(event));
        this.logout.addEventListener("click", () => this.logoutUser());
    }

    login(event) {
        event.preventDefault();
        const email = $("#loginEmail").value.trim();
        const password = $("#loginPassword").value.trim();
        if (!email.includes("@") || password.length < 6) {
            this.toast.show("Revisa correo y contraseña.");
            return;
        }
        const user = { email, date: new Date().toLocaleString() };
        if (this.remember.checked) saveJSON("kit-user", user);
        sessionStorage.setItem("kit-session", JSON.stringify(user));
        this.renderStatus(user);
        this.toast.show("Sesion iniciada correctamente.");
    }

    logoutUser() {
        sessionStorage.removeItem("kit-session");
        localStorage.removeItem("kit-user");
        this.renderStatus(null);
        this.toast.show("Sesion cerrada.");
    }

    renderStatus(user = null) {
        const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session") ||
"null");
        this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun
no hay sesion iniciada.";
    }
}

export class PasswordToggle {
    constructor() {
        this.buttons = $('[data-toggle-password]');
        this.init();
    }

    init() {
        this.buttons.forEach((button) => {
            button.addEventListener("click", () => {
                const input = document.getElementById(button.dataset.togglePassword);
                input.type = input.type === "password" ? "text" : "password";
                button.textContent = input.type === "password" ? "Ver" : "Ocultar";
            });
        });
    }
}

export class FormValidation {
    constructor(formId, toast) {
        this.form = document.getElementById(formId);
        this.toast = toast;
        if (this.form) this.init();
    }

    init() {
        this.form.addEventListener("submit", (event) => {
            event.preventDefault();

```

```

        const valid = this.validate();
        this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
    });
}

validate() {
    let isValid = true;
    $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
        const error = document.querySelector(`[data-error="${input.id}"]`);
        let message = "";
        if (!input.value.trim()) message = "Este campo es obligatorio.";
        if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no
valido.";
        if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo $
{input.minLength} caracteres.`;
        if (error) error.textContent = message;
        input.style.borderColor = message ? "#ef4444" : "#22c55e";
        if (message) isValid = false;
    });
    return isValid;
}
}

export class CharacterCounter {
    constructor(inputId, counterId) {
        this.input = document.getElementById(inputId);
        this.counter = document.getElementById(counterId);
        this.init();
    }

    init() {
        this.input.addEventListener("input", () => this.counter.textContent = this.input.value.length);
    }
}

export class RangePreview {
    constructor(inputId, valueId) {
        this.input = document.getElementById(inputId);
        this.value = document.getElementById(valueId);
        this.init();
    }

    init() {
        this.input.addEventListener("input", () => this.value.textContent = this.input.value);
    }
}

export class FileInfo {
    constructor(inputId, infoId) {
        this.input = document.getElementById(inputId);
        this.info = document.getElementById(infoId);
        this.init();
    }

    init() {
        this.input.addEventListener("change", () => {
            const file = this.input.files[0];
            this.info.textContent = file ? `${file.name} - ${(file.size / 1024).toFixed(1)} KB` : "No hay
archivo seleccionado.";
        });
    }
}

export class AutoSaveDraft {
    constructor(inputId, clearId, key, toast) {
        this.input = document.getElementById(inputId);

```

```

        this.clearButton = document.getElementById(clearId);
        this.key = key;
        this.toast = toast;
        this.init();
    }

    init() {
        this.input.value = localStorage.getItem(this.key) || "";
        this.input.dispatchEvent(new Event("input"));
        this.input.addEventListener("input", debounce(() => localStorage.setItem(this.key, this.input.value),
250));
        this.clearButton.addEventListener("click", () => {
            localStorage.removeItem(this.key);
            this.input.value = "";
            this.input.dispatchEvent(new Event("input"));
            this.toast.show("Borrador eliminado.");
        });
    }
}

export class Stepper {
    constructor(selector = "[data-stepper]") {
        this.stepper = $(selector);
        this.index = 0;
        this.texts = ["Paso 1: Datos personales", "Paso 2: Preferencias", "Paso 3: Confirmacion"];
        this.init();
    }

    init() {
        $('[data-step-next]', this.stepper).addEventListener("click", () => this.move(1));
        $('[data-step-prev]', this.stepper).addEventListener("click", () => this.move(-1));
        this.render();
    }

    move(direction) {
        this.index = Math.min(2, Math.max(0, this.index + direction));
        this.render();
    }

    render() {
        $$(".stepper__steps span", this.stepper).forEach((step, i) => step.classList.toggle("is-active", i <=
this.index));
        $(".stepper__text", this.stepper).textContent = this.texts[this.index];
    }
}

export class SkeletonLoader {
    constructor(buttonId, areaId, toast) {
        this.button = document.getElementById(buttonId);
        this.area = document.getElementById(areaId);
        this.toast = toast;
        this.init();
    }

    init() {
        this.button.addEventListener("click", () => {
            this.area.innerHTML = `<div class="skeleton"></div><div class="skeleton"></div><div
class="skeleton"></div>`;
            setTimeout(() => {
                this.area.innerHTML = `<p class="muted">Contenido cargado correctamente.</p>`;
                this.toast.show("Carga simulada finalizada.");
            }, 1200);
        });
    }
}

```

```

export class CatalogCards {
  constructor() {
    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
      .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
  }

  render() {
    const items = this.getFilteredData();
    const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
    this.page = Math.min(this.page, totalPages);
    const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
    this.grid.innerHTML = pageItems.map((item) => `
      <article class="product-card">
        <h3>${item.name}</h3>
        <p class="muted">Categoria: ${item.category}</p>
        <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
${item.rating}</span></div>
      </article>
    `).join("") || `<p class="muted">No hay resultados.</p>`;
    this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1 ===
this.page ? "is-active" : ""}" data-page="${i + 1}">${i + 1}</button>`).join("");
    $('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
  }
}

```

```

export class SortableTable {
  constructor(tableId, searchId) {
    this.table = document.getElementById(tableId);
    this.search = document.getElementById(searchId);
    this.direction = 1;
    this.init();
  }

  init() {
    $('[data-sort-table]', this.table).forEach((header, index) => {
      header.addEventListener("click", () => this.sort(index));
    });
    this.search.addEventListener("input", () => this.filter());
  }

  sort(index) {
    const rows = $('tbody tr', this.table);
    rows.sort((a, b) => a.children[index].textContent.localeCompare(b.children[index].textContent,
undefined, { numeric: true }) * this.direction);
    this.direction *= -1;
    rows.forEach((row) => this.table.tBodies[0].appendChild(row));
  }

  filter() {
    const term = this.search.value.toLowerCase();
    $('tbody tr', this.table).forEach((row) => row.style.display =
row.textContent.toLowerCase().includes(term) ? "" : "none");
  }
}

export class NotesManager {
  constructor(toast) {
    this.input = $("#noteInput");
    this.saveButton = $("#saveNote");
    this.list = $("#noteList");
    this.exportButton = $("#exportNotes");
    this.toast = toast;
    this.notes = readJSON("kit-notes", []);
    this.init();
  }

  init() {
    this.saveButton.addEventListener("click", () => this.add());
    this.exportButton.addEventListener("click", () => this.export());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una nota primero.");
    this.notes.push({ id: Date.now(), text });
    saveJSON("kit-notes", this.notes);
    this.input.value = "";
    this.render();
    this.toast.show("Nota guardada.");
  }

  remove(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON("kit-notes", this.notes);
    this.render();
  }

  render() {
    this.list.innerHTML = this.notes.map((note) => `<li><span>${note.text}</span><button class="danger-

```

```

button" data-id="${note.id}">Borrar</button></li>`).join("");
    $$('#button', this.list).forEach((button) => button.addEventListener("click", () =>
this.remove(Number(button.dataset.id))));
    }

    export() {
        const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type: "application/json" });
        const link = document.createElement("a");
        link.href = URL.createObjectURL(blob);
        link.download = "notas.json";
        link.click();
        URL.revokeObjectURL(link.href);
    }
}

export class TodoManager {
    constructor(toast) {
        this.input = $("#todoInput");
        this.addButton = $("#addTodo");
        this.clearButton = $("#clearTodos");
        this.list = $("#todoList");
        this.toast = toast;
        this.todos = readJSON("kit-todos", []);
        this.dragged = null;
        this.init();
    }

    init() {
        this.addButton.addEventListener("click", () => this.add());
        this.clearButton.addEventListener("click", () => this.clear());
        this.render();
    }

    add() {
        const text = this.input.value.trim();
        if (!text) return this.toast.show("Escribe una tarea.");
        this.todos.push({ id: Date.now(), text, done: false });
        this.input.value = "";
        this.saveRender();
    }

    toggle(id) {
        this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
        this.saveRender();
    }

    remove(id) {
        this.todos = this.todos.filter((todo) => todo.id !== id);
        this.saveRender();
    }

    clear() {
        this.todos = [];
        this.saveRender();
        this.toast.show("Tareas borradas.");
    }

    saveRender() {
        saveJSON("kit-todos", this.todos);
        this.render();
    }

    render() {
        this.list.innerHTML = this.todos.map((todo) => `
            <li draggable="true" data-id="${todo.id}">
                <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>

```

```

{todo.done ? "✅" : "❌"} ${todo.text}</span></label>
      <button class="danger-button">Borrar</button>
    </li>
  `).join("");
  $$('li', this.list).forEach((li) => {
    const id = Number(li.dataset.id);
    $('input', li).addEventListener("change", () => this.toggle(id));
    $('button', li).addEventListener("click", () => this.remove(id));
    li.addEventListener("dragstart", () => this.dragged = li);
    li.addEventListener("dragover", (event) => event.preventDefault());
    li.addEventListener("drop", () => {
      if (this.dragged && this.dragged !== li) {
        this.list.insertBefore(this.dragged, li);
        this.todos = $$('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
Number(item.dataset.id)));
        saveJSON("kit-todos", this.todos);
      }
    });
  });
}
}

export class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }

  selectUnit(unit) {
    this.selectedUnit = unit;
    this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    document.body.classList.add(`unit-${unit}`);
    this.message.textContent = `Unidad seleccionada: ${unit}`;
  }

  calculate() {
    const rawValue = this.input.value.trim();

    if (rawValue === "") {
      this.message.textContent = "El campo esta vacio. Ingresar una temperatura para convertir.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    const value = Number(rawValue);
    if (!Number.isFinite(value)) {
      this.message.textContent = "Ingresar un numero valido.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }
  }
}

```

```

    let celsius;
    if (this.selectedUnit === "celsius") celsius = value;
    if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
    if (this.selectedUnit === "kelvin") celsius = value - 273.15;

    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;

    this.result.innerHTML = `
      <strong>Resultado desde ${this.selectedUnit}</strong><br>
      Celsius: ${celsius.toFixed(2)} °C<br>
      Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
      Kelvin: ${kelvin.toFixed(2)} K
    `;
    this.toast?.show("Temperatura calculada.");
  }
}

export class CopyToClipboard {
  constructor(toast) {
    this.buttons = $('[data-copy]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", async () => {
        const target = document.querySelector(button.dataset.copy);
        await navigator.clipboard.writeText(target.textContent);
        this.toast.show("Texto copiado.");
      });
    });
  }
}

export class FeatureList {
  constructor() {
    this.features = [
      "Navbar fija glass", "Sidebar ocultable", "Submenus laterales", "Router de secciones",
      "Dropdown", "Tema claro/oscuro", "Scroll progress", "Back to top",
      "Reveal on scroll", "Contador animado", "Login demo", "Logout demo",
      "Recordar usuario", "Password toggle", "Validacion formularios", "Mensajes de error",
      "Character counter", "Range preview", "File info", "Autosave draft",
      "Stepper", "Skeleton loader", "Select personalizado", "Tabs",
      "Acordeon", "Modal", "Toast", "Carrusel",
      "Tooltip", "Badge", "Copy clipboard", "Cards dinamicas",
      "Busqueda", "Filtros chips", "Ordenamiento", "Paginacion",
      "Tabla ordenable", "Filtro de tabla", "LocalStorage notas", "Export JSON",
      "Todo list", "Drag and drop", "Confirmacion visual", "Conversor C/F/K",
      "Cambio fondo dinamico", "Print page", "Reset demo", "Debounce",
      "Helpers reutilizables", "Version modular", "Version normal", "Responsive design"
    ];
    this.container = $("#featureList");
    this.render();
  }

  render() {
    this.container.innerHTML = this.features.map((feature, index) => `
      <article class="feature-card">
        <strong>${index + 1}. ${feature}</strong>
        <span class="muted">Reutilizable en proyectos web.</span>
      </article>
    `).join("");
  }
}

```

```

export class UtilityActions {
  constructor(toast) {
    this.toast = toast;
    this.init();
  }

  init() {
    $('[data-toast]').forEach((button) => button.addEventListener("click", () =>
this.toast.show(button.dataset.toast)));
    $("#printPage").addEventListener("click", () => window.print());
    $("#resetDemo").addEventListener("click", () => {
      document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
      this.toast.show("Demo visual reiniciada.");
    });
    document.addEventListener("custom-select:change", (event) => {
      const target = $("#selectedProductText");
      if (target) target.textContent = event.detail.value;
    });
  }
}

export function initApp() {
  const toast = new ToastManager();
  new FeatureList();
  new RouterSections(toast);
  new ThemeManager();
  new SidebarMenu();
  new Dropdowns();
  new CustomSelect();
  new ModalManager();
  new Tabs();
  new Accordion();
  new Carousel();
  new ScrollTools();
  new RevealOnScroll();
  new CounterAnimation();
  new LoginDemo(toast);
  new PasswordToggle();
  new FormValidation("contactForm", toast);
  new CharacterCounter("messageText", "charCounter");
  new RangePreview("rangeInput", "rangeValue");
  new FileInfo("fileInput", "fileInfo");
  new AutoSaveDraft("messageText", "clearDraft", "kit-message-draft", toast);
  new Stepper();
  new SkeletonLoader("loadSkeleton", "skeletonArea", toast);
  new CatalogCards();
  new SortableTable("demoTable", "tableSearch");
  new NotesManager(toast);
  new TodoManager(toast);
  new TemperatureTool(toast);
  new CopyToClipboard(toast);
  new UtilityActions(toast);
}

```

Anexo C - script-normal.js completo corregido

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

class RouterSections {
  constructor(toast) {
    this.sections = $$('.[data-view-section]');
    this.buttons = $$('.[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.showSection(button.dataset.section));
    });
  }

  showSection(sectionId) {
    const target = document.getElementById(sectionId);
    if (!target) return;

    this.sections.forEach((section) => section.classList.remove("is-active"));
    target.classList.add("is-active");

    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });
  }
}
```

```

        window.scrollTo({ top: 0, behavior: "smooth" });
        this.toast?.show(`Seccion activa: ${sectionId}`);
    }
}

class ThemeManager {
    constructor(buttonId = "themeToggle") {
        this.button = document.getElementById(buttonId);
        this.theme = localStorage.getItem("kit-theme") || "dark";
        this.init();
    }

    init() {
        this.applyTheme();
        this.button.addEventListener("click", () => {
            this.theme = this.theme === "dark" ? "light" : "dark";
            localStorage.setItem("kit-theme", this.theme);
            this.applyTheme();
        });
    }

    applyTheme() {
        document.documentElement.dataset.theme = this.theme;
        this.button.textContent = this.theme === "dark" ? "☾" : "☀";
    }
}

class SidebarMenu {
    constructor() {
        this.sidebar = $("#sidebar");
        this.overlay = $("#overlay");
        this.openButton = $("#openSidebar");
        this.closeButton = $("#closeSidebar");
        this.groupButtons = $(".side-group__button");
        this.sectionButtons = $(".#sidebar [data-section]");
        this.init();
    }

    init() {
        this.openButton.addEventListener("click", () => this.open());
        this.closeButton.addEventListener("click", () => this.close());
        this.overlay.addEventListener("click", () => this.close());
        document.addEventListener("keydown", (event) => {
            if (event.key === "Escape") this.close();
        });
        this.groupButtons.forEach((button) => {
            button.addEventListener("click", () => button.parentElement.classList.toggle("is-open"));
        });
        this.sectionButtons.forEach((button) => button.addEventListener("click", () => this.close()));
    }

    open() {
        this.sidebar.classList.add("is-open");
        this.overlay.classList.add("is-active");
    }

    close() {
        this.sidebar.classList.remove("is-open");
        this.overlay.classList.remove("is-active");
    }
}

class Dropdowns {
    constructor() {
        this.dropdowns = $('[data-dropdown]');
    }
}

```

```

        this.init();
    }

    init() {
        this.dropdowns.forEach((dropdown) => {
            const button = $('[data-dropdown-button]', dropdown);
            button.addEventListener("click", (event) => {
                event.stopPropagation();
                dropdown.classList.toggle("is-open");
            });
        });
        document.addEventListener("click", () => this.dropdowns.forEach((d) => d.classList.remove("is-open")));
    }
}

class CustomSelect {
    constructor(selector = "[data-custom-select]", onChange = null) {
        this.selects = $(selector);
        this.onChange = onChange;
        this.init();
    }

    init() {
        this.selects.forEach((select) => {
            const button = $(".custom-select__button", select);
            const valueText = $(".custom-select__value", select);
            const hiddenInput = $("input[type='hidden']", select);
            const options = $(".custom-select__option", select);

            button.addEventListener("click", (event) => {
                event.stopPropagation();
                this.closeAll(select);
                select.classList.toggle("is-open");
            });

            options.forEach((option) => {
                option.addEventListener("click", () => {
                    hiddenInput.value = option.dataset.value;
                    valueText.textContent = option.textContent;
                    options.forEach((item) => item.classList.remove("is-selected"));
                    option.classList.add("is-selected");
                    select.classList.remove("is-open");
                    this.onChange?.(hiddenInput.value, select);
                    document.dispatchEvent(new CustomEvent("custom-select:change", { detail: { value:
hiddenInput.value } }));
                });
            });

            document.addEventListener("click", () => this.closeAll());
        });
    }

    closeAll(except = null) {
        this.selects.forEach((select) => {
            if (select !== except) select.classList.remove("is-open");
        });
    }
}

class ModalManager {
    constructor() {
        this.init();
    }

    init() {
        $('[data-open-modal]').forEach((button) => {

```

```

        button.addEventListener("click", () =>
document.getElementById(button.dataset.openModal)?.showModal());
    });
    $$('[data-close-modal]').forEach((button) => {
        button.addEventListener("click", () => button.closest("dialog)?.close());
    });
}
}

class Tabs {
    constructor(selector = "[data-tabs]") {
        this.components = $$(selector);
        this.init();
    }

    init() {
        this.components.forEach((tabs) => {
            const buttons = $$("[data-tab]", tabs);
            const panels = $$(".tab-panel", tabs);
            buttons.forEach((button) => {
                button.addEventListener("click", () => {
                    buttons.forEach((item) => item.classList.remove("is-active"));
                    panels.forEach((panel) => panel.classList.remove("is-active"));
                    button.classList.add("is-active");
                    document.getElementById(button.dataset.tab)?.classList.add("is-active");
                });
            });
        });
    }
}

class Accordion {
    constructor(selector = "[data-accordion]") {
        this.components = $$(selector);
        this.init();
    }

    init() {
        this.components.forEach((accordion) => {
            const buttons = $$(".accordion__button", accordion);
            buttons.forEach((button) => {
                button.addEventListener("click", () => button.nextElementSibling.classList.toggle("is-open"));
            });
        });
    }
}

class Carousel {
    constructor(selector = "[data-carousel]") {
        this.carousels = $$(selector);
        this.init();
    }

    init() {
        this.carousels.forEach((carousel) => {
            const slides = $$(".carousel__slide", carousel);
            const next = $("[data-carousel-next]", carousel);
            const prev = $("[data-carousel-prev]", carousel);
            let index = 0;
            const render = () => slides.forEach((slide, i) => slide.classList.toggle("is-active", i === index));
            next.addEventListener("click", () => { index = (index + 1) % slides.length; render(); });
            prev.addEventListener("click", () => { index = (index - 1 + slides.length) % slides.length;
render(); });
        });
    }
}

```

```

class ScrollTools {
  constructor() {
    this.progress = $("#scrollProgress");
    this.backButton = $("#backToTop");
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update());
    this.backButton.addEventListener("click", () => window.scrollTo({ top: 0, behavior: "smooth" }));
    this.update();
  }

  update() {
    const total = document.documentElement.scrollHeight - window.innerHeight;
    const progress = total > 0 ? (window.scrollY / total) * 100 : 0;
    this.progress.style.width = `${progress}%`;
    this.backButton.classList.toggle("is-visible", window.scrollY > 400);
  }
}

class RevealOnScroll {
  constructor(selector = ".reveal") {
    this.elements = $$(selector);
    this.observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) entry.target.classList.add("is-visible");
      });
    }, { threshold: 0.15 });
    this.elements.forEach((element) => this.observer.observe(element));
  }
}

class CounterAnimation {
  constructor(selector = "[data-counter]") {
    this.counters = $$(selector);
    this.init();
  }

  init() {
    this.counters.forEach((counter) => {
      const target = Number(counter.dataset.target || 0);
      let current = 0;
      const step = Math.max(1, Math.ceil(target / 40));
      const interval = setInterval(() => {
        current += step;
        if (current >= target) {
          current = target;
          clearInterval(interval);
        }
        counter.textContent = current;
      }, 30);
    });
  }
}

class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
    this.remember = $("#rememberUser");
    this.toast = toast;
    this.init();
  }
}

```

```

init() {
  this.renderStatus();
  this.form.addEventListener("submit", (event) => this.login(event));
  this.logout.addEventListener("click", () => this.logoutUser());
}

login(event) {
  event.preventDefault();
  const email = $("#loginEmail").value.trim();
  const password = $("#loginPassword").value.trim();
  if (!email.includes("@") || password.length < 6) {
    this.toast.show("Revisa correo y contraseña.");
    return;
  }
  const user = { email, date: new Date().toLocaleString() };
  if (this.remember.checked) saveJSON("kit-user", user);
  sessionStorage.setItem("kit-session", JSON.stringify(user));
  this.renderStatus(user);
  this.toast.show("Sesion iniciada correctamente.");
}

logoutUser() {
  sessionStorage.removeItem("kit-session");
  localStorage.removeItem("kit-user");
  this.renderStatus(null);
  this.toast.show("Sesion cerrada.");
}

renderStatus(user = null) {
  const current = user || readJSON("kit-user") || JSON.parse(sessionStorage.getItem("kit-session") || "null");
  this.status.textContent = current ? `Usuario activo: ${current.email}. Ingreso: ${current.date}` : "Aun no hay sesion iniciada.";
}

}

class PasswordToggle {
  constructor() {
    this.buttons = $('[data-toggle-password]');
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        const input = document.getElementById(button.dataset.togglePassword);
        input.type = input.type === "password" ? "text" : "password";
        button.textContent = input.type === "password" ? "Ver" : "Ocultar";
      });
    });
  }
}

class FormValidation {
  constructor(formId, toast) {
    this.form = document.getElementById(formId);
    this.toast = toast;
    if (this.form) this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();
      const valid = this.validate();
      this.toast.show(valid ? "Formulario enviado correctamente." : "Completa los campos requeridos.");
    });
  }
}

```

```

    });
}

validate() {
    let isValid = true;
    $$('input[required], textarea[required], select[required]', this.form).forEach((input) => {
        const error = document.querySelector(`[data-error="${input.id}"]`);
        let message = "";
        if (!input.value.trim()) message = "Este campo es obligatorio.";
        if (input.type === "email" && input.value && !input.value.includes("@")) message = "Correo no
valido.";
        if (input.minLength > 0 && input.value.length < input.minLength) message = `Minimo $
{input.minLength} caracteres.`;
        if (error) error.textContent = message;
        input.style.borderColor = message ? "#ef4444" : "#22c55e";
        if (message) isValid = false;
    });
    return isValid;
}
}

class CharacterCounter {
    constructor(inputId, counterId) {
        this.input = document.getElementById(inputId);
        this.counter = document.getElementById(counterId);
        this.init();
    }

    init() {
        this.input.addEventListener("input", () => this.counter.textContent = this.input.value.length);
    }
}

class RangePreview {
    constructor(inputId, valueId) {
        this.input = document.getElementById(inputId);
        this.value = document.getElementById(valueId);
        this.init();
    }

    init() {
        this.input.addEventListener("input", () => this.value.textContent = this.input.value);
    }
}

class FileInfo {
    constructor(inputId, infoId) {
        this.input = document.getElementById(inputId);
        this.info = document.getElementById(infoId);
        this.init();
    }

    init() {
        this.input.addEventListener("change", () => {
            const file = this.input.files[0];
            this.info.textContent = file ? `${file.name} - ${(file.size / 1024).toFixed(1)} KB` : "No hay
archivo seleccionado.";
        });
    }
}

class AutoSaveDraft {
    constructor(inputId, clearId, key, toast) {
        this.input = document.getElementById(inputId);
        this.clearButton = document.getElementById(clearId);
        this.key = key;
    }
}

```

```

        this.toast = toast;
        this.init();
    }

    init() {
        this.input.value = localStorage.getItem(this.key) || "";
        this.input.dispatchEvent(new Event("input"));
        this.input.addEventListener("input", debounce(() => localStorage.setItem(this.key, this.input.value),
250));
        this.clearButton.addEventListener("click", () => {
            localStorage.removeItem(this.key);
            this.input.value = "";
            this.input.dispatchEvent(new Event("input"));
            this.toast.show("Borrador eliminado.");
        });
    }
}

class Stepper {
    constructor(selector = "[data-stepper]") {
        this.stepper = $(selector);
        this.index = 0;
        this.texts = ["Paso 1: Datos personales", "Paso 2: Preferencias", "Paso 3: Confirmacion"];
        this.init();
    }

    init() {
        $('[data-step-next]', this.stepper).addEventListener("click", () => this.move(1));
        $('[data-step-prev]', this.stepper).addEventListener("click", () => this.move(-1));
        this.render();
    }

    move(direction) {
        this.index = Math.min(2, Math.max(0, this.index + direction));
        this.render();
    }

    render() {
        $(".stepper__steps span", this.stepper).forEach((step, i) => step.classList.toggle("is-active", i <=
this.index));
        $(".stepper__text", this.stepper).textContent = this.texts[this.index];
    }
}

class SkeletonLoader {
    constructor(buttonId, areaId, toast) {
        this.button = document.getElementById(buttonId);
        this.area = document.getElementById(areaId);
        this.toast = toast;
        this.init();
    }

    init() {
        this.button.addEventListener("click", () => {
            this.area.innerHTML = `<div class="skeleton"></div><div class="skeleton"></div><div
class="skeleton"></div>`;
            setTimeout(() => {
                this.area.innerHTML = `<p class="muted">Contenido cargado correctamente.</p>`;
                this.toast.show("Carga simulada finalizada.");
            }, 1200);
        });
    }
}

class CatalogCards {
    constructor() {

```

```

    this.data = [
      { name: "APA", category: "cerveza", price: 12000, rating: 4.8 },
      { name: "Stout", category: "cerveza", price: 14000, rating: 4.7 },
      { name: "Sour", category: "cerveza", price: 13000, rating: 4.6 },
      { name: "Barley Wine", category: "cerveza", price: 18000, rating: 4.9 },
      { name: "Hamburguesa Xue", category: "comida", price: 26000, rating: 4.9 },
      { name: "Sandwich artesanal", category: "comida", price: 22000, rating: 4.5 },
      { name: "Brownie", category: "postre", price: 9000, rating: 4.4 },
      { name: "Cheesecake", category: "postre", price: 11000, rating: 4.7 }
    ];
    this.page = 1;
    this.perPage = 6;
    this.category = "all";
    this.search = "";
    this.sort = "name";
    this.grid = $("#catalogGrid");
    this.pagination = $("#catalogPagination");
    this.init();
  }

  init() {
    $("#catalogSearch").addEventListener("input", debounce((event) => { this.search =
event.target.value.toLowerCase(); this.page = 1; this.render(); }, 250));
    $("#catalogSort").addEventListener("change", (event) => { this.sort = event.target.value;
this.render(); });
    $("#catalogChips .chip").forEach((chip) => {
      chip.addEventListener("click", () => {
        $("#catalogChips .chip").forEach((item) => item.classList.remove("is-active"));
        chip.classList.add("is-active");
        this.category = chip.dataset.category;
        this.page = 1;
        this.render();
      });
    });
    this.render();
  }

  getFilteredData() {
    return this.data
      .filter((item) => this.category === "all" || item.category === this.category)
      .filter((item) => item.name.toLowerCase().includes(this.search))
      .sort((a, b) => typeof a[this.sort] === "string" ? a[this.sort].localeCompare(b[this.sort]) :
a[this.sort] - b[this.sort]);
  }

  render() {
    const items = this.getFilteredData();
    const totalPages = Math.max(1, Math.ceil(items.length / this.perPage));
    this.page = Math.min(this.page, totalPages);
    const pageItems = items.slice((this.page - 1) * this.perPage, this.page * this.perPage);
    this.grid.innerHTML = pageItems.map((item) => `
      <article class="product-card">
        <h3>${item.name}</h3>
        <p class="muted">Categoria: ${item.category}</p>
        <div class="product-card__meta"><strong>${item.price.toLocaleString()}</strong><span>★ $
${item.rating}</span></div>
      </article>
    `).join("") || `<p class="muted">No hay resultados.</p>`;
    this.pagination.innerHTML = Array.from({ length: totalPages }, (_, i) => `<button class="${i + 1 ===
this.page ? "is-active" : ""}" data-page="${i + 1}">${i + 1}</button>`).join("");
    $('button', this.pagination).forEach((button) => button.addEventListener("click", () => { this.page =
Number(button.dataset.page); this.render(); }));
  }
}

class SortableTable {

```

```

constructor(tableId, searchId) {
  this.table = document.getElementById(tableId);
  this.search = document.getElementById(searchId);
  this.direction = 1;
  this.init();
}

init() {
  $$('[data-sort-table]', this.table).forEach((header, index) => {
    header.addEventListener("click", () => this.sort(index));
  });
  this.search.addEventListener("input", () => this.filter());
}

sort(index) {
  const rows = $$("tbody tr", this.table);
  rows.sort((a, b) => a.children[index].textContent.localeCompare(b.children[index].textContent,
undefined, { numeric: true }) * this.direction);
  this.direction *= -1;
  rows.forEach((row) => this.table.tBodies[0].appendChild(row));
}

filter() {
  const term = this.search.value.toLowerCase();
  $$("tbody tr", this.table).forEach((row) => row.style.display =
row.textContent.toLowerCase().includes(term) ? "" : "none");
}
}

class NotesManager {
  constructor(toast) {
    this.input = $("#noteInput");
    this.saveButton = $("#saveNote");
    this.list = $("#noteList");
    this.exportButton = $("#exportNotes");
    this.toast = toast;
    this.notes = readJSON("kit-notes", []);
    this.init();
  }

  init() {
    this.saveButton.addEventListener("click", () => this.add());
    this.exportButton.addEventListener("click", () => this.export());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una nota primero.");
    this.notes.push({ id: Date.now(), text });
    saveJSON("kit-notes", this.notes);
    this.input.value = "";
    this.render();
    this.toast.show("Nota guardada.");
  }

  remove(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON("kit-notes", this.notes);
    this.render();
  }

  render() {
    this.list.innerHTML = this.notes.map((note) => `<li><span>${note.text}</span><button class="danger-
button" data-id="${note.id}">Borrar</button></li>`).join("");
    $$('button', this.list).forEach((button) => button.addEventListener("click", () =>

```

```

this.remove(Number(button.dataset.id)));
}

export() {
  const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type: "application/json" });
  const link = document.createElement("a");
  link.href = URL.createObjectURL(blob);
  link.download = "notas.json";
  link.click();
  URL.revokeObjectURL(link.href);
}
}

class TodoManager {
  constructor(toast) {
    this.input = $("#todoInput");
    this.addButton = $("#addTodo");
    this.clearButton = $("#clearTodos");
    this.list = $("#todoList");
    this.toast = toast;
    this.todos = readJSON("kit-todos", []);
    this.dragged = null;
    this.init();
  }

  init() {
    this.addButton.addEventListener("click", () => this.add());
    this.clearButton.addEventListener("click", () => this.clear());
    this.render();
  }

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una tarea.");
    this.todos.push({ id: Date.now(), text, done: false });
    this.input.value = "";
    this.saveRender();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.saveRender();
  }

  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.saveRender();
  }

  clear() {
    this.todos = [];
    this.saveRender();
    this.toast.show("Tareas borradas.");
  }

  saveRender() {
    saveJSON("kit-todos", this.todos);
    this.render();
  }

  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li draggable="true" data-id="${todo.id}">
        <label class="check-line"><input type="checkbox" ${todo.done ? "checked" : ""}> <span>${
        {todo.done ? "✅" : "❑"} ${todo.text}</span></label>
        <button class="danger-button">Borrar</button>
    `
  }

```

```

    </li>
  `).join("");
  $$('li', this.list).forEach((li) => {
    const id = Number(li.dataset.id);
    $('input', li).addEventListener("change", () => this.toggle(id));
    $('button', li).addEventListener("click", () => this.remove(id));
    li.addEventListener("dragstart", () => this.dragged = li);
    li.addEventListener("dragover", (event) => event.preventDefault());
    li.addEventListener("drop", () => {
      if (this.dragged && this.dragged !== li) {
        this.list.insertBefore(this.dragged, li);
        this.todos = $$('li', this.list).map((item) => this.todos.find((todo) => todo.id ===
Number(item.dataset.id)));
        saveJSON("kit-todos", this.todos);
      }
    });
  });
}
}

class TemperatureTool {
  constructor(toast) {
    this.selectedUnit = "celsius";
    this.buttons = $$(".unit-button");
    this.input = $("#temperatureInput");
    this.result = $("#temperatureResult");
    this.message = $("#temperatureMessage");
    this.calculateButton = $("#calculateTemperature");
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.selectUnit(button.dataset.unit));
    });
    this.calculateButton.addEventListener("click", () => this.calculate());
    this.selectUnit("celsius");
  }

  selectUnit(unit) {
    this.selectedUnit = unit;
    this.buttons.forEach((button) => button.classList.toggle("is-active", button.dataset.unit === unit));
    document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
    document.body.classList.add(`unit-${unit}`);
    this.message.textContent = `Unidad seleccionada: ${unit}`;
  }

  calculate() {
    const rawValue = this.input.value.trim();

    if (rawValue === "") {
      this.message.textContent = "El campo esta vacio. Ingrese una temperatura para convertir.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    const value = Number(rawValue);
    if (!Number.isFinite(value)) {
      this.message.textContent = "Ingresa un numero valido.";
      this.result.textContent = "No se pudo calcular.";
      return;
    }

    let celsius;
    if (this.selectedUnit === "celsius") celsius = value;
  }
}

```

```

    if (this.selectedUnit === "fahrenheit") celsius = (value - 32) * 5 / 9;
    if (this.selectedUnit === "kelvin") celsius = value - 273.15;

    const fahrenheit = (celsius * 9 / 5) + 32;
    const kelvin = celsius + 273.15;

    this.result.innerHTML = `
      <strong>Resultado desde ${this.selectedUnit}</strong><br>
      Celsius: ${celsius.toFixed(2)} °C<br>
      Fahrenheit: ${fahrenheit.toFixed(2)} °F<br>
      Kelvin: ${kelvin.toFixed(2)} K
    `;
    this.toast?.show("Temperatura calculada.");
  }
}

class CopyToClipboard {
  constructor(toast) {
    this.buttons = $$('[data-copy]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", async () => {
        const target = document.querySelector(button.dataset.copy);
        await navigator.clipboard.writeText(target.textContent);
        this.toast.show("Texto copiado.");
      });
    });
  }
}

class FeatureList {
  constructor() {
    this.features = [
      "Navbar fija glass", "Sidebar ocultable", "Submenus laterales", "Router de secciones",
      "Dropdown", "Tema claro/oscuro", "Scroll progress", "Back to top",
      "Reveal on scroll", "Contador animado", "Login demo", "Logout demo",
      "Recordar usuario", "Password toggle", "Validacion formularios", "Mensajes de error",
      "Character counter", "Range preview", "File info", "Autosave draft",
      "Stepper", "Skeleton loader", "Select personalizado", "Tabs",
      "Acordeon", "Modal", "Toast", "Carrusel",
      "Tooltip", "Badge", "Copy clipboard", "Cards dinamicas",
      "Busqueda", "Filtros chips", "Ordenamiento", "Paginacion",
      "Tabla ordenable", "Filtro de tabla", "LocalStorage notas", "Export JSON",
      "Todo list", "Drag and drop", "Confirmacion visual", "Conversor C/F/K",
      "Cambio fondo dinamico", "Print page", "Reset demo", "Debounce",
      "Helpers reutilizables", "Version modular", "Version normal", "Responsive design"
    ];
    this.container = $("#featureList");
    this.render();
  }

  render() {
    this.container.innerHTML = this.features.map((feature, index) => `
      <article class="feature-card">
        <strong>${index + 1}. ${feature}</strong>
        <span class="muted">Reutilizable en proyectos web.</span>
      </article>
    `).join("");
  }
}

class UtilityActions {

```

```

    constructor(toast) {
        this.toast = toast;
        this.init();
    }

    init() {
        $('[data-toast]').forEach((button) => button.addEventListener("click", () =>
this.toast.show(button.dataset.toast)));
        $("#printPage").addEventListener("click", () => window.print());
        $("#resetDemo").addEventListener("click", () => {
            document.body.classList.remove("unit-celsius", "unit-fahrenheit", "unit-kelvin");
            this.toast.show("Demo visual reiniciada.");
        });
        document.addEventListener("custom-select:change", (event) => {
            const target = $("#selectedProductText");
            if (target) target.textContent = event.detail.value;
        });
    }
}

function initApp() {
    const toast = new ToastManager();
    new FeatureList();
    new RouterSections(toast);
    new ThemeManager();
    new SidebarMenu();
    new Dropdowns();
    new CustomSelect();
    new ModalManager();
    new Tabs();
    new Accordion();
    new Carousel();
    new ScrollTools();
    new RevealOnScroll();
    new CounterAnimation();
    new LoginDemo(toast);
    new PasswordToggle();
    new FormValidation("contactForm", toast);
    new CharacterCounter("messageText", "charCounter");
    new RangePreview("rangeInput", "rangeValue");
    new FileInfo("fileInput", "fileInfo");
    new AutoSaveDraft("messageText", "clearDraft", "kit-message-draft", toast);
    new Stepper();
    new SkeletonLoader("loadSkeleton", "skeletonArea", toast);
    new CatalogCards();
    new SortableTable("demoTable", "tableSearch");
    new NotesManager(toast);
    new TodoManager(toast);
    new TemperatureTool(toast);
    new CopyToClipboard(toast);
    new UtilityActions(toast);
}

document.addEventListener("DOMContentLoaded", () => {
    initApp();
});

```

Anexo D - index.html completo del proyecto base

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Proyecto unificado con componentes reutilizables de HTML, CSS y JavaScript para paginas web profesionales.">
  <title>Kit Web Reutilizable 40+ Componentes</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="scroll-progress" id="scrollProgress"></div>

  <header class="glass-navbar" id="topbar">
    <a href="#inicio" class="brand" data-section="inicio" aria-label="Ir al inicio">
      <span class="brand__mark">CW</span>
      <span class="brand__text">Componentes Web</span>
    </a>

    <nav class="navbar-links" aria-label="Navegacion principal">
      <button class="nav-link is-active" data-section="inicio">Inicio</button>
      <button class="nav-link" data-section="componentes">Componentes</button>
      <button class="nav-link" data-section="formularios">Formularios</button>
      <button class="nav-link" data-section="datos">Datos</button>
      <button class="nav-link" data-section="utilidades">Utilidades</button>

      <div class="dropdown" data-dropdown>
        <button class="nav-link dropdown__button" data-dropdown-button type="button">Mas ▼</button>
        <div class="dropdown__menu" data-dropdown-menu>
          <button data-section="login">Login demo</button>
          <button data-section="storage">Storage</button>
          <button data-section="guia">Guia</button>
        </div>
      </div>
    </nav>

    <div class="navbar-actions">
      <button class="icon-button" id="themeToggle" type="button" aria-label="Cambiar tema"><</button>
      <button class="icon-button" id="openSidebar" type="button" aria-label="Abrir menu lateral">≡</button>
    </div>
  </header>

  <aside class="sidebar" id="sidebar" aria-label="Menu lateral">
    <div class="sidebar__header">
      <strong>Menu lateral</strong>
      <button class="icon-button" id="closeSidebar" type="button" aria-label="Cerrar menu lateral">✕</button>
    </div>

    <div class="sidebar__content">
      <button class="side-link is-active" data-section="inicio">Panel principal</button>

      <div class="side-group">
        <button class="side-group__button" type="button">Interfaz ▼</button>
        <div class="side-group__content">
          <button data-section="componentes">Componentes UI</button>
          <button data-section="formularios">Formularios</button>
          <button data-section="utilidades">Convertor dinamico</button>
        </div>
      </div>
    </div>
  </aside>
</body>
</html>
```

```

</div>

<div class="side-group">
  <button class="side-group__button" type="button">Datos y estado ▾</button>
  <div class="side-group__content">
    <button data-section="datos">Catalogos y tablas</button>
    <button data-section="storage">LocalStorage</button>
    <button data-section="login">Login demo</button>
  </div>
</div>

  <button class="side-link" data-section="guia">Guia de implementacion</button>
</div>
</aside>

<div class="overlay" id="overlay"></div>

<main class="page-shell">
  <section class="view-section is-active" id="inicio" data-view-section>
    <div class="hero glass-panel reveal">
      <span class="eyebrow">HTML + CSS + JavaScript</span>
      <h1>Proyecto unificado con mas de 40 funcionalidades reutilizables</h1>
      <p>
        Este proyecto funciona como banco de componentes para copiar en tus tareas y paginas web:
        navbar fija con blur, sidebar, login, formularios, cards, tablas, select bonito, tema
        oscuro,
        conversor dinamico, modales, toasts, localStorage y mucho mas.
      </p>
      <div class="hero-actions">
        <button class="primary-button" data-section="componentes">Ver componentes</button>
        <button class="secondary-button" data-section="guia">Como implementarlo</button>
      </div>
    </div>

    <div class="stats-grid reveal">
      <article class="stat-card glass-panel">
        <strong data-counter data-target="52">0</strong>
        <span>funcionalidades incluidas</span>
      </article>
      <article class="stat-card glass-panel">
        <strong>2</strong>
        <span>versiones JS: modular y normal</span>
      </article>
      <article class="stat-card glass-panel">
        <strong>1</strong>
        <span>pagina demo unificada</span>
      </article>
    </div>

    <div class="feature-grid reveal" id="featureList"></div>
  </section>

  <section class="view-section" id="login" data-view-section>
    <div class="section-heading">
      <span class="eyebrow">Login reutilizable</span>
      <h2>Login demo con validacion, mostrar contraseña y sesion local</h2>
      <p>Es un ejemplo frontend. No reemplaza autenticacion real con backend.</p>
    </div>

    <div class="two-column">
      <form class="glass-panel form-card" id="loginForm" novalidate>
        <label for="loginEmail">Correo</label>
        <input id="loginEmail" name="email" type="email" placeholder="usuario@correo.com" required>
        <small class="field-error" data-error="loginEmail"></small>

        <label for="loginPassword">Contraseña</label>

```

```

        <div class="input-action">
            <input id="loginPassword" name="password" type="password" placeholder="Mínimo 6
caracteres" required minlength="6">
            <button type="button" data-toggle-password="loginPassword">Ver</button>
        </div>
        <small class="field-error" data-error="loginPassword"></small>

        <label class="check-line">
            <input type="checkbox" id="rememberUser">
            Recordar usuario en este navegador
        </label>

        <button class="primary-button" type="submit">Ingresar</button>
        <button class="secondary-button" id="logoutButton" type="button">Cerrar sesión</button>
    </form>

    <article class="glass-panel">
        <h3>Estado de sesión</h3>
        <p id="sessionStatus">Aun no hay sesión iniciada.</p>
        <div class="badge-row">
            <span class="badge">Validación email</span>
            <span class="badge">Contraseña toggle</span>
            <span class="badge">localStorage</span>
        </div>
    </article>
</div>
</section>

<section class="view-section" id="componentes" data-view-section>
    <div class="section-heading">
        <span class="eyebrow">Componentes UI</span>
        <h2>Componentes visuales reutilizables</h2>
        <p>Select personalizado, tabs, acordeón, modal, toast, dropdown, carrusel, tooltip y copiar
texto.</p>
    </div>

    <div class="component-grid">
        <article class="glass-panel">
            <h3>Selector personalizado editable</h3>
            <p>Para agregar opciones, solo copia otro <code>li</code> con <code>data-value</code>.</p>
            <div class="custom-select" data-custom-select>
                <input type="hidden" id="selectProducto" value="apa">
                <button type="button" class="custom-select__button">
                    <span class="custom-select__value">APA artesanal</span>
                    <span class="custom-select__arrow">v</span>
                </button>
                <ul class="custom-select__options">
                    <li class="custom-select__option is-selected" data-value="apa">APA artesanal</li>
                    <li class="custom-select__option" data-value="stout">Stout premium</li>
                    <li class="custom-select__option" data-value="sour">Sour de fruta</li>
                    <li class="custom-select__option" data-value="barley">Barley Wine</li>
                </ul>
            </div>
            <p class="muted">Valor seleccionado: <strong id="selectedProductText">apa</strong></p>
        </article>

        <article class="glass-panel">
            <h3>Tabs</h3>
            <div class="tabs" data-tabs>
                <div class="tabs__buttons">
                    <button class="is-active" data-tab="tab1">HTML</button>
                    <button data-tab="tab2">CSS</button>
                    <button data-tab="tab3">JS</button>
                </div>
                <div class="tab-panel is-active" id="tab1">HTML estructura el contenido.</div>
                <div class="tab-panel" id="tab2">CSS diseño colores, tamaños, layout y responsive.</div>
            </div>
        </article>
    </div>

```

```

        <div class="tab-panel" id="tab3">JS agrega interaccion y logica.</div>
    </div>
</article>

<article class="glass-panel">
    <h3>Acordeon FAQ</h3>
    <div class="accordion" data-accordion>
        <button class="accordion__button">¿Para que sirve un componente?</button>
        <div class="accordion__content">Para reutilizar una solucion en varias paginas sin
empezar desde cero.</div>
        <button class="accordion__button">¿Puedo cambiar colores?</button>
        <div class="accordion__content">Si. Cambia las variables CSS dentro de <code>:root</
code>.</div>
        <button class="accordion__button">¿Funciona con JS modular?</button>
        <div class="accordion__content">Si. El archivo principal importa clases desde
<code>funciones.js</code>.</div>
    </div>
</article>

<article class="glass-panel">
    <h3>Modal y toast</h3>
    <button class="primary-button" data-open-modal="demoModal">Abrir modal</button>
    <button class="secondary-button" data-toast="Este es un toast reutilizable">Mostrar toast</
button>
</article>

<article class="glass-panel">
    <h3>Carrusel</h3>
    <div class="carousel" data-carousel>
        <div class="carousel__track">
            <div class="carousel__slide is-active">Slide 1 - Hero</div>
            <div class="carousel__slide">Slide 2 - Producto</div>
            <div class="carousel__slide">Slide 3 - Contacto</div>
        </div>
        <div class="carousel__actions">
            <button data-carousel-prev>Anterior</button>
            <button data-carousel-next>Siguiente</button>
        </div>
    </div>
</article>

<article class="glass-panel">
    <h3>Copiar, tooltip y badge</h3>
    <p>Codigo promocional: <code id="couponCode">WEB-2026</code></p>
    <button class="secondary-button" data-copy="#couponCode">Copiar codigo</button>
    <button class="icon-button tooltip" data-tooltip="Texto de ayuda que aparece al pasar el
mouse">?</button>
    <span class="notification-badge" id="notificationBadge">3</span>
</article>
</div>
</section>

<section class="view-section" id="formularios" data-view-section>
    <div class="section-heading">
        <span class="eyebrow">Formularios</span>
        <h2>Validacion, contador, rango, archivo y autoguardado</h2>
    </div>

    <div class="two-column">
        <form class="glass-panel form-card" id="contactForm" novalidate>
            <label for="contactName">Nombre</label>
            <input id="contactName" name="nombre" type="text" placeholder="Tu nombre" required
minlength="3">
            <small class="field-error" data-error="contactName"></small>

            <label for="contactEmail">Correo</label>

```

```

required>      <input id="contactEmail" name="email" type="email" placeholder="correo@ejemplo.com"

                <small class="field-error" data-error="contactEmail"></small>

                <label for="messageText">Mensaje</label>
textarea>      <textarea id="messageText" rows="4" maxlength="160" placeholder="Escribe un mensaje"></

                <small><span id="charCounter">0</span>/160 caracteres</small>

                <label for="rangeInput">Nivel de interes</label>
                <input id="rangeInput" type="range" min="0" max="100" value="50">
                <p>Valor: <strong id="rangeValue">50</strong></p>

                <label for="fileInput">Archivo</label>
                <input id="fileInput" type="file">
                <p id="fileInfo" class="muted">No hay archivo seleccionado.</p>

                <button class="primary-button" type="submit">Enviar formulario</button>
                <button class="secondary-button" id="clearDraft" type="button">Limpiar borrador</button>
</form>

<article class="glass-panel">
  <h3>Stepper de progreso</h3>
  <div class="stepper" data-stepper>
    <div class="stepper__steps">
      <span class="is-active">1</span>
      <span>2</span>
      <span>3</span>
    </div>
    <p class="stepper__text">Paso 1: Datos personales</p>
    <button class="secondary-button" data-step-prev>Anterior</button>
    <button class="primary-button" data-step-next>Siguiente</button>
  </div>

  <h3>Skeleton loader</h3>
  <button class="secondary-button" id="loadSkeleton">Simular carga</button>
  <div id="skeletonArea"></div>
</article>
</div>
</section>

<section class="view-section" id="datos" data-view-section>
  <div class="section-heading">
    <span class="eyebrow">Datos dinamicos</span>
    <h2>Cards, filtros, busqueda, ordenamiento, paginacion y tabla ordenable</h2>
  </div>

  <div class="glass-panel data-toolbar">
    <input id="catalogSearch" type="search" placeholder="Buscar producto...">
    <select id="catalogSort">
      <option value="name">Ordenar por nombre</option>
      <option value="price">Ordenar por precio</option>
      <option value="rating">Ordenar por calificacion</option>
    </select>
    <div class="chips" id="catalogChips">
      <button class="chip is-active" data-category="all">Todos</button>
      <button class="chip" data-category="cerveza">Cerveza</button>
      <button class="chip" data-category="comida">Comida</button>
      <button class="chip" data-category="postre">Postre</button>
    </div>
  </div>

  <div class="card-grid" id="catalogGrid"></div>
  <div class="pagination" id="catalogPagination"></div>

  <div class="glass-panel table-wrap">

```

```

<input id="tableSearch" type="search" placeholder="Filtrar tabla...">
<table class="data-table" id="demoTable">
  <thead>
    <tr>
      <th data-sort-table="name">Nombre</th>
      <th data-sort-table="role">Rol</th>
      <th data-sort-table="score">Puntaje</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Carlos</td><td>Frontend</td><td>92</td></tr>
    <tr><td>Laura</td><td>UX</td><td>88</td></tr>
    <tr><td>Valeria</td><td>Backend</td><td>95</td></tr>
    <tr><td>Juan</td><td>QA</td><td>84</td></tr>
  </tbody>
</table>
</div>
</section>

<section class="view-section" id="storage" data-view-section>
  <div class="section-heading">
    <span class="eyebrow">Estado local</span>
    <h2>Notas, tareas, autoguardado, exportar JSON y limpiar datos</h2>
  </div>

  <div class="two-column">
    <article class="glass-panel">
      <h3>Notas con localStorage</h3>
      <textarea id="noteInput" rows="4" placeholder="Escribe una nota..."></textarea>
      <button class="primary-button" id="saveNote">Guardar nota</button>
      <button class="secondary-button" id="exportNotes">Exportar notas JSON</button>
      <ul class="item-list" id="noteList"></ul>
    </article>

    <article class="glass-panel">
      <h3>Lista de tareas</h3>
      <div class="input-action">
        <input id="todoInput" placeholder="Nueva tarea">
        <button id="addTodo" type="button">Agregar</button>
      </div>
      <ul class="item-list draggable-list" id="todoList"></ul>
      <button class="danger-button" id="clearTodos">Borrar tareas</button>
    </article>
  </div>
</section>

<section class="view-section" id="utilidades" data-view-section>
  <div class="section-heading">
    <span class="eyebrow">Ejemplo pedido</span>
    <h2>Convertor con 3 botones: C, F y K</h2>
    <p>Cada boton cambia la unidad seleccionada, el fondo y el resultado calculado.</p>
  </div>

  <div class="glass-panel temperature-tool" id="temperatureTool">
    <div class="unit-buttons">
      <button class="unit-button is-active" data-unit="celsius">C</button>
      <button class="unit-button" data-unit="fahrenheit">F</button>
      <button class="unit-button" data-unit="kelvin">K</button>
    </div>

    <label for="temperatureInput">Temperatura</label>
    <input id="temperatureInput" type="number" placeholder="Ingresa un valor">
    <button class="primary-button" id="calculateTemperature">Calcular</button>
    <p id="temperatureMessage" class="muted"></p>
    <div class="result-box" id="temperatureResult">Selecciona una unidad y calcula.</div>
  </div>

```

```

        <div class="glass-panel utilities-row">
            <button class="secondary-button" id="printPage">Imprimir pagina</button>
            <button class="secondary-button" id="resetDemo">Reiniciar demo visual</button>
            <button class="secondary-button" data-toast="Accion ejecutada correctamente">Probar accion
rapida</button>
        </div>
    </section>

    <section class="view-section" id="guia" data-view-section>
        <div class="section-heading">
            <span class="eyebrow">Guia de implementacion</span>
            <h2>Que cambiar para reutilizar este proyecto</h2>
        </div>

        <div class="glass-panel guide-content">
            <h3>1. Cambios principales en HTML</h3>
            <p>Cambia textos, ids y atributos <code>data-section</code>. Si agregas una nueva seccion, crea
un boton con el mismo valor en <code>data-section</code>.</p>
            <pre><code>&lt;button data-section="servicios"&gt;Servicios&lt;/button&gt;
&lt;section id="servicios" data-view-section&gt;...&lt;/section&gt;</code></pre>

            <h3>2. Cambios principales en CSS</h3>
            <p>Modifica las variables dentro de <code>:root</code> para cambiar colores, radios, sombras y
tamaño de navbar.</p>
            <pre><code>:root {
--primary: #2563eb;
--secondary: #7c3aed;
--radius-lg: 24px;
}</code></pre>

            <h3>3. Cambios principales en JS modular</h3>
            <p>Agrega clases en <code>funciones.js</code> usando <code>export</code> y llamalas desde
<code>script.js</code> con <code>import</code>.</p>
            <pre><code>import { CustomSelect } from "../funciones.js";
new CustomSelect("[data-custom-select]");</code></pre>

            <h3>4. Version normal sin modulos</h3>
            <p>Usa <code>script-normal.js</code> quitando <code>type="module"</code> si tu profesor no
quiere modulos.</p>
            <pre><code>&lt;script src="script-normal.js"&gt;&lt;/script&gt;</code></pre>
        </div>
    </section>
</main>

<button class="back-to-top" id="backToTop" type="button" aria-label="Volver arriba">↑</button>

<dialog class="modal" id="demoModal">
    <div class="modal__content">
        <button class="modal__close" data-close-modal type="button">x</button>
        <h3>Modal reutilizable</h3>
        <p>Este modal se puede usar para alertas, confirmaciones, detalles de producto o formularios
cortos.</p>
        <button class="primary-button" data-close-modal type="button">Entendido</button>
    </div>
</dialog>

<div class="toast-container" id="toastContainer"></div>

<script type="module" src="script.js"></script>
<!-- Version normal sin modulos: comenta la linea anterior y descomenta esta si la necesitas. -->
<!-- <script src="script-normal.js"></script> -->
</body>
</html>

```

Anexo E - Validación automática

#	Funcionalidad	CSS	JS	HTML
1	Navbar fija glass	OK	OK	OK
2	Sidebar ocultable	OK	OK	OK
3	Overlay para cerrar menú lateral	OK	OK	OK
4	Submenús desplegables dentro del sidebar	OK	OK	OK
5	Cambio de secciones del body con data-section	OK	OK	OK
6	Link activo en navbar y sidebar	OK	OK	OK
7	Dropdown superior	OK	OK	OK
8	Tema claro/oscuro	OK	OK	OK
9	Guardar tema en localStorage	OK	OK	OK
10	Barra de progreso de scroll	OK	OK	OK
11	Botón volver arriba	OK	OK	OK
12	Animación reveal on scroll	OK	OK	OK
13	Contador animado	OK	OK	OK
14	Login demo	OK	OK	OK
15	Logout demo	OK	OK	OK
16	Recordar usuario	OK	OK	OK
17	Mostrar/ocultar contraseña	OK	OK	OK
18	Validación de formulario	OK	OK	OK
19	Mensajes de error por campo	OK	OK	OK
20	Contador de caracteres	OK	OK	OK
21	Range slider con valor visible	OK	OK	OK
22	Información de archivo seleccionado	OK	OK	OK
23	Autoguardado de borrador	OK	OK	OK
24	Stepper de pasos	OK	OK	OK
25	Skeleton loader	OK	OK	OK
26	Select personalizado editable	OK	OK	OK
27	Tabs	OK	OK	OK
28	Acordeón	OK	OK	OK
29	Modal	OK	OK	OK
30	Toast notifications	OK	OK	OK
31	Carrusel	OK	OK	OK
32	Tooltip	OK	OK	OK
33	Badge de notificación	OK	OK	OK

34	Copiar al portapapeles	OK	OK	OK
35	Cards dinámicas desde array	OK	OK	OK
36	Búsqueda con debounce	OK	OK	OK
37	Filtros con chips	OK	OK	OK
38	Ordenamiento de cards	OK	OK	OK
39	Paginación de cards	OK	OK	OK
40	Tabla ordenable	OK	OK	OK
41	Filtro de tabla	OK	OK	OK
42	Notas con localStorage	OK	OK	OK
43	Exportar notas a JSON	OK	OK	OK
44	Lista de tareas	OK	OK	OK
45	Drag and drop en tareas	OK	OK	OK
46	Borrar tareas	OK	OK	OK
47	Conversor con botones C/F/K	OK	OK	OK
48	Cambio de fondo según unidad	OK	OK	OK
49	Cálculo dinámico de temperatura	OK	OK	OK
50	Imprimir página	OK	OK	OK
51	Reset visual de demo	OK	OK	OK
52	Helpers reutilizables para DOM, storage y debounce	OK	OK	OK
53	Tarjeta de presentación Bootstrap como actividad guiada	OK	OK	OK
54	Carrito de compras DOM como actividad guiada	OK	OK	OK
55	Footer reutilizable como pie de página profesional	OK	OK	OK
56	Tarjetas de precios Bootstrap como actividad guiada completa	OK	OK	OK
57	Barra de desplazamiento personalizada	OK	OK	OK